

UNIVERSIDADE DE SÃO PAULO

Instituto de Ciências Matemáticas e de Computação

**Navigating the Memory Wall: A Design Space
Exploration of Bandwidth and Latency Constraints in
SIMD Neural Accelerators**

Artur Brenner Weber

Bachelor's Thesis submitted to the Undergraduate Program for the
Engineer's Degree in Computer Engineering

Artur Brenner Weber

**Navigating the Memory Wall: A Design Space Exploration of
Bandwidth and Latency Constraints in SIMD Neural
Accelerators**

Bachelor's thesis submitted to the Undergraduate Office of the Instituto de Ciências Matemáticas e de Computação and the Escola de Engenharia de São Carlos, Universidade de São Paulo, as part of the requirements to obtain the Engineer's Degree in Computer Engineering.

Concentration area: Computer Engineering

Advisor: Prof. Dr. Vanderlei Bonato

Original version

São Carlos

2026

I AUTHORIZE THE REPRODUCTION AND DISSEMINATION OF TOTAL OR PARTIAL COPIES OF THIS DOCUMENT, BY CONVENTIONAL OR ELECTRONIC MEDIA FOR STUDY OR RESEARCH PURPOSES, PROVIDED THE SOURCE IS CITED.

B373n

Brenner Weber, Artur

Navigating the Memory Wall: A Design Space Exploration of Bandwidth and Latency Constraints in SIMD Neural Accelerators / Artur Brenner Weber ; advisor Vanderlei Bonato. – São Carlos, 2026.

114

p.

Undergraduate Thesis in Computer Engineering – São Carlos School of Engineering of University of São Paulo, 2026.

1. Design Space Exploration. 2. Matrix-Vector Unit (MVU). 3. Extended Roofline Model. 4. Energy-Delay Product (EDP). 5. Edge Neural Accelerator. I. Bonato, Vanderlei, advisor. II. Title.

Artur Brenner Weber

Contornando o Gargalo de Memória: Uma Exploração do Espaço de Projeto das Restrições de Largura de Banda e Latência em Aceleradores Neurais SIMD

Trabalho de conclusão de curso apresentado ao Programa de Graduação do Instituto de Ciências Matemáticas e de Computação e Escola de Engenharia de São Carlos, Universidade de São Paulo - ICMC - EESC/USP, como parte dos requisitos para obtenção do título de Engenheiro de Computação.

Área de concentração: Engenharia de Computação

Orientador: Prof. Dr. Vanderlei Bonato

Versão original

São Carlos

2026

*This work is dedicated to my family, Kennia, Marcelo, and Alexandre,
as well as to all my friends from Studas for always supporting me throughout my studies.*

ACKNOWLEDGEMENTS

I extend my eternal gratitude to my family and friends, especially the members of the "Studas" group for their sustained support throughout my academic and personal journey.

I am deeply indebted to my advisor, Prof. Dr. Vanderlei Bonato, for his mentorship. His continuous availability for technical discussion and his strategic direction and precise feedbacks were fundamental to the execution of this research.

I acknowledge MSc. Eduardo Sperle Honorato for his technical guidance and his contributions to the initial architectural draft evaluated in this study.

Lastly, I recognize the Universidade Federal do Rio Grande do Sul (UFRGS) for providing access to the CADmicro infrastructure and the requisite electronic design automation tools.

During the preparation of this manuscript, the author utilized Gemini 3.1 Pro for proof-reading, stylistic refinement, and grammatical correction, as well as LaTeX formatting correction. Additionally, NotebookLM was employed as a secondary verification tool to cross-reference bibliography citations against the source material. Following the use of these assistive technologies, the author thoroughly reviewed and edited the manuscript, assuming full and sole responsibility for the final content, originality, and technical accuracy of this work.

I also thank Henrique Passos Miranda for helping me review the text from an external perspective.

“Eu não sou senhor do tempo, mas eu sei que vai chover.”

Charlie Brown Jr.

Abstract

WEBER, A. B. **Navigating the Memory Wall: A Design Space Exploration of Bandwidth and Latency Constraints in SIMD Neural Accelerators**. 2026. 115 p. Monografia (Trabalho de Conclusão de Curso) - Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo; Escola de Engenharia de São Carlos, Universidade de São Paulo, São Carlos, 2026.

Domain-Specific Architectures deploy parallel arithmetic arrays to accelerate deep neural network inference. The energy penalty of data transfer restricts computational throughput across all hardware environments; edge devices face additional constraints imposed by finite thermal boundaries, battery capacities, and financial exclusion from High-Bandwidth Memory. Expanding computational logic in embedded accelerators without commensurate memory bandwidth produces execution stalls, establishing localized data delivery as the primary performance constraint. To address this limitation, this study executes a mixed-level Design Space Exploration of a parameterizable, weight-resident Matrix-Vector Unit utilizing a temporally folded SIMD datapath. The module was implemented at the Register-Transfer Level, verified for functional equivalence against a Python execution model, and synthesized using a 14nm FinFET library to extract operational frequency, silicon area, and power dissipation metrics across 116 geometric configurations. These physical parameters were mapped onto an Extended Roofline Model, and the Energy-Delay Product was calculated to quantify system scalability under variable memory bandwidths. Synthesis metrics indicated that expanding the SIMD width improved computation rates at low dimensions but introduced interconnect congestion that degraded clock frequencies in denser arrays. Expanding the output channel array linearly replicated sequential storage logic, increasing non-combinational silicon area and static leakage power. Although monolithic scale-up architectures masked memory latency under the minimum evaluated bandwidth constraint of 1.0 GB/s through extensive data reuse, their exponential physical resource costs precluded deployment in edge environments. Under a 25.0 GB/s memory constraint, a distributed array of smaller processing nodes surpassed a single contiguous architecture: the distributed topology achieved over 100 Giga-Multiply-Accumulate operations per second while reducing power consumption by 82% and silicon area by 79%. Scale-out architectural topologies exhibited superior energy-delay profiles compared to contiguous scale-up configurations, demonstrating that optimizing edge-deployed accelerators requires constraining localized weight register capacities to maximize physical resource utilization.

Keywords: Design Space Exploration. Matrix-Vector Unit (MVU). Extended Roofline Model. Energy-Delay Product (EDP). RTL Synthesis. Weight-Resident Architecture. Edge Neural Accelerator. Memory Bandwidth Constraints. ASIC Design. Temporal Folding.

RESUMO

WEBER, A. B. **Contornando o Gargalo de Memória: Uma Exploração do Espaço de Projeto das Restrições de Largura de Banda e Latência em Aceleradores Neurais SIMD.** 2026. 115 f. Monografia (Trabalho de Conclusão de Curso) - Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo; Escola de Engenharia de São Carlos, Universidade de São Paulo, São Carlos, 2026.

Arquiteturas de Domínio Específico empregam arranjos aritméticos paralelos para acelerar a inferência de redes neurais profundas. A penalidade energética da transferência de dados restringe o desempenho computacional em todos os ambientes de hardware; dispositivos de borda enfrentam restrições adicionais impostas por limites térmicos finitos, capacidades de bateria e exclusão financeira de memórias de alta largura de banda. A expansão da lógica computacional em aceleradores embarcados sem largura de banda de memória proporcional produz interrupções de execução, estabelecendo a entrega localizada de dados como restrição primária de desempenho. Este trabalho executa uma Exploração do Espaço de Projeto de nível misto de uma Unidade Matriz-Vetor parametrizável e residente em pesos, com caminho de dados SIMD temporalmente dobrado. O módulo foi implementado em nível de transferência de registros, verificado por equivalência funcional frente a um modelo de execução em Python e sintetizado com uma biblioteca FinFET de 14nm para extração de frequência operacional, área de silício e dissipação de potência em 116 configurações geométricas. Os parâmetros físicos obtidos foram mapeados em um Modelo Roofline Estendido, e o Produto Energia-Atraso foi calculado para quantificar a escalabilidade do sistema sob larguras de banda de memória variáveis. Os resultados de síntese indicaram que a expansão da largura SIMD melhorou as taxas de computação em dimensões reduzidas, mas introduziu congestionamento de interconexão que degradou as frequências de clock em arranjos mais densos. A expansão do arranjo de canais de saída replicou linearmente a lógica de armazenamento sequencial, aumentando a área de silício não combinacional e a potência de vazamento estático. Embora arquiteturas monolíticas de escalonamento vertical tenham mascarado a latência de memória sob a restrição mínima avaliada de 1,0 GB/s por meio de extensivo reúso de dados, seus custos exponenciais de recursos físicos inviabilizaram a implantação em ambientes de borda. Sob restrição de memória de 25,0 GB/s, um arranjo distribuído de nós de processamento menores superou uma arquitetura contígua única: a topologia distribuída atingiu mais de 100 Giga-Operações de Multiplicação-Acumulação por segundo com redução de 82% no consumo de potência e 79% na área de silício. Topologias de escalonamento horizontal exibiram perfis de produto energia-atraso superiores em comparação às configurações contíguas de escalonamento vertical, demonstrando que a otimização de aceleradores em borda requer o dimensionamento restrito das capacidades de registros de pesos locais para maximizar a utilização dos recursos físicos.

Palavras-chave: Exploração do Espaço de Projeto. Unidade Matriz-Vetor (MVU). Modelo Roofline Estendido. Produto Energia-Atraso (EDP). Síntese RTL. Arquitetura Residente em Pesos. Acelerador Neural de Borda. Restrições de Largura de Banda de Memória. Projeto ASIC. Dobramento Temporal.

LIST OF FIGURES

Figure 1 – Apple Silicon GPU performance analysis in Blender Open Data	26
Figure 2 – Sliding window mechanism in the computation of a high-dimensional CNN layer	32
Figure 3 – Roofline Model for an AMD Opteron X2.	37
Figure 4 – Roofline models for TPU v4, TPU v3, and A100 plus DNN models. Operational intensity is in parentheses.	38
Figure 5 – Extended Roofline model illustrating secondary performance ceilings constrained by microarchitectural bottlenecks, including cache latency and imperfect overlap.	40
Figure 6 – The dataflow in a logical PE set to process a 2D convolution. (a) rows of filter weight are reused across PEs horizontally. (b) rows of ifmap pixel are reused across PEs diagonally. (c) rows of psum are accumulated across PEs vertically. In this example, $R = 3$ and $H = 5$	42
Figure 7 – Eyeriss v2 top-level architecture illustrating the hierarchical subdivision of GLB, Router, and PE clusters.	42
Figure 8 – Dataflow Architecture with Per-Layer Tailored Compute Arrays, On-Chip Weights and Activations.	44
Figure 9 – Overview of the MVU.	45
Figure 10 – Macroscopic view of a Deep Neural Network of Y layers running on X MVUs.	47
Figure 11 – MVU’s Finite State Machine.	49
Figure 12 – Block diagram of the MVU highlighting input/output interfaces and structural parameters.	50
Figure 13 – Internal Architectural View of an MVU.	54
Figure 14 – Distribution density of maximum synthesized clock frequencies.	67
Figure 15 – Throughput scaling at PE=8 and PAD_LANES=128 (14 input channels).	68
Figure 16 – Throughput scaling at PE=16 and PAD_LANES=128 (14 input channels).	68
Figure 17 – Throughput scaling for high-input asymmetry: PE=16 and PAD_LANES=1024 (113 input channels).	68
Figure 18 – Throughput scaling at PE=32 and PAD_LANES=256 (28 input channels).	69
Figure 19 – Throughput scaling at PE=64 and PAD_LANES=512 (56 input channels).	69
Figure 20 – Density distribution of total synthesized silicon area.	70
Figure 21 – Area partitioning vs. SIMD width at PE=8 and PAD_LANES=128 (14 input channels).	71
Figure 22 – Area partitioning vs. SIMD width at PE=16 and PAD_LANES=128 (14 input channels).	71

Figure 23 – Area partitioning vs. SIMD width at PE=32 and PAD_LANES=256 (28 input channels).	72
Figure 24 – Area partitioning vs. SIMD width at PE=64 and PAD_LANES=512 (56 input channels).	72
Figure 25 – Area scaling vs. output channels at SIMD=16 and PAD_LANES=512 (56 input channels).	73
Figure 26 – Area scaling vs. output channels at SIMD=512 and PAD_LANES=512 (56 input channels).	73
Figure 27 – Density distribution of total synthesized silicon power.	74
Figure 28 – Power scaling vs. SIMD width at PE=8 and PAD_LANES=128 (14 input channels).	75
Figure 29 – Power scaling vs. SIMD width at PE=64 and PAD_LANES=512 (56 input channels).	75
Figure 30 – Power scaling vs. output channels at SIMD=16 and PAD_LANES=512 (56 input channels).	76
Figure 31 – Power scaling vs. output channels at SIMD=512 and PAD_LANES=512 (56 input channels).	76
Figure 32 – Roofline models under an extreme memory constraint (1.0 GB/s bandwidth, 20 ns latency)	81
Figure 33 – Roofline models at the operational inflection point (6.0 GB/s bandwidth, 10 ns latency).	82
Figure 34 – Roofline models under high-bandwidth conditions (25.0 GB/s bandwidth, 1 ns latency).	84

LIST OF TABLES

Table 1 – Per-core performance and memory bandwidth scaling in base Apple Silicon variants	27
Table 2 – Per-core performance and memory bandwidth scaling in M3 Ultra variants	27
Table 3 – Shape parameters of a CNN layer	33
Table 4 – Golden model configuration parameters.	57
Table 5 – Statistical distribution of maximum synthesized frequencies ($F_{\max}$).	66
Table 6 – Statistical distribution of total synthesized area.	70
Table 7 – Statistical distribution of total synthesized power.	74
Table 8 – Highest efficiency configurations for PE=8, categorized by input dimension.	77
Table 9 – Highest efficiency configurations for PE=16, categorized by input dimension.	78
Table 10 – Highest efficiency configurations for PE=32, categorized by input dimension.	78
Table 11 – Highest efficiency configurations for PE=64, categorized by input dimension.	79
Table 12 – Energy-Delay Product (EDP) comparison for a 10^6 MAC workload.	85
Table 13 – Apple Silicon GPU Performance Metrics	95

LIST OF SOURCE CODES

B.1	MVU Logic Implemented in SystemVerilog	97
B.2	Python Golden Model for generating inputs and expected outputs.	103
B.3	Testbench with memory simulation and Golden Model inputs and outputs check.	105
B.4	Synopsys Design Compiler synthesis script.	112
B.5	Synopsys Primetime Static Timing Analysis script.	113

LIST OF ABBREVIATIONS AND ACRONYMS

AI	Artificial Intelligence
ALU	Arithmetic Logic Unit
ANE	Apple Neural Engine
ASIC	Application-Specific Integrated Circuit
BFP	Block Floating Point
BRAM	Block Random-Access Memory
BW	Memory Bandwidth
CMOS	Complementary Metal-Oxide-Semiconductor
CNN	Convolutional Neural Network
CONV	Convolutional
CPU	Central Processing Unit
CTS	Clock Tree Synthesis
DAG	Directed Acyclic Graph
DDR5	Double Data Rate 5
DLRM	Deep Learning Recommendation Model
DNN	Deep Neural Network
DRAM	Dynamic Random-Access Memory
DSA	Domain-Specific Architecture
DSE	Design Space Exploration
EDA	Electronic Design Automation
EDP	Energy-Delay Product
FC	Fully Connected
FF	Fast-Fast (PVT corner)
FSM	Finite State Machine

GLB	Global Buffer
GMAC/s	Giga Multiply-Accumulate operations per second
GOP/s	Giga Operations per Second
GPU	Graphics Processing Unit
HBM	High-Bandwidth Memory
HM-NoC	Hierarchical Mesh Network-on-Chip
ifmap	Input Feature Map
IS	Input-Stationary
LLM	Large Language Model
LSTM	Long Short-Term Memory
LUT	Look-Up Table
MAC	Multiply-Accumulate
MLP	Multi-Layer Perceptron
MVU	Matrix-Vector Unit
NMOS	N-type Metal-Oxide-Semiconductor
NN	Neural Network
NoC	Network-on-Chip
NPU	Neural Processing Unit
ofmap	Output Feature Map
OI	Operational Intensity
OoO	Out-of-Order
OS	Output-Stationary
PDK	Process Design Kit
PE	Processing Element
Perf.	Performance Metrics
PMD	Personal Mobile Device

PMOS	P-type Metal-Oxide-Semiconductor
PRAM	Parallel Random-Access Machine
psum	Partial Sum
PVT	Process, Voltage, and Temperature
ReLU	Rectified Linear Unit
RF	Register File
RGB	Red, Green, and Blue
RNN	Recurrent Neural Network
ROB	Reorder Buffer
RS	Reservation Station / Row-Stationary
RTL	Register-Transfer Level
SAIF	Switching Activity Interchange Format
SDC	Synopsys Design Constraints
SIMD	Single Instruction, Multiple Data
SoC	System-on-Chip
SPad	Scratchpad
SRAM	Static Random-Access Memory
SS	Slow-Slow (PVT corner)
STA	Static Timing Analysis
TBP	Total Board Power
TPU	Tensor Processing Unit
TSV	Through-Silicon Via
VCD	Value Change Dump
VCS	Verilog Compiler Simulator
VLSI	Very Large-Scale Integration
WNS	Worst Negative Slack
WS	Weight-Stationary

LIST OF SYMBOLS

acc	Accumulator register value
b	Bias scalar
β_{L1}	Peak memory bandwidth for L1 cache
β_{L2}	Peak memory bandwidth for L2 cache
BW	Peak memory bandwidth
c	Precision rounding constant
C	Capacitance / Number of input feature map channels
C_{comp}	Active computation time in cycles
C_{idle}	Stall cycles induced by memory transfer delays
C_{mem}	Memory transfer time in cycles
C_{total}	Total processing latency per inference
D	Delay (total seconds required to process workload)
E	Spatial height of output feature map / Energy (Joules)
F	Spatial width of output feature map
F_{clk}	Clock frequency in Hertz
F_{max}	Maximum operational frequency
H	Spatial height of input feature map
I	Operational intensity
k	Temporal fold index
L	Initial memory access latency
M	Number of 3D filters / Quantization scaling multiplier / Precision parameter
M_{int}	Integer requantization multiplier
N	Batch size / Precision width parameter
P	Parallel processing elements / Output channel index

P_{peak}	Peak computational throughput
π	Peak computational throughput (Extended Roofline Model)
q	Quantized 8-bit integer value
R	Spatial height of filter
r	Real (floating-point) activation or weight value
S	Spatial width of filter / Quantization scale factor / SIMD fan-in / Data volume
s	Per-layer requantization scale
s_{in}	Input activation scale factor
s_{out}	Output tensor scale factor
s_w	Weight matrix scale factor
T_{comp}	Compute time
T_{mem}	Memory access time
U	Filter stride
V_{DD}	Supply voltage
W	Spatial width of input feature map / Weight matrix
w	Weight vector
x	Input activation vector
y	Output vector
y_{temp}	Intermediate requantization value
Z	Quantization zero-point offset
z	Output of artificial neuron

CONTENTS

	List of Figures	11
	List of Tables	13
	Contents	20
1	INTRODUCTION	22
1.1	Contextualization and Motivation	22
1.1.1	Architectural Scaling Bounds in Consumer Edge SoCs	25
1.2	Objectives	28
1.2.1	General Objective	28
1.2.2	Specific Objectives	29
1.3	Document Structure	29
2	LITERATURE REVIEW	31
2.1	Initial Considerations	31
2.2	Theoretical Background	31
2.2.1	Fundamentals of Deep Neural Networks	31
2.2.2	Edge DNN Hardware Architectures	33
2.2.3	Analytical Performance Models: EDP and Roofline	35
2.3	Related Work	40
2.3.1	Spatial Architectures and Dataflows	40
2.3.2	Streaming Architectures and Weight Residency	43
2.4	Final Considerations	45
3	DEVELOPMENT	46
3.1	Initial Considerations	46
3.2	The Parameterizable MVU: Architecture and Evaluation Methodology .	46
3.3	Implementation	48
3.3.1	Development Environment and Toolchain	48
3.3.2	RTL Implementation of the MVU Datapath	48
3.3.3	Functional Verification via Python Golden Model	56
3.3.4	Memory Constraint Modeling	59
3.3.5	Synthesis and Static Timing Analysis	62
3.4	Results	64
3.4.1	Functional Simulation Validation	64
3.4.2	Logic Synthesis	64

3.4.3	Static Timing Analysis	65
3.4.4	Maximum Clock Frequency and Performance	65
3.4.5	Silicon Area Analysis	69
3.4.5.1	SIMD Scaling versus Logic Partitioning	70
3.4.5.2	Storage Dominance versus Output Channel Scaling	72
3.4.6	Power Dissipation	73
3.4.6.1	SIMD Scaling and Static Dominance	74
3.4.6.2	Output Channel Scaling	75
3.4.7	Power and Area Efficiency Ranking	76
3.4.7.1	Efficiency Analysis: 8-Output-Channel Configurations	77
3.4.7.2	Efficiency Analysis: 16-Output-Channel Configurations	77
3.4.7.3	Efficiency Analysis: 32 and 64-Output-Channel Configurations	78
3.4.8	System-Level Evaluation: The Extended Roofline Model	79
3.4.8.1	Case Study 1: 1.0 GB/s	80
3.4.8.2	Case Study 2: 6.0 GB/s	81
3.4.8.3	Case Study 3: 25.0 GB/s	82
3.4.9	Energy-Delay Product Assessment	85
3.5	Methodological Limitations	86
3.5.1	Architectural Constraints	86
3.5.2	Synthesis Methodology Constraints	86
3.5.3	Infrastructural Instability	87
3.6	Final Considerations	87
4	CONCLUSION	88
4.1	Challenges	88
4.2	Future Work	88
	REFERENCES	90
	APPENDIX	94
	APPENDIX A – APPLE SILICON GPU PERFORMANCE IN BLENDER OPEN DATA BENCHMARK	95
	APPENDIX B – CODE	97

1 INTRODUCTION

1.1 Contextualization and Motivation

Neural network (NN) applications are expanding rapidly across computing. Advances in NN algorithms enable more accurate results across a wide range of areas such as computer vision (Krizhevsky; Sutskever; Hinton, 2017 *apud* Sze *et al.*, 2020), speech recognition (Deng *et al.*, 2013 *apud* Sze *et al.*, 2020), and robotics. Beyond these foundational tasks, deep learning models have revolutionized natural language processing tasks like machine translation (Russell; Norvig, 2021), powered sophisticated recommender systems for online services and advertising (Goodfellow; Bengio; Courville, 2016), and enabled the development of autonomous self-driving vehicles (Chen *et al.*, 2015 *apud* Sze *et al.*, 2020). Today, approximately one-third of all computer interaction worldwide is performed via voice rather than a traditional keyboard, a feat driven largely by neural network models (Russell; Norvig, 2021). Neural networks are also driving critical healthcare breakthroughs, achieving expert-level accuracy in medical diagnoses (Russell; Norvig, 2021).

NN models demand substantial computational resources (Sze *et al.*, 2020). Commercial cloud environments provision computational infrastructure based on prevailing neural network architectures. During July 2016, Google’s inference workloads comprised 61% Multi-Layer Perceptrons (MLPs) and 29% Recurrent Neural Networks (RNNs) (Jouppi *et al.*, 2017). Production workloads adapt continuously to integrate algorithmic advancements. Thus, the development of the attention mechanism and subsequent Transformer-based architectures, such as Large Language Models (LLMs), altered this baseline distribution (Jouppi *et al.*, 2023). By October 2022, Transformers accounted for 57% of Google’s Tensor Processing Unit (TPU) training capacity, reducing RNN usage to 2%, whereas MLPs and Deep Learning Recommendation Models (DLRMs) retained a 24% allocation (Jouppi *et al.*, 2023). These architectures depend extensively on Multiply-Accumulate operations (MAC), which involve multiplying large sets of operands and accumulating their partial sums. MAC operations form the primary computational bottleneck of both convolutional and fully connected (FC) layers (Sze *et al.*, 2020), making them the most demanding operations required for deep learning workloads. NNs also require substantial memory capacity and, especially, bandwidth (BW). CNNs, even a decade ago, required billions of operations and tens to hundreds of megabytes of parameter storage for a single inference pass (Chen *et al.*, 2017). Historically, neural network parameter counts have doubled approximately every 2.4 years (Goodfellow; Bengio; Courville, 2016). Today, this growth trajectory has culminated in Transformer-based LLMs, such as PaLM, which comprise up to 540 billion parameters (Jouppi *et al.*, 2023). At the current frontier, specialized training systems operate at exascale throughput (on the order of 10^{18} operations per second) (Russell; Norvig, 2021); for example, TPU v4 deployments report peak performance above 1 exaflop and

sustained utilization over multi-week training runs (Jouppi *et al.*, 2023).

The high computational cost of neural networks generates critical energy and thermal limitations. As these demanding applications emerge alongside the breakdown of physical hardware scaling laws, a fundamental shift in hardware design is required (Patterson; Hennessy, 2020). In the mid-2000s, energy-efficient hardware design relied on adding more cores for parallel processing to general-purpose processors, paired with advanced clock-frequency control, to boost performance within power budgets (Horowitz, 2014). However, this approach was insufficient to meet emerging neural network constraints, such as in highly constrained mobile devices (Sze *et al.*, 2020) and warehouse-scale datacenters (Jouppi *et al.*, 2017). This architectural insufficiency occurred primarily because a general-purpose instruction typically incurs tens of pJ of control and data-movement energy, while the corresponding compute primitive is only a few pJ (Horowitz, 2014). This is because fetching instructions and clocking state registers consumes substantially more power than executing the actual mathematical operations themselves. Furthermore, the decline of Moore’s Law and Dennard scaling, combined with the parallel processing bottlenecks defined by Amdahl’s Law, severely restricted any further performance gains from traditional processor improvements (Patterson; Hennessy, 2020). Since these traditional scaling methods could no longer satisfy strict power constraints, a new paradigm emerged through specialized hardware design. Domain-Specific Architectures (DSA), or hardware accelerators, overcome these historical inefficiencies by sacrificing broad programmability to execute a highly targeted set of tasks with maximum efficiency and speed (Patterson; Hennessy, 2020). Examples of NN DSAs include Google’s TPU (Jouppi *et al.*, 2017), Apple’s Neural Engine (ANE) and NVIDIA’s Tensor Core Graphics Processing Units (GPU).

Although the widespread adoption of DSAs significantly improved deep learning efficiency, the rapidly expanding computational demand of modern artificial intelligence has precipitated a severe global energy crisis. At the macro level, this crisis is evidenced by warehouse-scale data centers, where cutting-edge AI server racks now draw up to 140 kW—significantly exceeding traditional enterprise deployments—and projected aggregate energy demands threaten to outpace national power infrastructures (NVIDIA, 2024; Fransoo; Peels; Udenio, 2026). However, while data centers can combat this unprecedented demand with megawatt power grids and elaborate active cooling mechanisms, edge devices face a far more restrictive microarchitectural crisis. Deeply embedded systems and Personal Mobile Devices (PMDs) operate under finite battery capacities (Patterson; Hennessy, 2020) and strict passive ambient cooling limits, mandating thermal design points well below twenty degrees above room temperature to ensure user safety and prevent hardware degradation (Seshan; Schepis, 2018).

Given the unsustainable power trajectory of these workloads, as well as edge limitations, energy dissipation at the chip level must be minimized. Within neural network accelerators, the core energy bottleneck is not the arithmetic computation itself, but rather the memory subsystem. Crucially, data movement dominates the overall hardware energy budget, amplifying the data-

fetching inefficiencies established previously. For example, in a 45 nm process, accessing off-chip Dynamic Random-Access Memory (DRAM) can consume 640 pJ per 32-bit read, which is over one hundred times more energy-intensive than an on-chip Static Random-Access Memory (SRAM) read (5 pJ) and thousands of times more power-consuming than the 8-bit arithmetic multiplication common in neural accelerators (0.2 pJ) (Horowitz, 2014 *apud* Sze *et al.*, 2020). Moreover, even when designers mitigate DRAM costs by bringing memory on-chip, the static power dissipation remains substantial; the leakage power of a modern last-level cache is now larger than the entire active power draw of a simple processor core running at full capacity (Horowitz, 2014). Consequently, Application-Specific Integrated Circuit (ASIC) designers are forced to navigate complex architectural trade-offs, as attempting to implement lower-power memory solutions—such as reducing the supply voltage or limiting cache sizes—can severely compromise cell read stability, reduce data throughput, and ultimately force the system to increase reliance on the prohibitively expensive off-chip DRAM (Weste; Harris, 2010). Thus, the massive energy penalty of fetching weights and activations dictates the absolute thermal and energy limits of the entire chip.

Beyond its profound impact on the system’s energy budget, the memory subsystem also acts as the principal limiter of overall performance. Because physical factors restrict the speed at which data can be shuttled between memory and processing elements (PEs)—a phenomenon known as the "memory wall"—the vast majority of neural network applications are severely memory-bound (Sze *et al.*, 2020). For example, an analysis of Google’s datacenter workloads in 2015 revealed that MLPs and LSTM networks, which collectively represented 90% of their deployed neural network demand, were strictly limited by memory bandwidth rather than the peak computational capacity (Jouppi *et al.*, 2017).

Consequently, hardware designers must meticulously analyze the complex trade-offs among memory bandwidth, area footprint, access latency, and power dissipation. For instance, attempting to achieve higher performance by increasing the capacity and bandwidth of on-chip SRAMs reduces off-chip latency, but this comes at the steep cost of dedicating large portions of the silicon area to storage rather than active computation, which significantly escalates static leakage power (Weste; Harris, 2010) and reduces the number of processing elements that can fit on the die (Sze *et al.*, 2020). As a result, modern domain-specific architectures are focused heavily on data-movement orchestration; ASIC designers must optimize the flow and sizing of data transfers as they move from slow, off-chip DRAM into faster, smaller on-chip SRAMs and local caches to maximize data reuse without violating area and thermal constraints.

Because the energy and performance limitations of traditional memory subsystems threatened to halt neural network scaling, designers engineered a new memory organization: High-Bandwidth Memory (HBM). By vertically stacking DRAM dies on a silicon interposer and connecting them via through-silicon vias (TSVs), HBM delivers orders of magnitude higher bandwidth at a much lower power-per-bit (Zhang, 2026). The profound necessity of this architecture

shift was quantified by Google’s architectural analysis of their first-generation TPU; engineers projected that bypassing standard memory for high-bandwidth alternatives would overcome their existing memory wall, tripling throughput to achieve a theoretical performance-per-watt nearly 70 times greater than contemporary GPUs and 200 times that of Central Processing Units (CPUs) (Jouppi *et al.*, 2017). However, the intense structural complexity and substantial die sizes required to actually manufacture HBM, as well as other factors, have triggered a severe supply chain crisis. Driven by the artificial intelligence market, manufacturers aggressively reallocated production to feed data center demand, inducing an unprecedented shortage that caused standard DDR5 contract prices to surge by nearly 300% in a single quarter (Tom’s Hardware, 2025; Reuters, 2025). As hyperscalers monopolize the global supply of advanced memory, edge and embedded System-on-Chip (SoC) platforms are financially excluded from these hardware solutions. This market scarcity motivates engineers to aggressively optimize edge neural accelerators, maximizing on-chip memory efficiency and data reuse to deploy deep learning models under cost limitations.

Achieving this requisite efficiency requires employing Design Space Exploration (DSE). DSE acts as the systematic evaluation of competing microarchitectural parameters—such as memory hierarchy sizing, processing element arrays, and dataflow mapping—to identify the optimal balance between power dissipation, silicon area, and computational throughput. De Micheli formally defines this methodology as a mathematical abstraction (Micheli, 1994). Navigating this vast multidimensional design space allows engineers to pinpoint precise architectural configurations, for instance to maximize on-chip data reuse while adhering to the physical limitations of edge environments, an optimization methodology successfully demonstrated by the Eyeriss architecture (Chen; Emer; Sze, 2016).

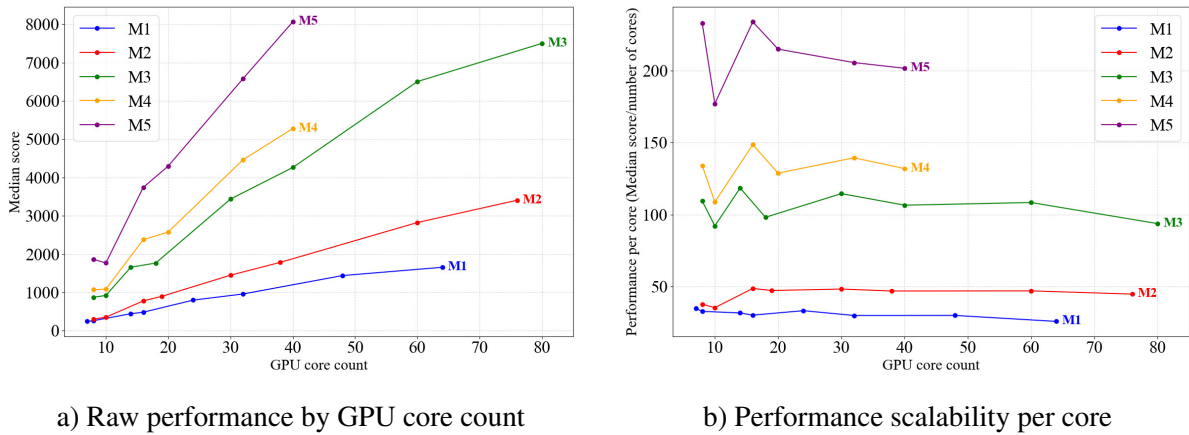
Memory bandwidth and latency in edge devices have also become defining characteristics for GPUs and DSAs such as hardware ray-tracing accelerators. These processing units are similar in concept to neural accelerators in that they are optimized for extensive data-level parallelism—executed via Single Instruction, Multiple Data (SIMD) architectures—dedicating a significant portion of their silicon area to active floating-point datapaths rather than the large, multi-level caches typical of general-purpose processors (Patterson; Hennessy, 2020). Because these architectures process thousands of concurrent operations to hide memory access delays, their overall computational throughput is constrained by the maximum bandwidth of the memory subsystem rather than its access latency (Patterson; Hennessy, 2020). The targeted case study of empirical performance metrics that follows demonstrates the impact of this memory optimization on consumer edge devices.

1.1.1 Architectural Scaling Bounds in Consumer Edge SoCs

Figure 1 illustrates how Apple Silicon embedded SoCs perform under the Blender Open Data Benchmark (Blender Foundation, 2026), representing scene rendering performance

under the "Cycles" rendering engine. M1 through M5 represent successive generations of Apple's custom ARM-based SoCs. To analyze architectural scaling, all tier variants within a specific family (Standard, Pro, Max, and Ultra) were aggregated by their base generation. The comprehensive dataset detailing the median scores and sample sizes for all evaluated configurations is provided in Appendix A. Apple's architectural design philosophy justifies this grouping; across a single generation, the manufacturer scales performance by expanding unified memory bandwidth and adding discrete GPU and CPU cores, rather than altering the core microarchitecture itself (Apple Inc., 2021). Some variants even comprise two lower-tier dies connected via Apple's proprietary UltraFusion packaging architecture (Apple Inc., 2022).

Figure 1 – Apple Silicon GPU performance analysis in Blender Open Data



Source: Adapted from Blender Foundation (2026). Data grouped from Blender versions 4.5.0, 5.0.0, 5.1.0, and 5.1.1 for higher sample sizes.

Analyzing these aggregated generations reveals a distinct scaling tendency across different configurations. As illustrated in Figure 1(a), raw rendering performance generally scales upwards as GPU core counts increase within a given generation. However, isolating this data to evaluate efficiency per core reveals an absolute microarchitectural bottleneck; Figure 1(b) demonstrates a significant variance in per-core performance between sequential core counts. Notably, as derived from the benchmark dataset, the M5 generation displays lower raw performance when expanding from the 8-core to the 10-core GPU configuration. This phenomenon exemplifies chip binning (a form of defect tolerance (Wang; Wu; Wen, 2006)), a manufacturing practice where fabricators deactivate defective cores on a silicon die and repurpose the chip as a lower-tier processor, maximizing production yields (Weste; Harris, 2010; Tanenbaum, 2013).

Crucially, despite this reduction in active core count, the unified memory interface maintains its full bandwidth capacity. Therefore, binned processors feature fewer active cores competing for the same memory resources. This architectural trait yields higher per-core efficiency during memory-bound workloads, such as 3D scene rendering. As an example, Apple offers the M3, M4, and M5 base architectures in two distinct hardware configurations: an 8-core

binned variant and a fully unlocked 10-core variant. Table 1 compares the per-core performance metrics between these two versions.

Table 1 – Per-core performance and memory bandwidth scaling in base Apple Silicon variants

Architecture Variant	GPU Cores	Total BW (GB/s)	BW / Core (GB/s)	Perf. / Core (Median Score)
M3 (Binned)	8	100	12.50	109.45
M3 (Fully-enabled)	10	100	10.00 (−20.0%)	92.17 (−15.8%)
M4 (Binned)	8	120	15.00	134.07
M4 (Fully-enabled)	10	120	12.00 (−20.0%)	108.79 (−18.9%)
M5 (Binned)	8	150	18.75	233.06
M5 (Fully-enabled)	10	150	15.00 (−20.0%)	177.06 (−24.0%)

Source: Elaborated by the author based on Blender Foundation (2026) and manufacturer hardware specifications (Apple Inc., 2024b; Apple Inc., 2024a; Apple Inc., 2026a).

Table 1 shows that the reduction in performance per core is approximately proportional to the loss in bandwidth per core. More complex systems such as the M3 Ultra exhibit this trend to a lesser extent, as seen in Table 2.

Table 2 – Per-core performance and memory bandwidth scaling in M3 Ultra variants

Architecture Variant	GPU Cores	Total BW (GB/s)	BW / Core (GB/s)	Perf. / Core (Median Score)
M3 Ultra (Binned)	60	819	13.65	108.43
M3 Ultra (Fully-enabled)	80	819	10.24 (−25.0%)	93.79 (−13.5%)

Source: Elaborated by the author based on Blender Foundation (2026) and manufacturer hardware specifications (Apple Inc., 2026b).

While attributing the drop in per-core performance primarily to memory bandwidth limitations provides a robust correlation, it relies on necessary microarchitectural abstractions. Subtle structural differences between the binned and fully-enabled configurations—such as unmodeled variations in shared cache allocations, structural hazards, or out-of-order execution buffer sizing—could also influence this efficiency gap (Williams; Waterman; Patterson, 2009). Additionally, because the empirical data is derived from commercial deployments across varied thermal chassis—including fanless environments—disparities in thermal throttling, Total Board Power (TBP) limits, and variable DRAM capacities likely contribute to the observed degradation in multi-core scaling efficiency. Nevertheless, given the data-level parallelism required by these rendering workloads, memory bandwidth contention remains the dominant constraint driving the observed degradation.

This empirical data confirms that hardware performance scales proportionally with memory bandwidth per core, rather than raw core count. As small-die DSAs integrated into consumer edge devices face absolute power, thermal, and memory access limits (Sze *et al.*, 2020), adding computational logic without commensurate bandwidth yields diminishing returns. While commercial System-on-Chips demonstrate the macro-effects of this memory wall, isolating and manipulating its microarchitectural root causes requires a highly targeted approach. Therefore, evaluating the exact point where memory constraints throttle computational throughput at the foundational level is essential. To address the fundamental performance and energy bottlenecks within these architectures, this work explores the design space of a foundational DSA building block: a layer-level, weight-resident Matrix-Vector Unit (MVU) utilizing a temporal folded SIMD datapath.

To reduce the energy and latency associated with operand movement, this work evaluates a parameterizable MVU. Functioning as a layer-level compute engine, the MVU calculates the output vector $y = Wx$ for an 8-bit input vector x and weight matrix W . The architecture employs a folded SIMD dot-product micro-kernel with local accumulation, integer requantization (Jacob *et al.*, 2017), and a selectable activation function. By retaining all layer weights within internal registers, the design eliminates off-chip memory accesses during inference (Alam *et al.*, 2023; Fowers *et al.*, 2018). The evaluation isolates a single MVU instance to quantify microarchitectural scaling independent of multi-core array dynamics. Within this scope, output channels are generated sequentially via temporal folding over the SIMD datapath, precluding spatial parallelism (Alam *et al.*, 2023).

The potential contributions of this work include a quantitative analysis of the physical impact of the Memory Wall in edge neural accelerators. By executing a mixed-level Design Space Exploration (DSE), this project physically demonstrates—at the silicon level—how and when scaling computational hardware ceases to be efficient due to memory starvation. Consequently, this study provides practical design guidelines for selecting SIMD width and on-unit storage capacity as a function of available memory bandwidth in edge-oriented deployments.

1.2 Objectives

1.2.1 General Objective

The primary objective of this project is to conduct a quantitative analysis of memory bandwidth limitations in edge-oriented neural accelerators. Through a mixed-level DSE, the study evaluates the parameterized MVU by varying SIMD lane counts and on-unit storage capacities. This evaluation determines the specific hardware thresholds where computational scaling yields diminishing returns due to data transfer constraints.

1.2.2 Specific Objectives

- **Functional and Physical Implementation:** Implementing the MVU at the Register-Transfer Level (RTL) and synthesizing the logic to extract physical area, maximum clock frequency ($F_{\max}$), and power dissipation metrics.
- **Throughput Evaluation:** Simulating constrained memory interfaces to measure empirical cycle counts and achievable Giga-Multiply-Accumulate Operations per Second (GMAC/s) against theoretical peak performance.
- **Constraint Modeling:** Parameterizing the Extended Roofline Model (Cabezas; Püschel, 2014) and Energy-Delay Product (EDP) (Gonzalez; Horowitz, 1996) to correlate physical hardware limits with the operational intensity (OI) of neural network workloads.
- **Design Guidelines:** Establishing empirical criteria for provisioning SIMD width and internal register sizes based on available external bandwidth in edge environments.
- **Methodological Rigor:** Executing the architectural synthesis and verification pipeline within an industry-standard Electronic Design Automation (EDA) framework, ensuring the derived empirical metrics reflect realistic physical implementations rather than theoretical approximations.

1.3 Document Structure

The second chapter reviews the foundational principles of deep neural networks and hardware optimization strategies for edge computing environments. The text introduces analytical evaluation frameworks, specifically the Energy-Delay Product and the Roofline model, and examines existing literature concerning spatial dataflows and weight-resident streaming architectures.

The third chapter documents the implementation and systematic evaluation of the parameterizable MVU. The section describes the register-transfer level datapath, the functional verification procedures, and the simulated memory constraints. Subsequent subsections quantify the extracted synthesis metrics, reporting operational frequencies, silicon area allocations, and static power dissipation across the parameterized design space. The chapter concludes with a system-level performance analysis utilizing the Extended Roofline Model, an Energy-Delay Product assessment, and an objective analysis of the methodological limitations affecting the data extraction.

The fourth chapter summarizes the technical contributions derived from the architectural exploration. The text records the specific engineering constraints resolved during the implementation phase and outlines structural directives for subsequent microarchitectural research.

Two appendices supplement the core text. Appendix A supplies empirical benchmarking data for commercial graphics processing units. Appendix B contains the full register-transfer level source code and the configuration scripts utilized for physical logic synthesis. Finally, an annex is attached to provide additional project documentation.

2 LITERATURE REVIEW

2.1 Initial Considerations

This chapter establishes the theoretical foundation required to evaluate memory constraints within neural accelerators. Section 2.2 details the computational primitives of Deep Neural Networks (DNNs) and the microarchitectural mechanisms used to exploit data-level parallelism in edge environments. The section also defines the analytical models required to quantify hardware efficiency, focusing on the EDP and the Roofline performance framework. Section 2.3 reviews existing literature on DSAs. By contrasting conventional intra-layer spatial dataflows such as Eyeriss against the layer-level streaming paradigms implemented by the FINN framework and Microsoft Project Brainwave, the review contextualizes the specific MVU evaluated in this thesis.

2.2 Theoretical Background

2.2.1 Fundamentals of Deep Neural Networks

DNNs are a class of machine learning models that extract hierarchical features from raw data through multiple computational layers. Rather than relying on handcrafted heuristics, DNNs learn internal representations from data (Goodfellow; Bengio; Courville, 2016). The fundamental building block is the artificial neuron. During inference, each neuron receives a vector of input activations (x), multiplies them by a vector of learned parameters called weights (w), accumulates the products into a weighted sum, and adds a scalar bias (b) (Sze *et al.*, 2020). This operation is expressed as

$$z = w^T x + b, \quad (2.1)$$

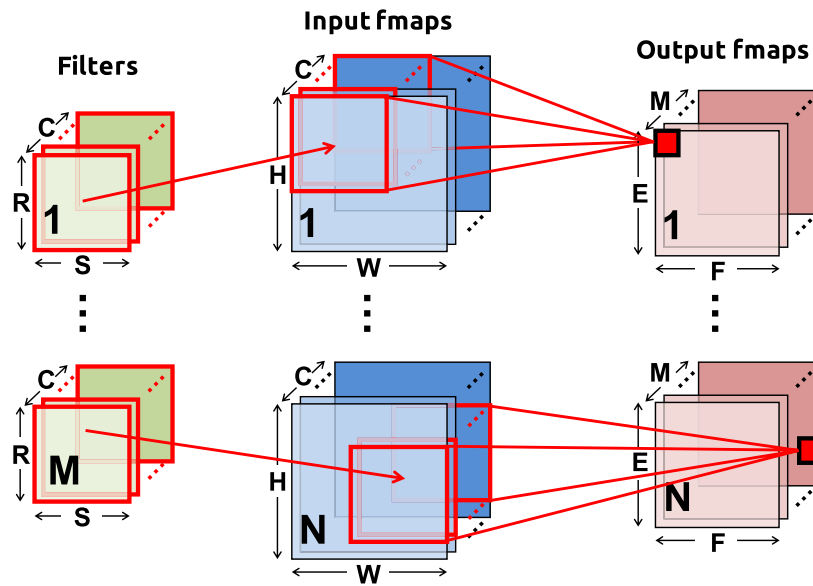
where $w^T x$ denotes an inner product (Goodfellow; Bengio; Courville, 2016). Deploying these models on resource-constrained devices motivates lightweight algorithmic and system designs, often discussed under the TinyML paradigm (Banbury *et al.*, 2021).

DNN data are represented as multi-dimensional arrays (tensors) (Goodfellow; Bengio; Courville, 2016). In computer architecture literature, the input supplied to a layer is denoted Input Feature Maps (ifmaps), which comprise individual input activations (Sze *et al.*, 2020). For example, in computer vision, an ifmap can represent a color image over two spatial dimensions (image height and width) and a depth of three channels corresponding to red, green, and blue (RGB) intensities (Goodfellow; Bengio; Courville, 2016). To extract features, the layer applies learned parameters (filters) composed of individual weights (Chen; Emer; Sze, 2016; Sze *et al.*, 2020). Hardware multiplies ifmap activations by filter weights to generate partial sums (psums), which are accumulated to produce Output Feature Maps (ofmaps) (Chen; Emer; Sze, 2016; Sze *et al.*, 2020). Consistent definitions for tensor dimensions are necessary for reproducible analysis,

since tensor sizes determine memory footprint and compute requirements (Chen; Emer; Sze, 2016).

In computer vision workloads, convolutional (CONV) layers typically dominate MAC activity (Chen; Emer; Sze, 2016) in CNNs—a class of DNNs that process grid-structured data such as images through stacked, locally-connected filters (Goodfellow; Bengio; Courville, 2016). A high-dimensional convolution slides a three-dimensional filter across the spatial dimensions of the input feature map (Sze *et al.*, 2020). At each overlapping window, the computation forms the dot product between input activations and filter weights and accumulates the resulting partial sums to produce one output value (Chen; Emer; Sze, 2016; Sze *et al.*, 2020). Because the same set of weights is applied across many spatial windows, CONV layers exhibit weight reuse and data-parallel structure (Chen; Emer; Sze, 2016; Sze *et al.*, 2020). Figure 2 illustrates the sliding-window mechanism, showing how a multi-dimensional filter kernel multiplies a specific window of the input feature map to produce an output channel in the output feature map (Chen *et al.*, 2017).

Figure 2 – Sliding window mechanism in the computation of a high-dimensional CNN layer



Source: Extracted from Chen *et al.* (2017).

Hardware analysis requires explicit specification of tensor shape parameters, as summarized in Figure 2 (Chen; Emer; Sze, 2016). An input feature map is parameterized by spatial height (H) and width (W), and by a channel dimension (C) (Chen; Emer; Sze, 2016). The filter matches this channel depth (C) and has spatial dimensions (R) and (S), which are commonly smaller than those of the input feature map (Sze *et al.*, 2020). Processing multiple independent inputs introduces the batch dimension (N) (Chen; Emer; Sze, 2016). Applying M distinct three-dimensional filters yields an output feature map with M channels; the output spatial height (E)

and width (F) depend on filter dimensions and stride (U) (Chen; Emer; Sze, 2016). Between convolutional stages, networks may include pooling layers (e.g., max or average pooling) that downsample (E , F), reducing activation sizes and subsequent workload (Sze *et al.*, 2020). Table 3 summarizes this nomenclature for the dimensional variables that define the nested-loop computation mapped onto Processing Elements (PEs) (Chen *et al.*, 2017).

Table 3 – Shape parameters of a CNN layer

Shape Parameter	Description
N	batch size of 3D fmaps
M	# of 3D filters / # of ofmap channels
C	# of ifmap/filter channels
H/W	ifmap plane height/width
R/S	filter plane height/width
E/F	ofmap plane height/width

Source: Reproduced from Chen *et al.* (2017).

After feature extraction via convolutional layers, DNNs commonly use FC layers and non-linear activation functions to produce final predictions (Chen; Emer; Sze, 2016). An FC layer can be expressed as a special case of convolution in which the filter spatial dimensions match the ifmap spatial dimensions ($H = R$ and $W = S$) (Sze *et al.*, 2020; Chen; Emer; Sze, 2016). Under this mapping, each output activation depends on all input activations, and the weight-reuse behavior present in CONV layers does not apply (Sze *et al.*, 2020; Chen; Emer; Sze, 2016). The activation function then applies a non-linear transform to the accumulated output (Sze *et al.*, 2020). Sigmoid and hyperbolic tangent (tanh) functions require transcendental computation and are commonly approximated in hardware via piecewise-linear Look-Up Tables (LUTs) (Kwon; Samajdar; Krishna, 2018). Many accelerator datapaths therefore implement Rectified Linear Unit (ReLU) behavior, which clamps negative values to zero (Goodfellow; Bengio; Courville, 2016; Sze *et al.*, 2020).

Feedforward inference proceeds sequentially across layers (Goodfellow; Bengio; Courville, 2016). The ofmap produced by layer N becomes the ifmap consumed by layer $N + 1$ (Sze *et al.*, 2020). These intermediate feature maps frequently exceed register-file capacity and therefore require repeated eviction and retrieval across the memory hierarchy (Chen; Emer; Sze, 2016). This layer-to-layer data dependency drives sustained read and write activity in on-chip SRAM buffers and, when on-chip capacity is insufficient, in off-chip memory (Sze *et al.*, 2020).

2.2.2 Edge DNN Hardware Architectures

Neural accelerators often use SIMD architectures to execute the matrix and tensor operations that dominate DNN workloads. By replicating MAC units in SIMD arrays, these designs exploit data-level parallelism to increase throughput.

Sustaining SIMD utilization requires a memory hierarchy that trades capacity for access latency (Patterson; Hennessy, 2020). Data staging spans off-chip DRAM, on-chip SRAM, and registers. Because DNN access patterns are structured and dataflow is largely deterministic, implicitly managed CPU-style caches may be inefficient for these workloads; many DSAs instead use explicit, software-managed memories (Jouppi *et al.*, 2017). For example, Google’s TPU v1 includes a 24 MiB on-chip SRAM Unified Buffer that supplies operands to its MAC array (Jouppi *et al.*, 2017), whereas Eyeriss uses distributed local register files (RF) and SRAM scratchpads to retain operands near the compute units (Chen; Emer; Sze, 2016).

Edge deployment commonly applies quantization-aware training to reduce arithmetic and storage requirements while maintaining accuracy (Jacob *et al.*, 2017). During training, the forward pass simulates quantization so the model adapts its parameters to discrete integer representations (Jacob *et al.*, 2017). During inference, the datapath typically multiplies 8-bit integer weights and activations and accumulates products into a wider 32-bit register to control overflow and preserve intermediate precision (Jouppi *et al.*, 2017; Sze *et al.*, 2020). After accumulation, the datapath applies a precomputed scaling multiplier and re-quantizes the result to an 8-bit output (Jacob *et al.*, 2017). The output is then clamped to the 8-bit integer range (e.g., 0 to 255) (Jacob *et al.*, 2017).

The quantized representation uses an affine mapping between real values (r) and integer values (q), following Equation 2.2 (Jacob *et al.*, 2017):

$$r = S(q - Z) \quad (2.2)$$

where the scale (S) is a positive real number and the zero-point (Z) is an integer offset that ensures the real value zero is representable within the integer domain (Jacob *et al.*, 2017).

For matrix multiplication, the quantized output feature map element (q_3) is computed from quantized weights (q_1) and activations (q_2) as shown in Equation 2.3 (Jacob *et al.*, 2017):

$$q_3^{(i,k)} = Z_3 + M \sum_{j=1}^N (q_1^{(i,j)} - Z_1)(q_2^{(j,k)} - Z_2) \quad (2.3)$$

with:

- $q_1^{(i,j)}$ and $q_2^{(j,k)}$ denoting 8-bit integer elements of the weight and activation tensors (Jacob *et al.*, 2017);
- Z_1 , Z_2 , and Z_3 denoting the corresponding integer zero-points (Jacob *et al.*, 2017);
- M denoting a real-valued scaling multiplier defined by $M = \frac{S_1 S_2}{S_3}$ (Jacob *et al.*, 2017). Implementations may compute M once per tensor (per-layer quantization) or separately per output channel (per-filter quantization) (Jacob *et al.*, 2017; Sze *et al.*, 2020).

This formulation factors the non-integer multiplier (M) outside the inner accumulation, allowing the main loop to use integer arithmetic for MAC operations (Jacob *et al.*, 2017). The resulting datapath reduces dependence on floating-point units while preserving a scaling mechanism that maps accumulated integers back to the target quantized domain (Jacob *et al.*, 2017; Sze *et al.*, 2020).

2.2.3 Analytical Performance Models: EDP and Roofline

Mathematical analysis of hardware designs and objective comparison of experimental results require standardized measurement techniques. Relying exclusively on isolated variables fails to capture the complex trade-offs inherent to semiconductor physics and digital logic design.

Raw power dissipation, for instance, provides an incomplete metric for comparing processor designs, as it correlates directly with clock frequency (Gonzalez; Horowitz, 1996). Lowering the operating frequency reduces power consumption but proportionally increases execution time, leaving the underlying microarchitectural efficiency unaltered (Horowitz, 2014; Gonzalez; Horowitz, 1996).

Evaluating architectures based solely on minimum total energy consumption introduces comparable analytical limitations. In Complementary Metal-Oxide-Semiconductor (CMOS) digital circuits, dynamic energy dissipation scales quadratically with the supply voltage (V_{DD}), proportional to CV_{DD}^2 (Horowitz; Indermaur; Gonzalez, 1994; Weste; Harris, 2010). Reducing the supply voltage lowers total energy consumption but concurrently increases circuit propagation delay. Therefore, optimizing strictly for minimum energy yields slow designs, compromising computational throughput for isolated energy reductions (Horowitz; Indermaur; Gonzalez, 1994).

This structural increase in execution delay is fundamentally incompatible with modern neural network inference, which operates under defined real-time constraints. Domains such as autonomous driving and computer vision mandate bounded response times. Furthermore, empirical analysis of warehouse-scale deployments confirms that interactive inference applications impose hard response-time deadlines, fundamentally diverging from the throughput-centric paradigm of neural network training (Jouppi *et al.*, 2017). This operational reality dictates that commercial inference architectures prioritize 99th-percentile response-time targets over aggregate batch throughput (Jouppi *et al.*, 2017).

Minimizing either power or energy in isolation inherently compromises processing speed (Horowitz; Indermaur; Gonzalez, 1994; Gonzalez; Horowitz, 1996). The Energy-Delay Product (EDP)—defined mathematically as total energy consumption multiplied by execution time—mitigates this inverse proportionality by penalizing logic designs that achieve nominal power reductions through clock frequency or supply voltage scaling (Horowitz; Indermaur; Gonzalez, 1994). Evaluating EDP within a constant fabrication technology node isolates microarchitectural efficiency from the physical improvements of transistor shrinking (Gonzalez; Horowitz, 1996). Nevertheless, EDP fails to quantify silicon area, a critical geometric variable

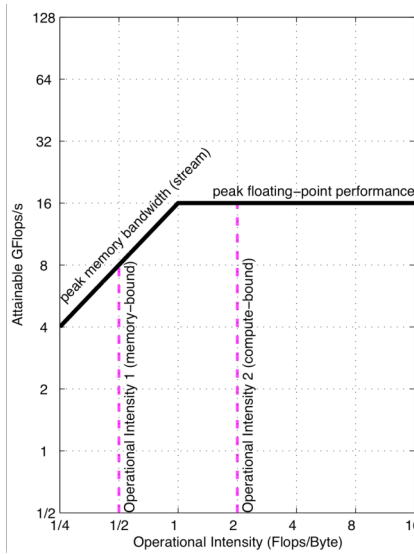
dictating both static leakage and fabrication viability (Weste; Harris, 2010; Gonzalez; Horowitz, 1996). Because manufacturing defects distribute randomly across a silicon wafer, net die yield decreases exponentially as the active integration area increases (Weste; Harris, 2010).

Foundational VLSI methodologies rely on EDP to calculate optimal supply and threshold voltages for low-power digital circuits (Weste; Harris, 2010). This evaluation framework also extends to neural accelerators. In the Eyeriss architecture, for example, EDP provides a quantitative baseline for comparing spatial dataflows (Chen; Emer; Sze, 2016). Applying this metric guarantees that memory-access optimizations do not achieve energy reductions by artificially restricting the parallel computation of the PE array (Chen; Emer; Sze, 2016).

EDP quantifies the quality of isolated switching events but cannot capture systemic data starvation. Deep neural network (DNN) workloads process dense multi-dimensional matrices that frequently exceed on-chip storage capacities, shifting the primary limitation from arithmetic logic unit (ALU) efficiency to memory bandwidth (Sze *et al.*, 2020). Consequently, an architecture with a mathematically optimal EDP datapath registers high hardware underutilization if the memory hierarchy fails to supply operands at the required frequency. To formalize this intersection of peak computational throughput and memory bandwidth, Williams, Waterman, and Patterson developed the Roofline model (Williams; Waterman; Patterson, 2009). This two-dimensional framework applies bound and bottleneck analysis to correlate physical hardware capabilities with algorithmic requirements, specifically isolating off-chip memory bandwidth as the primary limiting factor in contemporary processing architectures (Williams; Waterman; Patterson, 2009).

The Roofline model graphs attainable performance as a function of operational intensity on a base-2 logarithmic scale, accommodating the multiple orders of magnitude that separate distinct hardware configurations (Williams; Waterman; Patterson, 2009). Mathematically, achievable performance is constrained by a piecewise linear equation: $\min(P_{\text{peak}}, BW \times I)$ (Williams; Waterman; Patterson, 2009), where P_{peak} represents peak computational throughput, BW defines peak memory bandwidth, and I denotes operational intensity. Because memory bandwidth operates as a linear coefficient in this formulation, mapping the parameters onto a log-log coordinate system projects the memory constraint as a continuous diagonal boundary with a constant slope of one, thereby preserving geometric uniformity across varied hardware specifications, as illustrated in Figure 3 (Williams; Waterman; Patterson, 2009).

Figure 3 – Roofline Model for an AMD Opteron X2.



Source: Extracted from Williams, Waterman and Patterson (2009).

As demonstrated in Figure 3, plotting discrete software kernels on this coordinate system visually isolates their respective hardware limitations. A workload with low operational intensity maps to the left of the ridge point, remaining strictly memory-bound beneath the diagonal boundary. Conversely, a highly mathematically intensive workload maps to the right of the ridge point, achieving higher absolute performance but ultimately constrained by the horizontal compute ceiling.

The independent variable, operational intensity, quantifies the computational operations executed per byte of memory traffic (Williams; Waterman; Patterson, 2009). By measuring this traffic strictly after the internal cache hierarchy filters the data stream, the metric isolates main-memory limitations from localized processor-cache exchanges (Williams; Waterman; Patterson, 2009). The generalized nomenclature of this parameter accommodates domain-specific adaptations; rather than relying exclusively on floating-point operations, the formulation readily accepts specialized arithmetic equivalents (Williams; Waterman; Patterson, 2009). When evaluating neural accelerators, the performance axis typically scales in GMAC/s to reflect the fundamental compute primitives of DNNs (Sze *et al.*, 2020).

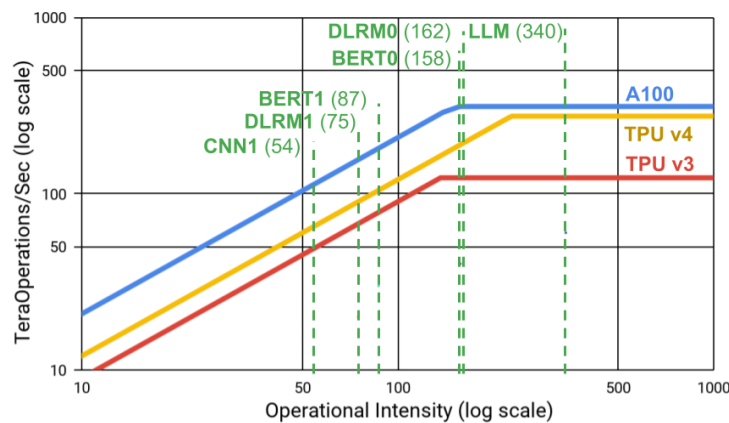
Intersecting the horizontal computational bound and the diagonal memory boundary yields the ridge point, which dictates the precise operational intensity required to transition a system from memory-constrained to compute-constrained execution (Williams; Waterman; Patterson, 2009). Assuming a fixed algorithm and dataset, a specific software application maps to a static horizontal coordinate (Williams; Waterman; Patterson, 2009). Within this coordinate space, the absolute memory boundary derives from empirically sustained transfer rates—frequently evaluated via standardized memory tests—rather than theoretical hardware pin capacities (Williams;

Waterman; Patterson, 2009). Below these maximum theoretical limits, engineers project secondary ceilings corresponding to discrete architectural optimizations (Williams; Waterman; Patterson, 2009). Evaluating the vertical delta between an application’s current throughput and the immediate overlying ceiling provides a direct quantitative measure of the performance gain achievable by resolving that specific hardware restriction (Williams; Waterman; Patterson, 2009). In this analysis, algorithmic variables exclusively govern the operational intensity, whereas physical parameters—including memory frequency and SIMD lane dimensions—establish the placement of the performance bounds and optimization ceilings.

This mathematical framework facilitates bidirectional co-design, aligning physical hardware provisioning with algorithmic requirements. Silicon architects evaluate the operational intensity of anticipated workloads to determine the optimal placement of the ridge point. This diagnostic process prevents the provisioning of excess arithmetic logic units that would remain idle under strict memory transfer limits (Williams; Waterman; Patterson, 2009)—a hardware constraint directly observed during the empirical analysis of Apple Silicon architectures in Subsection 1.1.1. Programmers apply the inverse methodology, restructuring software to increase operational intensity, thereby shifting the application rightward along the horizontal axis toward the hardware’s existing ridge point (Williams; Waterman; Patterson, 2009).

DSAs adopt this framework to quantify DNN performance against physical memory and logic limits. When evaluating the TPU v1 architecture, Jouppi et al. calibrated the model for deep learning inference by substituting floating-point arithmetic with 8-bit integer operations and redefining the independent axis as operations per byte of weight data retrieved (Jouppi *et al.*, 2017). This adjustment demonstrated that specific topologies operate strictly under the diagonal memory constraint (Jouppi *et al.*, 2017). Subsequent analyses of the TPU v4 supercomputer utilized the model to overlay the performance boundaries of three distinct hardware architectures alongside multiple DNN workloads within a unified graph (Figure 4) (Jouppi *et al.*, 2023).

Figure 4 – Roofline models for TPU v4, TPU v3, and A100 plus DNN models. Operational intensity is in parentheses.



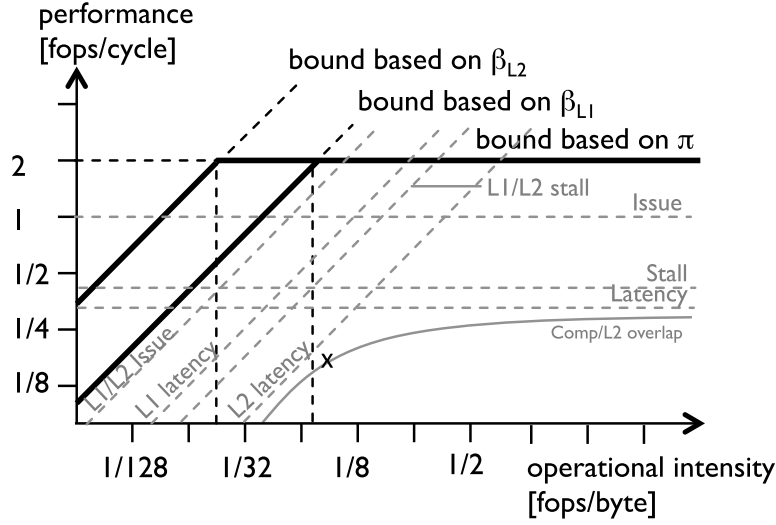
Source: Extracted from Jouppi *et al.* (2023).

Although the foundational Roofline framework effectively isolates throughput limitations, its mathematical simplifications omit non-throughput variables, notably memory access latency and finite structural capacities (Williams; Waterman; Patterson, 2009). The original two-dimensional formulation postulates a perfect temporal overlap between arithmetic computations and memory transfers, assessing the memory hierarchy exclusively through sustained bandwidth rates (Williams; Waterman; Patterson, 2009). By declining to decouple bandwidth from latency, the standard model remains unresponsive to hardware underutilization induced by prolonged data retrievals or exhausted out-of-order (OoO) execution resources, such as reorder buffers (ROB) and reservation stations (RS) (Williams; Waterman; Patterson, 2009). Characterizing these discrete microarchitectural restrictions dictates a transition toward extended multi-roof (Cabezas; Püschel, 2014) or multi-dimensional (Lorenzo *et al.*, 2014) analytical models.

Caparrós Cabezas and Püschel address these limitations by augmenting the standard Roofline model, which originally assumes perfect temporal overlap between computation and memory transfers to hide latency, mathematically omitting non-throughput variables (Williams; Waterman; Patterson, 2009). Their extended formulation projects secondary bottleneck ceilings beneath the absolute peak roofs by isolating the specific clock cycles stalled by memory latency, computational latency, or finite execution buffers (Cabezas; Püschel, 2014).

To extract these empirical delay fractions, the authors use an extended LLVM interpreter to dynamically schedule the execution of a computation Directed Acyclic Graph (DAG) (Cabezas; Püschel, 2014). However, extracting bottlenecks from a dynamically scheduled execution creates a critical limitation: the resulting boundaries are mathematically interdependent. Altering a single microarchitectural parameter forces the out-of-order hardware algorithm to adapt and generate a completely new execution schedule, shifting the rooflines unpredictably (Cabezas; Püschel, 2014). While this dynamic scheduling simulation falls outside the scope of statically scheduled DSAs, the visual concept of projecting secondary latency and bandwidth boundaries remains a highly effective analytical tool for evaluating neural accelerators, as illustrated in Figure 5.

Figure 5 – Extended Roofline model illustrating secondary performance ceilings constrained by microarchitectural bottlenecks, including cache latency and imperfect overlap.



Source: Extracted from Cabezas and Püschel (2014).

As depicted in Figure 5, the extended framework visualizes these interdependent constraints by projecting a series of subsidiary limits beneath the absolute theoretical roofs. The solid black boundaries represent the traditional Roofline constraints: peak computational throughput (π) and peak memory bandwidths for distinct levels of the memory hierarchy (β_{L1} and β_{L2}). The dashed lines represent the empirical microarchitectural ceilings extracted from the simulation. Horizontal boundaries isolate throughput degradation caused by computational latency and finite instruction issue limits, whereas the secondary diagonal boundaries reflect the performance penalties of specific memory access latencies. Furthermore, the model plots curved boundaries to represent operational regions where the dynamic schedule fails to achieve perfect temporal overlap between arithmetic operations and memory transfers. Although the specific multi-level cache variables (L1, L2) and dynamic instruction limits mapped in this example apply to general-purpose processors, the underlying geometric methodology of isolating distinct latency and bandwidth ceilings directly informs the architectural evaluation of the proposed accelerator.

2.3 Related Work

2.3.1 Spatial Architectures and Dataflows

Spatial architectures process neural network layers by distributing parallel computations across two-dimensional arrays of compute nodes (Sze *et al.*, 2020). These nodes, called a PE, usually encapsulate an ALU configured for MAC operations, integrated control logic, and an internal scratchpad (SPad) or register file (RF) (Chen; Emer; Sze, 2016). Incorporating these local memory primitives establishes a distributed memory hierarchy that temporarily retains operands, thereby preventing the energy-intensive retrieval of data from the global buffer or

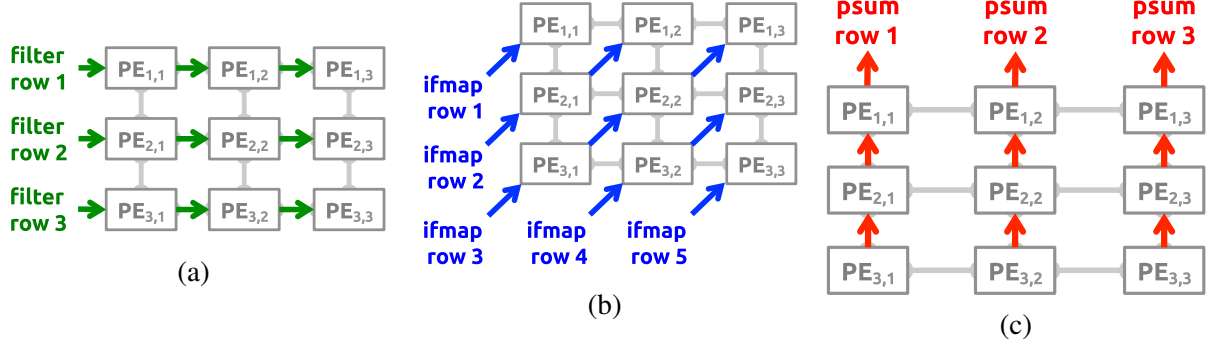
off-chip DRAM (Sze *et al.*, 2020). To sustain this localized execution, a dedicated Network-on-Chip (NoC) facilitates direct operand exchange among the interconnected PEs (Chen; Emer; Sze, 2016).

Finite scratchpad capacities compel spatial architectures to prioritize the retention of specific operands, establishing a dataflow taxonomy defined by the stationary data type (Sze *et al.*, 2020). In Weight-Stationary (WS) configurations, filter parameters reside within the processing element (PE) register file while the network-on-chip broadcasts input activations and accumulates partial sums spatially. Output-Stationary (OS) designs retain partial sums locally for temporal accumulation, streaming both inputs and weights through the array. Alternatively, Input-Stationary (IS) mappings anchor input activations in the register file to exploit convolutional reuse, necessitating the unicast delivery of weights to individual PEs.

Resolving the mutual exclusivity of classical stationary mappings, the Eyeriss architecture implements a Row-Stationary (RS) dataflow that balances the simultaneous reuse of all operands. Rather than anchoring a single data type, the RS mapping projects one-dimensional convolution primitives directly onto individual PEs (Chen; Emer; Sze, 2016). Under this configuration, the local scratchpad concurrently retains one row of filter weights, a sliding window of input feature map pixels, and the corresponding one-dimensional partial sums (Chen *et al.*, 2019). Confining one-dimensional convolutional reuse entirely within the PE boundaries circumvents energy-intensive data transfers across the global array.

Evaluating a complete two-dimensional convolution requires organizing multiple one-dimensional primitives into a "logical PE set", a structural construct that dictates the physical data routing across the spatial array (Chen; Emer; Sze, 2016). As illustrated in Figure 6, this configuration establishes three distinct routing trajectories: rows of filter weights propagate horizontally for reuse across adjacent PEs, rows of input feature map pixels traverse the array diagonally, and partial sums accumulate vertically (Chen; Emer; Sze, 2016). By leveraging inter-PE communication, this spatial orchestration maximizes the simultaneous reuse of weights, input activations, and partial sums, thereby bounding the bandwidth demanded from the global buffer (Chen; Emer; Sze, 2016).

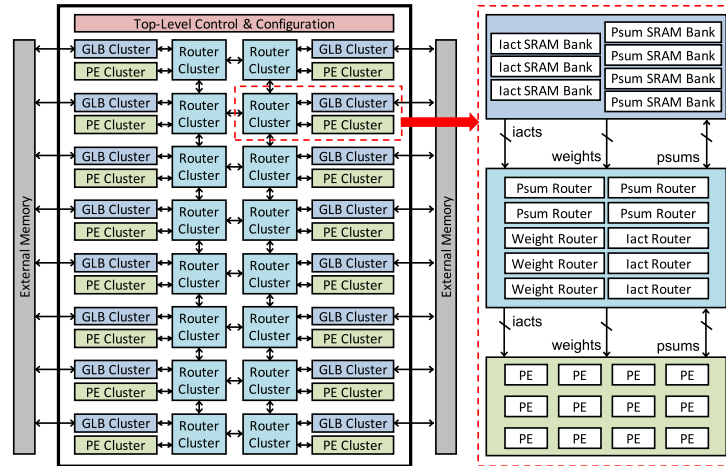
Figure 6 – The dataflow in a logical PE set to process a 2D convolution. (a) rows of filter weight are reused across PEs horizontally. (b) rows of ifmap pixel are reused across PEs diagonally. (c) rows of psum are accumulated across PEs vertically. In this example, $R = 3$ and $H = 5$.



Source: Extracted from Chen, Emer and Sze (2016).

Advancing this distributed memory methodology, Figure 7 illustrates the hierarchical memory organization of the Eyeriss v2 accelerator. To scale the Row-Stationary dataflow efficiently, the architecture partitions the system into localized functional groups, each consisting of a Global Buffer (GLB) cluster, a Router cluster, and a PE cluster (Chen *et al.*, 2019). The GLB utilizes dedicated SRAM banks to temporarily store input activations and partial sums, whereas filter weights bypass the GLB storage entirely to stream directly from external memory into the routing infrastructure (Chen *et al.*, 2019). Operating as the communication backbone, the router clusters establish a Hierarchical Mesh Network-on-Chip (HM-NoC). A localized all-to-all NoC then distributes the targeted operands from the routers directly to the grouped PEs to execute the convolution primitives (Chen *et al.*, 2019).

Figure 7 – Eyeriss v2 top-level architecture illustrating the hierarchical subdivision of GLB, Router, and PE clusters.



Source: Extracted from Chen *et al.* (2019).

This hierarchical topology diverges structurally from the first-generation Eyeriss, which relied on a monolithic global buffer connected to a single, flat PE array via a multicast NoC (Chen *et al.*, 2019). By decentralizing physical storage and arithmetic logic, Eyeriss v2 confines expensive all-to-all communication strictly to the localized PE subgroups (Chen *et al.*, 2019). This modification resolves previous bandwidth limitations by enabling the NoC to be configured per layer to alternate between high-bandwidth unicast delivery for low-reuse workloads and multicast delivery for highly reusable operands (Chen *et al.*, 2019). Consequently, the hierarchical architecture sustains PE utilization across diverse tensor geometries (Chen *et al.*, 2019).

Although these intra-layer dataflows optimize operation scheduling within individual layers, conventional sequential execution mandates transferring intermediate feature maps to and from off-chip DRAM. In early convolutional layers, the memory footprint of these intermediate feature maps frequently exceeds the combined data volume of the filter weights and original inputs, dominating the total memory traffic (Alwani *et al.*, 2016). Cross-layer processing resolves this data-movement limitation by retaining intermediate activations within the on-chip memory hierarchy, enabling direct consumption by the subsequent layer and circumventing external memory access (Sze *et al.*, 2020). Mitigating this specific inter-layer traffic necessitates specialized scheduling methodologies that alter the standard sequential evaluation order of the network.

Fused-layer accelerators operationalize cross-layer processing by evaluating a sliding window of inputs through multiple layers concurrently, thereby averting the full materialization of intermediate feature maps (Alwani *et al.*, 2016). Implementing this localized dataflow necessitates the management of overlapping receptive fields, establishing a fundamental architectural trade-off between redundantly recomputing intermediate values and allocating supplemental on-chip cache (Alwani *et al.*, 2016). While fused-layer configurations employ complex scheduling over a shared compute array to retain data, alternative topologies distribute the computational layers spatially.

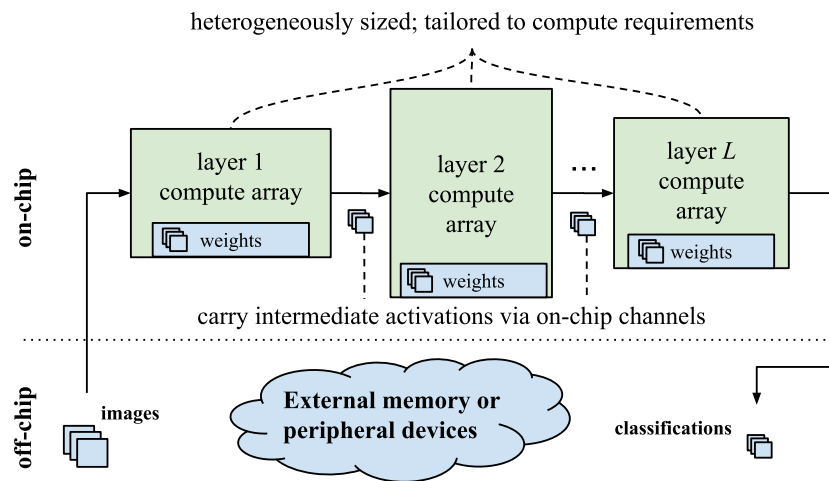
2.3.2 Streaming Architectures and Weight Residency

Streaming architectures generalize the cross-layer paradigm by interconnecting dedicated, structurally independent layer engines via continuous activation streams. This macroscopic pipelining establishes inter-layer data bandwidth as a primary architectural limitation, given that each engine must sustain the operational throughput of the subsequent hardware block. Within this context, the present thesis evaluates a single MVU; the source of its input stream may constitute either external memory or an upstream layer engine. Quantifying the compute-memory trade-offs of such a system requires isolating a single streaming datapath and modeling its performance under explicit operand-delivery constraints.

As illustrated in Figure 8, streaming architectures comprise dedicated, structurally independent hardware engines for individual neural network layers (Blott *et al.*, 2018). The FINN framework operationalizes this topology by lowering convolutional and fully-connected opera-

tions into matrix-vector products, which a Matrix-Vector Unit (MVU) computes (Umuroglu *et al.*, 2017). The original FINN framework designated this primitive as the Matrix-Vector-Threshold Unit to denote the integration of activation logic directly within the datapath; subsequent literature, including FINN-R, abbreviated the nomenclature to MVU, a convention adopted in this thesis (Umuroglu *et al.*, 2017; Blott *et al.*, 2018). Sustaining continuous data throughput in this configuration relies on retaining all associated layer weights within distributed on-chip memory (Umuroglu *et al.*, 2017). Expanding this generation methodology, FINN-R scales the arithmetic logic to support variable, reduced-precision formats (Blott *et al.*, 2018). Because macroscopic streaming pipelines instantiate heterogeneous, layer-specific MVUs rather than uniform spatial arrays, establishing rate-balanced execution requires the proportional allocation of input memory bandwidth, physical compute units, and internal storage capacities for each individual engine (Umuroglu *et al.*, 2017; Blott *et al.*, 2018). Quantifying the performance limits of these distinct datapaths depends on assigning these specific internal microarchitectural resources. Resolving this structural dependency forms the primary objective of this thesis.

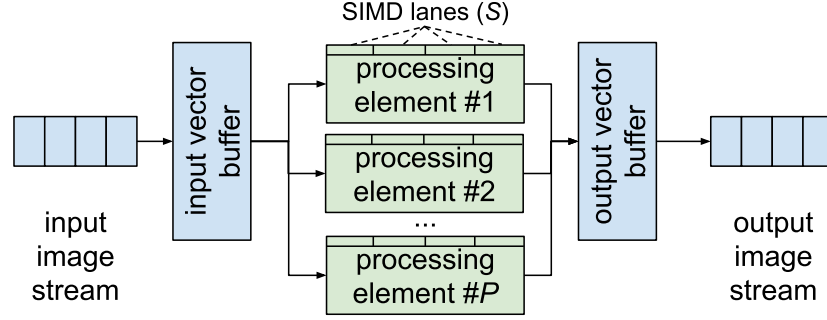
Figure 8 – Dataflow Architecture with Per-Layer Tailored Compute Arrays, On-Chip Weights and Activations.



Source: Extracted from Blott *et al.* (2018).

Figure 9 defines the native spatial parallelism parameters of the individual MVU: P quantifies the parallel processing elements, corresponding to the output channels computed per clock cycle, and S designates the SIMD fan-in, dictating the parallel dot-product lanes (Umuroglu *et al.*, 2017). When logical weight matrix dimensions exceed the physical P and S allocations, the hardware time-multiplexes the computation (Umuroglu *et al.*, 2017). This temporal folding mechanism sequentially accumulates partial results across multiple clock cycles to complete the matrix-vector product (Umuroglu *et al.*, 2017). Adjusting these parameters permits the exchange of physical area for execution time while preserving the continuous dataflow necessary for low-latency evaluation.

Figure 9 – Overview of the MVU.



Source: Extracted from Umuroglu *et al.* (2017).

While the FINN framework validates the layer-level streaming paradigm using customized compute arrays, Microsoft Project Brainwave establishes the industrial precedent for applying permanent weight residency to cloud-scale Neural Processing Units (NPUs). Pinning model parameters across distributed on-chip SRAMs eliminates external memory retrieval delays, facilitating high-throughput, batch-1 streaming inference (Fowers *et al.*, 2018). Diverging from extensively unrolled spatial arrays, the architecture proposed in this thesis adopts this weight-resident strategy but computes output channels sequentially via a temporal folding schedule over a singular datapath (i.e., $P = 1$).

2.4 Final Considerations

The reviewed literature establishes two constraints that jointly govern the design space explored in Chapter 3. First, weight-resident streaming architectures — as realized in FINN and Project Brainwave — eliminate off-chip operand retrieval during inference by pinning parameters in distributed on-chip registers, shifting the primary performance variable from memory access frequency to the physical capacity and bandwidth of localized storage. Second, the Extended Roofline model and EDP provide the quantitative instruments to evaluate that trade-off: the Roofline isolates the operational intensity threshold at which memory delivery, rather than arithmetic throughput, limits performance; EDP penalizes configurations that reduce energy by sacrificing execution speed. Together, these frameworks define the axes along which the parameterized MVU is evaluated — the degree to which expanding SIMD width and register storage translates into proportional throughput gains before silicon area and memory delivery constraints render further scaling counterproductive.

3 DEVELOPMENT

3.1 Initial Considerations

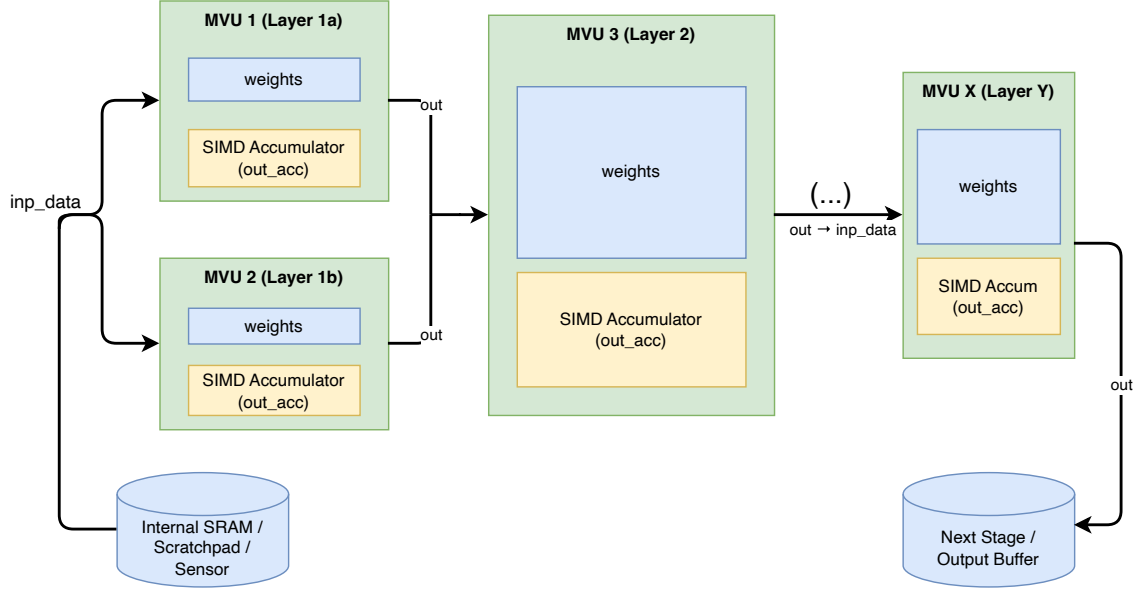
This chapter details the methodological execution of the architectural evaluation. It systematically documents the register-transfer level construction of the MVU, the deployment of the functional verification environment, and the implementation of constrained memory simulation models. Following the methodological framework, the chapter analyzes the extracted logic synthesis metrics, mapping physical hardware constraints against the Extended Roofline Model to formulate definitive conclusions regarding system-level scalability and energy proportionality.

3.2 The Parameterizable MVU: Architecture and Evaluation Methodology

The MVU functions as a dedicated hardware engine for computing neural network layers. The baseline RTL datapath utilized in this study adapts an initial implementation authored by Eduardo Sperle Honorato, which translates the structural principles of the FINN framework (Umuroglu *et al.*, 2017) into synthesizable hardware logic based on (Alam *et al.*, 2023). Modifications to this baseline source code enable parameterized configuration, dynamic testbench integration, and physical synthesis. Furthermore, functional verification of this datapath relies on a proprietary Python-based golden model, developed sequentially to emulate the unit’s precise integer quantization mechanisms.

At the system level, illustrated in Figure 10, heterogeneous MVUs interconnect to form a streaming computational graph. This topology replaces the store-and-reload paradigm—which writes intermediate activations to off-chip DRAM—with direct inter-MVU data transfers. Each unit evaluates a specific network layer and forwards the resulting activations to the subsequent stage. Initial input vectors originate from localized storage, such as SRAM buffers or scratchpad memory, or directly from sensor interfaces. Although the default configuration assigns a single MVU per layer, spatial parallelization enables multiple units to process the same input stream concurrently. This replication distributes output channel computation at the cost of increased hardware area and localized memory bandwidth utilization.

Figure 10 – Macroscopic view of a Deep Neural Network of Y layers running on X MVUs.



Source: Author.

Pipeline synchronization determines the aggregate throughput of this cascaded architecture. While system-level pipelining reduces external memory requests for intermediate feature maps, bandwidth limitations shift to the network boundaries (data ingestion and extraction) and internal inter-stage buffers where computational throughput exceeds data transfer rates. Aggregate performance remains bounded by the slowest processing stage; insufficient data delivery at any interface forces the corresponding datapath into idle cycles. Isolating a single MVU to characterize its operational intensity under constrained data delivery provides the analytical foundation for system-level scaling. Determining the exact memory bandwidth required to sustain compute-bound execution for an individual node informs the design of inter-MVU interconnects and the global memory subsystem supporting a cascaded network.

To evaluate this architecture, the project employs a mixed-abstraction methodology divided into three sequential phases. First, the parameterized MVU is implemented in SystemVerilog. Functional verification relies on a Python-based golden model to validate integer arithmetic, followed by an initial testbench execution under a Parallel Random-Access Machine (PRAM) model (Savitch; Stimson, 1979) to confirm core logic functionality independent of memory timing. Second, the microarchitectural evaluation introduces parameterized SRAM constraints. The testbench applies specific bandwidth limits and access latencies to simulate pipeline stalls caused by insufficient operand delivery, extracting empirical cycle counts and computational throughput (GMAC/s). Finally, physical synthesis utilizing the Synopsys SAED 14nm FinFET library (Synopsys,) extracts theoretical maximum clock frequencies, silicon area, and power dissipation. These physical metrics parameterize the Extended Roofline (Cabezas; Püschel, 2014;

Williams; Waterman; Patterson, 2009) and Energy-Delay Product (EDP) (Gonzalez; Horowitz, 1996) models, providing a framework to quantify the trade-offs between hardware parallelization, memory availability, and energy proportionality across the design space.

3.3 Implementation

3.3.1 Development Environment and Toolchain

Code debugging was facilitated by large language models, including GPT-5.2 (via GitHub Copilot), Gemini 3.1 Pro, and Claude 4.6 Sonnet. The development environment utilized Visual Studio Code (version 1.121.0), with remote SSH execution on a virtual machine hosted by the CADMicro facility at the Federal University of Rio Grande do Sul (UFRGS). Access to the Synopsys toolchain was granted through authorized use of this infrastructure. The complete source code is provided in Appendix B.

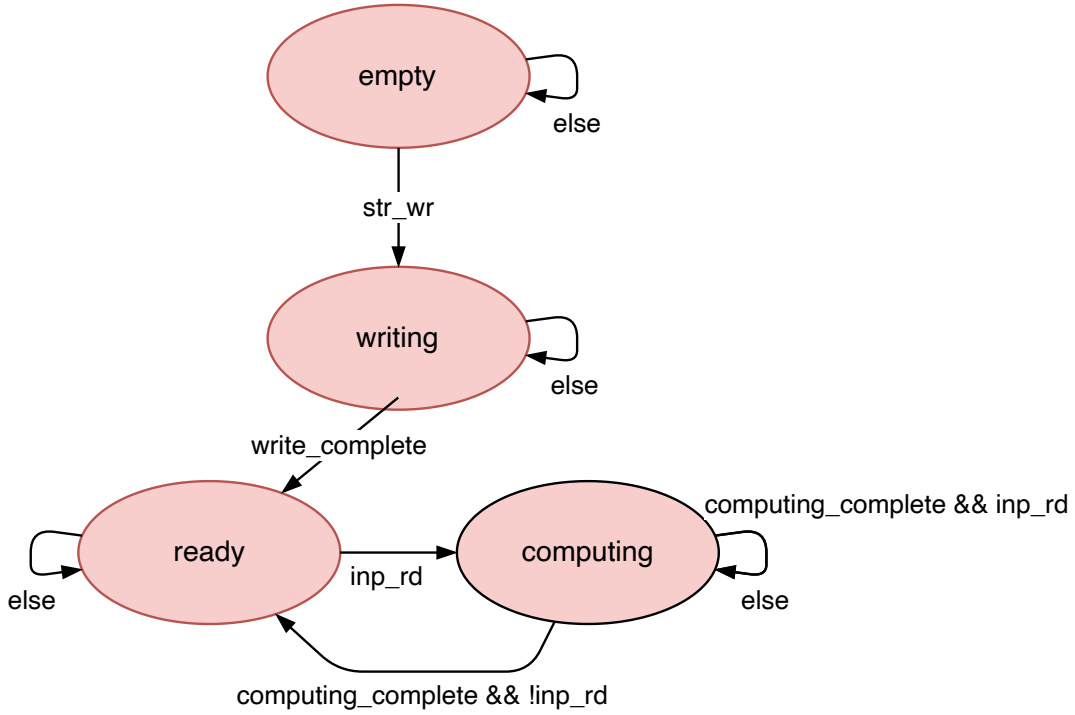
3.3.2 RTL Implementation of the MVU Datapath

To ensure synthesis viability and microarchitectural scalability, significant modifications were applied to the baseline RTL code. The most consequential revision eliminated combinational latch inference caused by blocking assignments inside sequential logic blocks, replacing them with a next-state variable pattern that confines all register updates to synchronous flip-flop boundaries, preserving timing closure across the explored design space. A functional correctness error was corrected in which the activation function and requantization logic executed on every fold iteration rather than exclusively on the terminal fold, producing invalid outputs for any configuration requiring more than one temporal fold. State machine pointer updates were restructured to eliminate same-cycle read-after-write race conditions in the weight-loading phase, aligning boundary behavior with the parameterized hardware allocation. A dedicated data purging mechanism was introduced to zero accumulation registers, output arrays, and routing counters atomically upon each new input ingestion, preventing stale operand contamination across sequential inference frames. The output packing stage was restructured as an unconditional loop over an unpacked array representation, eliminating variable-index bit-slice multiplexer chains that would otherwise degrade maximum operational frequency. Finally, simulation-only debug ports and signals were isolated behind preprocessor synthesis guards, and all datapath parameters were unified through a shared configuration header to support parameterized testbench integration across the full design space sweep.

Logic simulation was conducted using the Synopsys Verilog Compiler Simulator (VCS) version W-2024.09-SP2-3. The implementation adheres to the IEEE 1800-2017 SystemVerilog standard (IEEE Computer Society and the IEEE Standards Association Corporate Advisory Group, 2018), selected to ensure compatibility with the deployed EDA toolchain while providing sufficient language features for the parameterized design.

The internal control flow of the MVU is governed by a Finite State Machine (FSM), illustrated in Figure 11. This control logic coordinates the sequence of operations through four distinct states: *Empty*, *Writing*, *Ready*, and *Computing*.

Figure 11 – MVU’s Finite State Machine.



Source: Author.

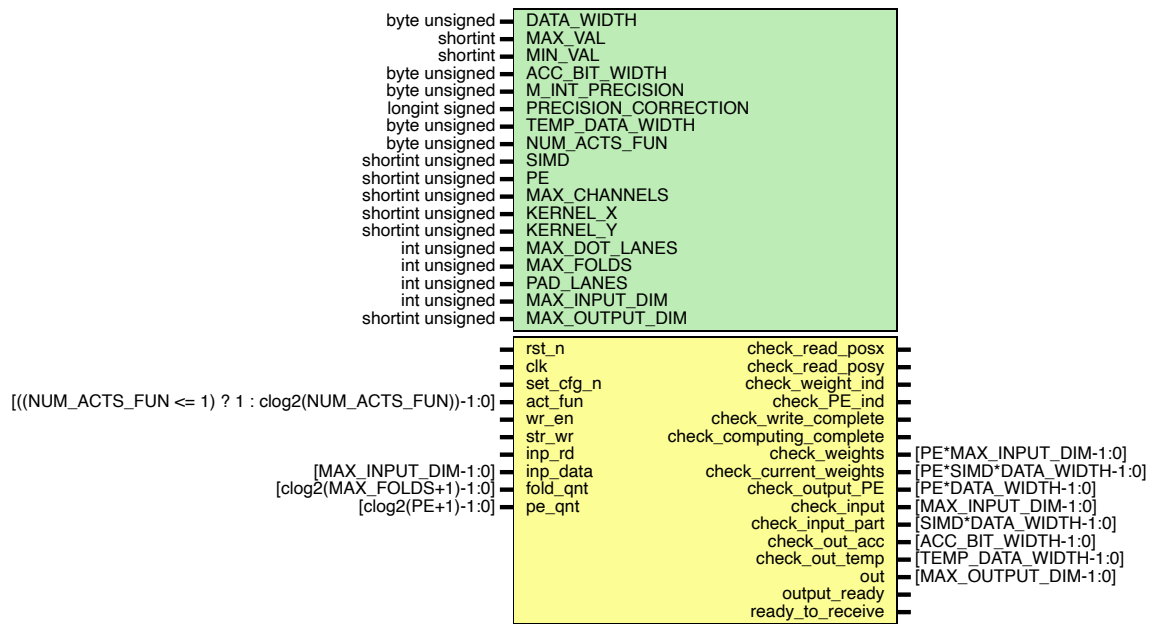
The *Empty* state functions as the default initialization, where the unit remains idle until a configuration command is asserted. Upon receiving the write-enable signal, the logic transitions to the *Writing* state. During this phase, the unit is configured for the target neural network layer by sequentially loading the weight matrix into the internal registers in SIMD-sized segments. The control logic executes $pe_qnt_reg \times fold_qnt_reg$ clock cycles. Following the storage of the final weight segment, the logic allocates a single appended cycle to capture the requantization scale from the multiplexed input bus. The complete configuration latency therefore requires exactly $pe_qnt_reg \times fold_qnt_reg + 1$ clock cycles. Because the hardware relies on weight-resident execution, this load phase occurs once per model initialization. Consequently, the latency incurred during the *Writing* state does not impede steady-state inference throughput, provided the target model remains constant during execution.

Once weight initialization concludes, the FSM enters the *Ready* state. In this configuration, the unit stands by for operand delivery, awaiting an input vector from a localized memory element (SRAM or scratchpad), an external sensor, or a preceding MVU in the cascaded pipeline.

Upon asserting the input-ready signal, the control logic transitions to the *Computing* state, activating the datapath to execute the required multiply-accumulate operations. Following the completion of the vector computation, the system returns to the *Ready* state to process subsequent input frames.

The external interface of the MVU, encompassing its input and output ports alongside parameterized configurations, is depicted in Figure 12.

Figure 12 – Block diagram of the MVU highlighting input/output interfaces and structural parameters.



Source: Author.

Global synchronization is maintained through the `clk` port, while the active-low `rst_n` signal provides a system reset. Asserting `rst_n` forces the FSM into the *Empty* state and clears all internal control variables. Notably, this reset operation deliberately excludes the localized weight memory. Bypassing memory clearance prevents the routing and fan-out penalties associated with simultaneously resetting high-density register arrays. Because subsequent layer configurations strictly overwrite these addresses sequentially, uninitialized memory data poses no functional risk to the datapath.

Layer-specific operational boundaries are established via the active-low `set_cfg_n` control pin. When asserted, the unit latches three configuration parameters: `pe_qnt` defining the number of active output channels, `fold_qnt` specifying the required temporal SIMD folds per channel, and `act_fun` selecting the post-processing activation function (e.g., identity or ReLU). The ReLU function is prioritized as it is standard in edge deployments (e.g., TinyML) and computes using simple combinational comparators, circumventing the logic overhead required by LUTs. A structural limitation exists within this configuration protocol since the control

logic evaluates `set_cfg_n` independently of the state progression. Updating these parameters without resetting the system alters operational bounds without returning the FSM to the *Writing* state. If external control logic modifies these values during active inference, the system may read invalid memory segments or prematurely truncate channel computations. Consequently, the current architecture mandates a static execution model, requiring configuration updates to strictly accompany a hard reset between distinct NN layers.

Data ingestion is governed by three primary interface pins. The `str_wr` (start write) signal triggers the FSM transition from the *Empty* to the *Writing* state. During this initialization phase, the `wr_en` (write enable) flag validates data transmission, permitting the sequential latching of weights and quantization scales. The `inp_data` bus operates as a multiplexed conduit, delivering structural weights during configuration and routing input activations into the `inp_data_reg` register during standard inference.

Finally, the hardware designates specific ports for result extraction and pipeline handshaking. A suite of diagnostic ports, prefixed with `check_`, exposes internal state transitions exclusively for simulation and functional verification. To conserve silicon area and optimize critical path timing, physical synthesis directives prune these debug signals prior to hardware compilation. The core functional outputs consist of the `out` bus, which broadcasts the completed activation vectors; `output_ready`, which asserts the validity of the computed data on the `out` bus; and `ready_to_receive`, an upstream signaling flag indicating the unit has entered the *Ready* state and is provisioned to accept subsequent input frames.

The architectural configuration of the MVU is defined by a set of parameterizable variables that govern data precision, datapath parallelism, and memory allocation.

Data representations throughout the datapath are governed by the `DATA_WIDTH` parameter. This variable establishes the bit-width of both the input activations and the final output vectors, typically set to 8 bits for quantized deep neural network inference (Chen *et al.*, 2019). Derived from this configuration, the `MAX_VAL` and `MIN_VAL` parameters dynamically establish the upper and lower mathematical bounds used during the clipping phase of integer requantization.

Intermediate arithmetic calculations utilize independent precision parameters to prevent overflow. The `ACC_BIT_WIDTH` defines the capacity of the local accumulators. While reducing this width conserves silicon area, the architecture defaults to 32 bits to maintain alignment with standard base-2 memory structures and to provide sufficient accumulation headroom. Following accumulation, the final down-scaling operation relies on the parameter `M_INT_PRECISION`. This parameter configures the bit-width of the fixed-point fractional scale factor to 32 bits, aligning with the standard precision utilized by most quantized neural network frameworks to prevent catastrophic accuracy degradation. Alternative quantization schemes, such as Block Floating Point (BFP)—which shares an exponent across multiple variables to preserve dynamic range (Sze *et al.*, 2020)—are excluded from this implementation to constrain architectural scope. To ensure mathematical accuracy during the arithmetic right-shift of the scaling phase,

`PRECISION_CORRECTION` applies a rounding constant derived from the precision scale. The aggregate width required for this transient calculation is stored in `TEMP_DATA_WIDTH`. `NUM_ACTS_FUN` is used to store the amount of hardware-implemented activation functions.

Computational throughput and loop execution are defined by two primary variables: `SIMD` and `PE`. The `SIMD` parameter determines the width of the vector datapath, establishing the exact number of multiply-accumulate operations executed per clock cycle. The `PE` parameter denotes the number of logical Processing Elements, corresponding to the total output channels or filters computed for a given layer (equivalent to M in Table 3). Unlike spatially parallel architectures that instantiate discrete physical engines for each filter, this datapath evaluates the `PE` iterations sequentially over a single physical `SIMD` array.

Input tensor geometries dictate the memory boundaries of the unit. These geometries are specified by `MAX_CHANNELS` (input depth), alongside the spatial dimensions `KERNEL_X` and `KERNEL_Y`. Convolutional kernels conventionally utilize odd dimensions (e.g., 3×3 or 5×5) to maintain spatial symmetry around a central pixel (Russell; Norvig, 2021). To process these multi-dimensional tensors, the architectural scope assumes the integration of an upstream memory controller or line-buffer mechanism. This external logic extracts the localized sliding window from the broader input feature map and flattens it into a contiguous one-dimensional vector prior to delivery. The MVU abstracts this spatial formatting, ingesting only the isolated receptive field required for a single inference cycle. For FC layers, this input vector inherently encompasses the entire activation space, neutralizing the need for discrete spatial windowing. The total number of scalar multiplications required per filter evaluation is defined as `MAX_DOT_LANES`, calculated using Equation 3.1.

$$\text{MAX_DOT_LANES} = \text{MAX_CHANNELS} \times \text{KERNEL_X} \times \text{KERNEL_Y} \quad (3.1)$$

Translating spatial convolutions into linear memory arrays introduces alignment discrepancies. Odd-dimensioned kernels yield non-power-of-two products (e.g., a 3×3 kernel dictates 9 elements per channel). Multiplying this by an arbitrary channel depth rarely aligns with standard base-2 boundaries. To optimize hardware utilization, dimensions are frequently tuned; for example, configuring 56 input channels yields 504 elements, closely approaching a 512-bit alignment.

To process arbitrary vector lengths across a fixed-width `SIMD` datapath, the execution schedule is divided into temporal folds. The `MAX_FOLDS` parameter calculates the total clock cycles required to process the flattened vector, defined mathematically as the ceiling of the division in Equation 3.2.

$$\text{MAX_FOLDS} = \left\lceil \frac{\text{MAX_DOT_LANES}}{\text{SIMD}} \right\rceil \quad (3.2)$$

To ensure the final execution cycle receives a fully populated operand vector, the hardware pads the raw input. The `PAD_LANES` parameter defines this aligned element count, as shown in Equation 3.3.

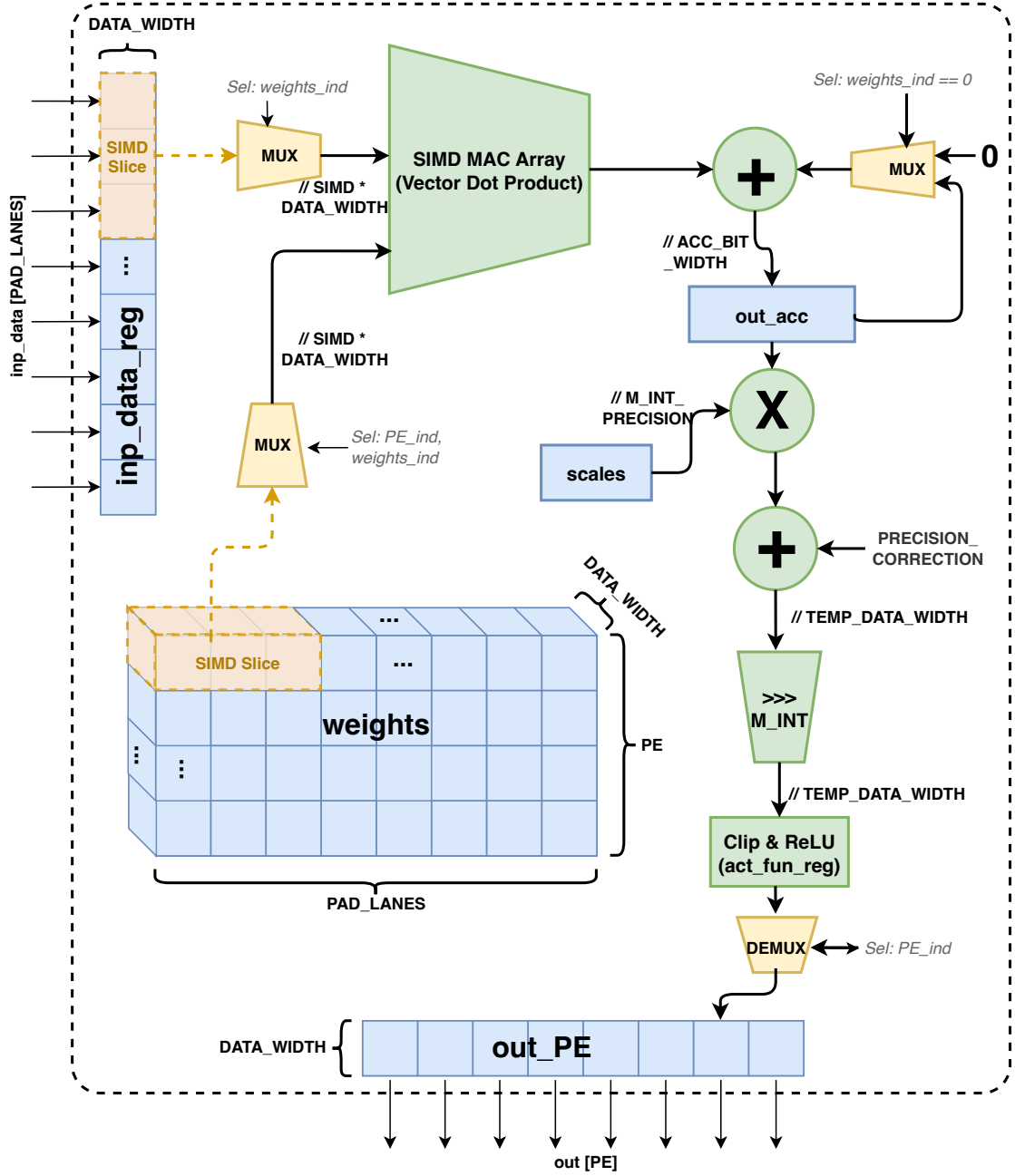
$$\text{PAD_LANES} = \text{MAX_FOLDS} \times \text{SIMD} \quad (3.3)$$

Finally, these aligned vector lengths define the physical capacity of the internal memory registers. The `MAX_INPUT_DIM` parameter establishes the total bit-width allocated to store the padded input tensor, computed in Equation 3.4, while `MAX_OUTPUT_DIM` provisions the accumulation registers required to store the resultant vectors across all logical PEs.

$$\text{MAX_INPUT_DIM} = \text{PAD_LANES} \times \text{DATA_WIDTH} \quad (3.4)$$

The internal microarchitecture of the MVU, illustrating the physical datapath and memory structures, is detailed in Figure 13. The core logic executes a temporally folded dot product, sequentially processing input vectors against resident weight matrices to produce quantized output channels.

Figure 13 – Internal Architectural View of an MVU.



Source: Author.

Memory allocation within the unit isolates static layer parameters from dynamic inference data. During the configuration phase, structural weights are loaded into the 2D array register in discrete, SIMD-sized segments. Conversely, the datapath establishes a single scalar register to hold the requantization scale for the entire layer. The architecture implements strict per-layer quantization rather than per-channel (per-filter) quantization. This design aligns with the operational constraints of prevalent post-training quantized models such as constrained MobileNets(Jacob *et al.*, 2017), thereby avoiding the logic overhead associated with maintaining

discrete scales for individual output channels.

During active inference, the system captures the fully padded input vector into the data register within a single clock cycle. This data capture operation is governed by strict conditional gating; the `inp_data_reg` updates exclusively when the unit resides in the *Ready* state or upon the synchronized completion of the preceding vector within the *Computing* state. Coinciding with this latching event, the hardware synchronously drives the internal accumulation registers, output arrays, and routing counters to zero. Executing this systematic flush purges stale operational data from the datapath, preventing residual accumulation overlap between discrete inference frames. Loading maximum-dimension inputs concurrently abstracts realistic bus bandwidth limitations. However, it accurately models the behavior of a discrete MVU instance supported by intermediate buffers or wide inter-layer interconnects in a pipelined topology.

The computational pipeline activates during the *Computing* state. Because the vector length exceeds the physical SIMD width, structural multiplexers segment the input register and the corresponding weight array into processable blocks. The control logic utilizes a temporal fold index (k) to advance this sliding window. The extracted vectors route into the SIMD MAC array, which computes the inner product of the segment. This partial result feeds into an adder coupled with an accumulation register. A synchronous datapath multiplexer prevents inter-channel contamination; when $k = 0$, the multiplexer routes a constant zero into the adder, discarding the previous channel's sum. For subsequent iterations ($k > 0$), the multiplexer routes the accumulation register output to finalize the temporal partial sum. Let S denote the SIMD width, p denote the target output channel, W represent the weight matrix, and x represent the input vector. Equation 3.5 defines this hardware accumulation step:

$$acc_k = \begin{cases} 0 & \text{if } k = 0 \\ acc_{k-1} & \text{if } k > 0 \end{cases} + \sum_{i=0}^{S-1} W_{p, kS+i} \cdot x_{kS+i} \quad (3.5)$$

Notably, this implementation omits the addition of bias terms following the MAC accumulation. Excluding bias logic constrains the architectural scope, isolating the performance metrics of the primary matrix-vector datapath.

Upon reaching the final fold, the completed scalar dot product progresses to the requantization stage. The architecture executes a fixed-point scaling operation to remap the high-precision 32-bit accumulator back to the target output format. The hardware multiplies the accumulator by the stored per-layer scale (s), adds a precision rounding constant (c), and applies an arithmetic right shift governed by the integer precision parameter (M). Equation 3.6 defines this intermediate transform:

$$y_{temp} = (acc \cdot s + c) \gg M \quad (3.6)$$

The datapath immediately subjects this intermediate value to boundary clipping and

activation functions. Based on the selected activation mode, the hardware bounds the signal within the maximum and minimum representable values defined by the target data width. If the Rectified Linear Unit (ReLU) function is enabled, the logic truncates all negative values. Equation 3.7 models this combinational post-processing stage:

$$out_p = \begin{cases} \text{MAX_VAL} & \text{if } y_{temp} > \text{MAX_VAL} \\ 0 & \text{if } y_{temp} < 0 \text{ and ReLU Enabled} \\ \text{MIN_VAL} & \text{if } y_{temp} < \text{MIN_VAL} \\ y_{temp} & \text{otherwise} \end{cases} \quad (3.7)$$

From a microarchitectural perspective, the datapath consolidates the final segmental MAC operation, the fixed-point requantization, and the boundary activation into a unified combinational block. During the terminal fold iteration ($k = \text{fold_qnt_reg} - 1$), the resultant sum propagates directly through the scaling multipliers, shifting logic, and combinational comparators prior to synchronous registration. While eliminating multi-cycle pipeline latency, this cascaded arithmetic depth establishes the critical path of the design, ultimately constraining the maximum achievable operating frequency of the unit.

Compounding the limitations of this non-pipelined critical path, the datapath does not exploit activation sparsity. Advanced neural accelerators, such as Eyeriss (Chen; Emer; Sze, 2016), leverage the zero-valued outputs inherently generated by ReLU functions to bypass redundant multiplications, thereby conserving dynamic power and accelerating execution. The evaluated MVU lacks zero-skipping control logic. As a result, the SIMD array unconditionally computes arithmetic operations for all supplied operands. This dense execution model multiplies both inherent algorithmic zeros and the artificial zero-padding injected to satisfy fold-alignment constraints. While specific architectural modifications—such as clock-gating or dynamic window sizing—could mitigate this padding overhead, the design maintains dense matrix multiplication to restrict the complexity of the baseline implementation.

Finally, a demultiplexer controlled by the current channel index (p) routes the truncated scalar into the appropriate position within the output array register. The control unit increments the channel index and resets the fold index to initiate the computation of the subsequent output channel. Once all programmed logical processing elements complete their computations, the hardware broadcasts the entire array concurrently onto the output bus and asserts the requisite synchronization flags.

3.3.3 Functional Verification via Python Golden Model

Functional verification of the RTL datapath relies on a software-based golden model to generate deterministic test vectors and validate integer arithmetic. The simulation environment utilizes Python (version 3.12.13_1) as the base execution runtime. Tensor operations and

mathematical modeling are governed by PyTorch (version 2.8.0), an open-source deep learning framework. Furthermore, the model implements Brevitas (version 0.12.1), a PyTorch-native library specialized for quantization-aware training. Brevitas is specifically utilized to model the low-precision linear layers, enforce uniform 8-bit quantization bounds, and extract the floating-point scale factors required to calculate the integer requantization multiplier.

The execution of the golden model script parses the specified hardware geometries and outputs four primary file artifacts used to drive the SystemVerilog testbench:

- `weights.txt`: Encodes the static weight matrices. The data is partitioned and zero-padded into SIMD-aligned temporal folds, terminating with the calculated integer requantization multiplier (`Mint`).
- `inputs.txt`: Contains the pseudo-randomly generated input vectors. These vectors are mapped to the target bit-width and formatted to align with the hardware’s folding schedule.
- `expected.txt`: Stores the true computed output activations for each input vector, incorporating hardware-accurate arithmetic right shifts and boundary clipping.
- `cfg.txt`: Exports the static architectural parameters (including unpadded vector length, fold quantities, active output channels, and the activation function flag) required to accurately initialize the control logic of the MVU prior to simulation.

To facilitate automated DSE, the script is fully parameterized via command-line arguments. These parameters control the matrix dimensions, the hardware datapath alignment, and the deterministic generation of inference data. Table 4 details the function and structural constraints of each configuration argument.

Table 4 – Golden model configuration parameters.

Parameter	Description
<code>-seed</code>	Random Number Generator seed. Enforces deterministic execution for reproducible verification.
<code>-bit_width</code>	Precision target (default: 8). Defines quantization limits for tensors.
<code>-cin</code>	Number of input channels.
<code>-kx / -ky</code>	Kernel spatial dimensions (width and height).
<code>-pe_qnt</code>	Active output channels (logical PEs). Must conform to the hardware limit.
<code>-simd</code>	Datapath vector width. Must strictly match the RTL configuration for alignment.
<code>-n_inputs</code>	Test batch size. Defines the total number of random input vectors generated.
<code>-out_dir</code>	Target directory path for the generated vector files.

Source: Author.

The structural parameters `-cin`, `-kx`, and `-ky` dictate the aggregate length of the flattened input vector. This derived length must conform to the maximum memory boundaries established by the hardware (`PAD_LANES`). Similarly, the `-simd` parameter operates as a strict architectural constraint; deviations between the script’s SIMD alignment and the physical datapath width result in fatal structural misalignment during memory loading phases.

The execution of the golden model script proceeds through a deterministic data generation pipeline. Initially, the script computes the scalar dot product length and the requisite number of SIMD folds derived from the specified kernel geometries. An FC layer is instantiated utilizing Brevitas quantization primitives. Conforming strictly to the architectural constraints established for the RTL datapath, this software model explicitly omits bias parameters. It also enforces a per-tensor quantization scheme. This configuration ensures that a single floating-point scale factor governs the entire weight matrix, averting the logic overhead associated with per-channel (per-filter) scaling. Random floating-point input matrices are generated and subsequently discretized into 8-bit integer tensors alongside the initialized network weights.

Following tensor generation, mathematical evaluation determines the discrete arithmetic parameters necessary to emulate the datapath’s requantization stage. The framework extracts the floating-point scale factors associated with the input activations (s_{in}), the static weight matrix (s_w), and the expected output tensor (s_{out}). These values combine to calculate the master fractional scaling factor, M , defined in Equation 3.8.

$$M = \frac{s_{in} \cdot s_w}{s_{out}} \quad (3.8)$$

To mirror the fixed-point arithmetic of the hardware multiplier, M undergoes conversion into a high-precision integer, M_{int} , scaling by a factor of 2^N , where N represents the target internal precision width (configured to 32 bits). The script then computes the exact integer matrix multiplication corresponding to the datapath accumulation. The accumulated sums are multiplied by M_{int} , adjusted by a rounding constant, and arithmetically right-shifted by N bits, exactly like inside the MVU. Subsequent boundary logic clips the scaled integers. If the ReLU activation function is active, negative values truncate to zero; otherwise, outputs saturate strictly to the standard 8-bit signed integer limits.

The parameterized SystemVerilog testbench environment is designed to initialize the RTL module, simulate sequential external inputs, and validate the computed outputs against the predefined expected values generated by the Python golden model.

Execution begins with a compile-time and run-time setup phase. The testbench extracts structural dimensions—such as input channels, kernel sizes, and active output channels—via command-line arguments (`plusargs`). These parameters dictate the specific test vectors to be loaded from the designated directory. The environment then calculates the necessary operational constants, notably the required temporal folds (`fold_qnt`), ensuring strict alignment with

the static SIMD boundaries of the hardware. Concurrently, a localized simulation clock is generated based on a parameterized frequency (F_{max}), providing accurate temporal resolution for evaluating critical path execution and memory-induced latency.

Following parameter resolution, the testbench drives a strict hardware initialization sequence. The datapath undergoes a synchronous reset to clear internal control flags and accumulation registers. The testbench then asserts the configuration pin, compelling the MVU to latch the calculated fold limits, active processing elements, and the specified activation function flag.

With the bounds established, the testbench transitions the hardware into the *Writing* state. The verification environment sequentially parses the formatted `weights.txt` file, streaming the padded integer weights into the input bus across consecutive clock cycles. This load phase simulates a continuous data stream, terminating with the latching of the layer-specific requantization scale.

Upon transitioning into the *Ready* state, the inference validation phase commences. The testbench loops over the pre-generated test batch. The input-ready flag is asserted, triggering the active computation pipeline. The testbench then monitors the `output_ready` synchronization flag. When asserted by the MVU, the testbench captures the resultant 1D output bus.

Finally, validation logic parses the corresponding target activations from `expected.txt` and executes a direct, per-channel integer comparison against the captured hardware output. Discrepancies immediately halt the simulation and trigger an error report detailing the internal state registers to facilitate datapath debugging. If all test vectors compute successfully, the testbench confirms functional equivalence with the software model.

3.3.4 Memory Constraint Modeling

The architectural evaluation scopes the memory constraints strictly to the active inference phase. The weight initialization phase is excluded from the performance analysis. As established in the architectural description, the datapath operates on a weight-resident model, requiring static parameters to be loaded exclusively during layer initialization. While initialization latency is measurable, it occurs infrequently relative to the continuous execution of inference operations. Furthermore, assigning multiple layers sequentially to a single hardware unit—which would necessitate dynamic weight reloading—is highly inefficient within this cascaded topology. Therefore, the analysis isolates dynamic input ingestion as the primary variable for steady-state throughput.

To evaluate data delivery bottlenecks, the testbench incorporates a parameterized SRAM model. The selection of SRAM over DRAM aligns with standard spatial accelerator architectures (Chen; Emer; Sze, 2016). In a pipelined execution model, external DRAM rarely supplies input vectors directly to the intermediate datapath nodes (Chen; Emer; Sze, 2016). Instead, data propagates through localized SRAM structures, such as inter-layer line buffers (Blott *et al.*,

2018), scratchpad memories (Jouppi *et al.*, 2017), or directly from localized sensor interfaces.

SRAM architecture also permits a linear mathematical abstraction. The memory access time (T_{mem}) is modeled using the transfer equation defined in Equation 3.9, where L denotes the initial access latency, BW represents the aggregate bandwidth, and $S = \text{PAD_LANES}$ specifies the required data volume per inference, equivalent to the padded input vector length.

$$T_{mem} = L + \frac{S}{BW} \quad (3.9)$$

This linear transfer function is valid for SRAM because its digital logic inverter cells maintain state actively, resulting in deterministic access times and an operational cycle time equivalent to its access latency (Patterson; Hennessy, 2020). Conversely, modeling DRAM with this equation fails to capture critical analog and microarchitectural delays. DRAM performance is heavily state-dependent; reading data from an active row buffer (page-mode access) exhibits low latency, while activating a new row imposes severe penalties (Patterson; Hennessy, 2020). Furthermore, DRAM requires destructive reads followed by precharge cycles, and mandates periodic refresh intervals to prevent charge leakage, which unpredictably halts data flow. Finally, peak theoretical bandwidth in DRAM is contingent upon perfect request pipelining across independent physical memory banks to hide precharge latencies. Sustained bank conflicts cause the effective bandwidth to degrade. Because SRAM avoids row activation delays, precharge cycles, and refresh stalls, it conforms accurately to the linear relationship defined in Equation 3.9.

The simulation framework models an ideal overlapping pipeline between data ingestion and datapath computation. The testbench deliberately omits output serialization logic and intermediate First-In-First-Out (FIFO) buffers. While buffering mechanisms temporarily absorb architectural back-pressure during variable execution phases, they introduce transient microarchitectural states that mask fundamental bandwidth constraints during finite-length simulations. By directly exposing the raw computational width of the datapath to the parameterized memory interface, the simulation environment strips away transient buffering artifacts. This isolation ensures that every measured stall cycle is a mathematically pure reflection of the underlying memory wall, yielding a deterministic Roofline evaluation.

During the simulated inference batch, the testbench extracts execution metrics to quantify both architectural throughput and memory dependency. The active computation time in cycles (C_{comp}) is governed by the datapath’s state machine, which sequentially evaluates `pe_qnt` active output channels across `fold_qnt` temporal folds. While this establishes a theoretical minimum of $\text{pe_qnt} \times \text{fold_qnt}$ cycles per inference vector, the serialized control logic requires two additional cycles to return to the *Ready* state and latch the subsequent input. Without a fully pipelined state machine to mask this transition overhead, the effective computation time is defined in Equation 3.10.

$$C_{comp} = (\text{pe_qnt} \times \text{fold_qnt}) + 2 \quad (3.10)$$

The total processing latency (C_{total}) per inference resolves to the maximum value between this execution time (C_{comp}) and the independent memory transfer time (C_{mem}). Under bandwidth-constrained conditions where C_{mem} exceeds C_{comp} , the testbench injects proportional pipeline stalls. This mechanism suspends the subsequent input-ready assertion until the simulated memory subsystem completes the requisite data delivery, accurately modeling the datapath's response to external memory bottlenecks.

To standardize the performance evaluation regardless of input tensor alignment, the computation metric includes padded logic zeros. Zero-padding guarantees that the unaligned input vector matches the fixed boundary of the SIMD array across all folds. While operations involving these artificial zeros yield null arithmetic contributions, the datapath unconditionally processes them. Consequently, performance metrics reflect the gross hardware utilization relative to the SIMD array capacity rather than the net algorithmic efficiency, which fluctuates depending on the tensor dimensions.

Throughput is evaluated using two standard units: Giga-Operations per Second (GOP/s) and GMAC/s. The total number of MAC operations executed per inference is defined strictly by the hardware bounds in Equation 3.11.

$$\text{MACs}_{total} = \text{pe_qnt} \times \text{fold_qnt} \times \text{SIMD} \quad (3.11)$$

Conversely, the calculation for total standard operations incorporates the additional scalar arithmetic occurring within the requantization and activation stages. Specifically, it accounts for the scalar multiplication, the precision correction addition, and the arithmetic right shift, adding a constant factor of 3 operations per output channel. The operation count is defined in Equation 3.12, where a single MAC is counted as two discrete operations (one multiplication, one addition):

$$\text{OPs}_{total} = \text{pe_qnt} \times [(2 \times \text{fold_qnt} \times \text{PAD_LANES}) + 3] \quad (3.12)$$

The effective throughput is calculated by dividing these operation counts by the elapsed time. The elapsed time per inference is modeled as the maximum between the computation latency and the memory delivery latency. Equation 3.13 defines the generic throughput calculation, where C_{idle} represents stall cycles induced by memory transfer delays ($T_{mem} > T_{comp}$), with T_{comp} being the measured active computation time, and F_{clk} is the simulated clock frequency in Hertz. This frequency corresponds to the peak operational limit (F_{max}) extracted during static timing analysis.

$$\text{Throughput} = \frac{\text{OPs}_{total} \text{ (or MACs}_{total}) \cdot F_{clk}}{10^9 \cdot (C_{idle} + C_{comp})} \quad (3.13)$$

In this evaluation, GMAC/s was chosen as the preferred metric; including the +3 scalar overhead found in the GOP/s metric distorts the primary comparative variable, which is the density and utilization of the parallel multiplier array.

The testbench then computes the OI. In architectures utilizing weight-streaming mechanisms, such as the Google TPU v1 (Jouppi *et al.*, 2017), OI is traditionally defined as operations per byte of weight data transferred. However, because the evaluated MVU utilizes a weight-resident architecture where weights remain static during inference, OI is redefined in this work as the number of computations performed per byte of input activation data read.

The OI measured in MACs is fundamentally bounded by the active channel parameter (`pe_qnt`). Because the datapath reuses the entire input vector to compute each of the `pe_qnt` output channels, exactly `pe_qnt` multiply-accumulate operations are executed for every individual scalar value within the input vector. Consequently, the relationship scales linearly. However, calculating OI using standard operations (OPs) introduces a non-linear distortion due to the fixed scalar operations executed during requantization. This mathematical discrepancy further justifies the utilization of GMAC-based metrics for accurate architectural comparison. Equation 3.14 defines the true Operational Intensity of the datapath.

$$\text{OI}_{\text{MAC}} = \frac{\text{MACs}_{total}}{\text{PAD_LANES}} = \frac{\text{pe_qnt} \times \text{fold_qnt} \times \text{SIMD}}{\text{PAD_LANES}}$$

$$\text{OI}_{\text{MAC}} = \text{pe_qnt} \quad (\text{assuming fold_qnt} = \text{MAX_FOLDS, where PAD_LANES} = \text{fold_qnt} \times \text{SIMD}) \quad (3.14)$$

3.3.5 Synthesis and Static Timing Analysis

Automated logic synthesis and Static Timing Analysis (STA) translate the RTL datapath into a physical hardware representation. Synopsys Design Compiler Graphical (Version W-2024.09-SP5-4) executes the logic synthesis, while Synopsys PrimeTime (Version W-2024.09-SP5-2) conducts post-synthesis STA and power estimation. The target technology is the Synopsys SAED 14nm FinFET library.

This evaluation is constrained to preliminary, front-end logic synthesis for an isolated module. Back-end physical design phases, including floorplanning, clock tree synthesis, and placement and routing, are omitted. The extracted metrics represent theoretical baselines, excluding routing capacitance and interconnect delays inherent to finalized multi-node silicon layouts. Furthermore, the analysis utilizes the typical Process, Voltage, and Temperature (PVT) corner. Characterization across extreme physical limits, specifically the Fast-Fast (FF) and Slow-Slow (SS) corners, is omitted. Constraining the evaluation to nominal operating conditions isolates

the targeted microarchitectural variables from silicon-level manufacturing variations, which fall outside the architectural scope of this research.

Synthesis begins by linking the SystemVerilog RTL to the target technology library, providing characterized standard cells. Following compilation and elaboration, the script enforces virtual hierarchies to group high-density register arrays (weight storage, input buffers, accumulators, and output registers). This logical partitioning provides localized visibility into area and power distribution during analysis. The compiler retains the flexibility to dissolve these virtual boundaries if cross-boundary optimization yields superior timing or area metrics. In this implementation, the compiler maintained the weight register hierarchy but amalgamated the remaining logic.

Constraint definition establishes physical limits. The test framework defines a primary system clock period, dictating the upper bound for combinational logic propagation delays. A 0.05 margin of clock uncertainty models temporal jitter and spatial clock skew. Boundary constraints define localized input and output delays alongside a fixed output load capacitance, preventing the compiler from assuming infinite transition times at the module interfaces.

During synthesis, the compiler prioritizes meeting the defined clock constraint. Once the timing constraint is satisfied, algorithms execute technology mapping, selecting standard cells to optimize for area and power dissipation. The tool does not iteratively synthesize to discover the absolute maximum achievable frequency of the architecture. Such characterization requires iterative re-synthesis at progressively tighter timing constraints until timing closure fails. Given the extensive simulation time required for the explored design space, this iterative maximum-frequency synthesis was not performed. The resulting structure is exported as a gate-level Verilog netlist with a standard delay format file.

Physical analysis transitions to PrimeTime for STA and power estimation using a two-pass methodology. The primary pass evaluates the datapath at the target clock frequency without dynamic simulation vectors. To estimate dynamic power, the analysis applies a generic, statistical 10% toggle rate across all input stimuli. Because this estimation relies on probabilistic activity rather than explicit functional simulation waveforms (e.g., Switching Activity Interchange Format (SAIF) or Value Change Dump (VCD) files), it does not capture the precise, data-dependent switching behavior inherent to specific neural network workloads. Consequently, the extracted power figures represent an approximation of active dissipation under generic operational loads, serving as comparative architectural baselines rather than absolute, application-specific measurements.

The secondary pass algorithmically extrapolates the peak operational frequency ($F_{\max}$) of the synthesized logic. PrimeTime extracts the Worst Negative Slack (WNS)—the difference between the Required Arrival Time and the Actual Arrival Time—from the primary timing report. The WNS identifies the topologically longest delay path. The script recalculates the clock period by subtracting the WNS from the initial target period. The constraint engine overwrites the core

clock definition with this newly derived minimum period.

Finally, PrimeTime recalculates power dissipation under this constrained, high-frequency clock model. Because dynamic power scales linearly with frequency, this secondary pass defines the thermal and power metrics utilized throughout the remainder of this evaluation, representing the module operating at its physical switching boundary. This is the value for power used in analysis, to account for a design working at its maximum operating frequency.

3.4 Results

The architectural design space exploration required extensive computational resources, exceeding 200 hours of cumulative compute time. Due to the exponential scaling of logic synthesis, where the largest configurations demanded up to 7.5 hours per execution, extreme bounds were excluded. The evaluation yielded 116 unique, successfully synthesized hardware configurations.

The parameter sweep isolated three core architectural variables: maximum output channels ($PE \in \{8, 16, 32, 64\}$), datapath width ($SIMD \in \{4, 8, 16, 32, 64, 128, 256, 512, 1024\}$), and the maximum input vector length (PAD_LANES). Because input vectors must accommodate a 3×3 spatial kernel, input channel counts were constrained to specific boundaries (7, 14, 28, 56, 113) to maintain SIMD alignment, thereby dictating the PAD_LANES dimension. This way, MAX_DOT_LANES remained the closest possible to powers of 2, at 63, 126, 252, 504 and 1017. All configurations were synthesized to a unified target clock frequency of 70 MHz to enable a correct timing closure for most of the designs considered.

3.4.1 Functional Simulation Validation

The MVU architecture passed functional verification across multiple pseudo-randomized input batches. The testbench verified up to 1000 input iterations per geometric configuration, confirming exact alignment between the SystemVerilog datapath outputs and the Python golden model expected values. Furthermore, the simulation confirmed the determinism of the architectural execution schedule; the active computation latency, C_{comp} , remained constant across every inference, confirming the mathematical reliability of Equation 3.10.

3.4.2 Logic Synthesis

Logic synthesis executed successfully, producing mapped gate-level netlists for all evaluated configurations. The compiler logs generated standard diagnostic warnings inherent to the configured methodology.

Warnings regarding non-driven cells indicate surplus logic generated by the compiler that lacks a fanout to internal registers or external output ports. These disconnected cells represent auxiliary or truncated logic vectors that do not affect the active critical path or functional

validity of the primary datapath. Similarly, warnings documenting signed-to-unsigned datatypes align with the expected truncation obtained during the requantization and clipping phase of the arithmetic pipeline.

The physical metrics extraction generated warnings concerning incomplete internal power characterization for specific standard cells within the SAED14 educational library. Because this is an academic process design kit (PDK) rather than a commercial foundry library, sparse characterization data for niche standard cells is anticipated and does not invalidate the aggregate architectural power baselines.

Following technology mapping, the compiler flagged the global clock net for exceeding standard fanout limits, defaulting to an arbitrary maximum load value for delay calculations. This warning is an expected consequence of front-end logic synthesis. Managing massive fanout across thousands of sequential elements requires Clock Tree Synthesis (CTS)—a dedicated back-end physical design phase that constructs a balanced buffer network to distribute the clock signal with minimal skew. As CTS falls outside the microarchitectural scope of this evaluation, the ideal clock network assumption remains standard and mathematically acceptable for baseline static timing analysis.

3.4.3 Static Timing Analysis

Post-synthesis STA evaluated the netlists without fatal timing violations. Minor diagnostic discrepancies were logged during tool initialization. Warnings regarding the absence of the Library Compiler executable path and mismatched Synopsys Design Constraints (SDC) versions (SDC-2) are artifacts of the specific localized Computer-Aided Design infrastructure versioning and do not impede the algorithmic execution of the delay calculation engine.

PrimeTime also disabled specific timing arcs to prevent infinite loop propagation during path analysis. In combinational circuits containing static structural feedback or hardware-tied constants, the STA engine automatically breaks these loops to permit directed acyclic graph analysis. This optimization ensures accurate critical path extraction without altering the physical structure of the netlist.

3.4.4 Maximum Clock Frequency and Performance

Peak architectural throughput is evaluated during the steady-state streaming phase. This isolates the datapath computation limits from the initial memory fetch latency incurred during the first inference cycle. The performance evaluation utilizes the maximum operational frequency ($F_{\max}$) extracted via STA. Across the 116 configurations, $F_{\max}$ exhibited a wide variance, bounded between 60.0 MHz and 279.3 MHz. Table 5 details the statistical distribution of these results.

Table 5 – Statistical distribution of maximum synthesized frequencies ($F_{\max}$).

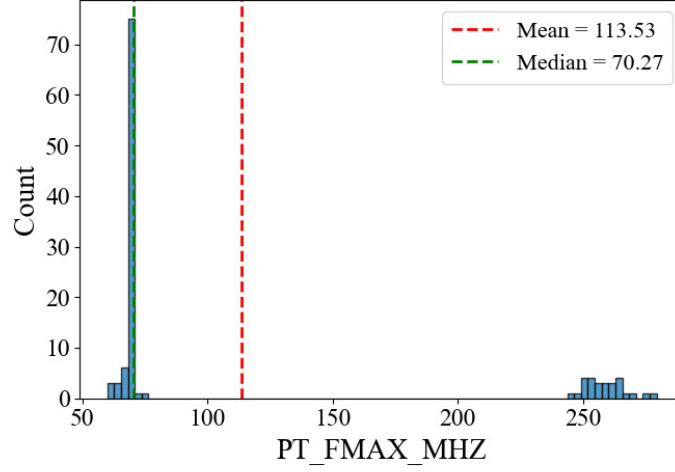
Metric	Value (MHz)
Mean	113.53
Median	70.27
Standard Deviation	80.36
Minimum	59.99
Maximum	279.33
25th Percentile	70.22
75th Percentile	70.82
Total Samples (N)	116

Source: Author.

As illustrated in Figure 14, the frequency distribution bifurcates into two distinct clusters: a primary concentration directly on the 70 MHz target threshold and a secondary distribution above 250 MHz. This clustering is an inherent artifact of constraint-driven logic synthesis. EDA compilers do not balance performance and area equally; rather, they enforce a strict optimization hierarchy. Timing closure is treated as a hard constraint. Once the compiler satisfies the established 70 MHz target, it ceases any further routing speed optimization and reallocates algorithmic effort toward area recovery and dynamic power minimization.

Consequently, the theoretical throughput of configurations clustered at 70 MHz is artificially capped. Extracting their absolute physical limits would require iterative synthesis sweeps with progressively tighter clock targets. However, while this constraint obscures the absolute peak frequency of smaller designs, it establishes a rigorously equitable baseline for subsequent area and power comparisons, as the tool actively minimized these metrics for all configurations meeting the 70 MHz threshold.

Figure 14 – Distribution density of maximum synthesized clock frequencies.



Source: Author.

The theoretical peak throughput is mathematically defined by the intersection of the derived maximum frequency and the datapath width. Equation 3.15 formalizes this limit. The numerator calculates the total MAC operations scaled by the clock frequency, while the denominator accounts for the active cycle count required to process the fully padded input vector, explicitly including the two-cycle state transition overhead.

$$\text{GMAC/s}_{\max} = \frac{\text{PE} \cdot \text{PAD_LANES} \cdot F_{\max}}{10^9 \cdot \left(\frac{2 + \text{PE} \cdot \text{PAD_LANES}}{\text{SIMD}} \right)} = \frac{\text{PE} \cdot \text{PAD_LANES} \cdot F_{\max} \cdot \text{SIMD}}{10^9 \cdot (2 + \text{PE} \cdot \text{PAD_LANES})} \quad (3.15)$$

The subsequent analysis evaluates throughput scaling relative to the datapath width across fixed topological parameters. To facilitate direct comparison, parallel scaling trends are grouped visually.

At low structural complexities (Figure 15), the combinational logic depth remains shallow. Expanding the SIMD width to 128 induces only a fractional frequency degradation (from 264 MHz to 254 MHz). In this unconstrained routing regime, the frequency remains sufficiently high that increasing parallelization yields direct, linear throughput gains, peaking at 26.06 GMAC/s. Conversely, increasing the active output channels to $\text{PE} = 16$ (Figure 16) exposes a critical structural limitation. While the architecture sustains high frequencies up to $\text{SIMD} = 32$ (achieving around 8 GMAC/s), further expansion to $\text{SIMD} = 64$ triggers substantial interconnect congestion. This congestion forces the maximum frequency down to around 75 MHz, which entirely negates the computational advantage of the wider datapath and halves the total throughput to approximately 4.5 GMAC/s. It takes expanding to $\text{SIMD} = 128$ to recover the lost throughput, but at a significant complexity growth.

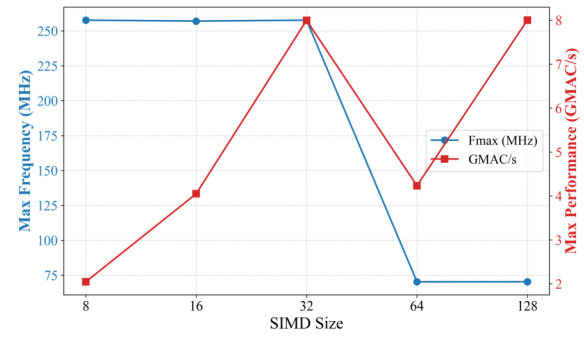
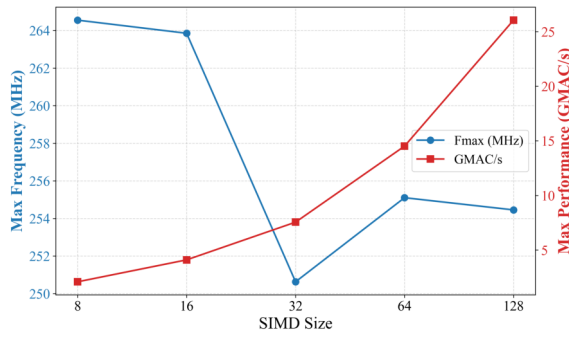
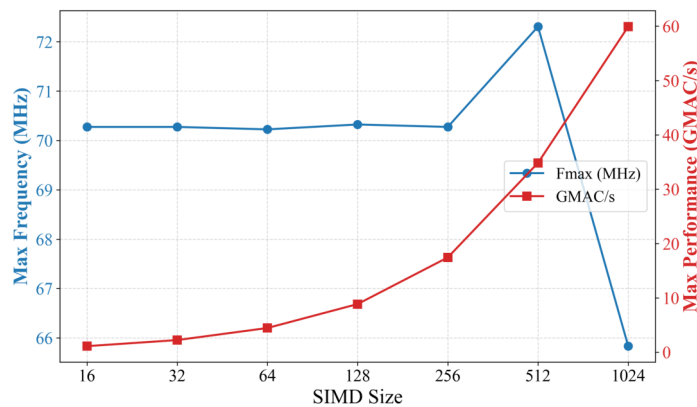


Figure 15 – Throughput scaling at PE=8 and Figure 16 – Throughput scaling at PE=16 and PAD_LANES=128 (14 input channels).

Source: Author.

When both input and output dimensions scale significantly, the synthesis constraint dictates the operational behavior. Figure 17 evaluates a highly asymmetrical configuration processing 113 input channels against 16 output channels. Across all evaluated SIMD widths, internal routing complexity restricts the frequency to the 70 MHz boundary. Because the clock speed is statically tethered to this baseline constraint, expanding the SIMD width up to 1024 produces near-linear throughput scaling, peaking at 59.92 GMAC/s.

Figure 17 – Throughput scaling for high-input asymmetry: PE=16 and PAD_LANES=1024 (113 input channels).



Source: Author.

This pseudo-linear scaling trend persists across dense, symmetrical configurations. Figure 18 evaluates scaling for PE = 32 and 28 input channels, where the synthesized frequency adheres strictly to the 70 MHz target across the entire parameter sweep, yielding uniform performance gains up to 16.94 GMAC/s. In the denser PE = 64 configuration processing 56 input channels

(Figure 19), the layout fails to achieve timing closure starting at $\text{SIMD} = 128$, resulting in frequencies decaying to approximately 60 MHz. However, this gradual frequency decay is mathematically insufficient to outpace the raw computational volume provided by the expansive parallelization array, allowing the theoretical throughput to continue rising to 31.89 GMAC/s.

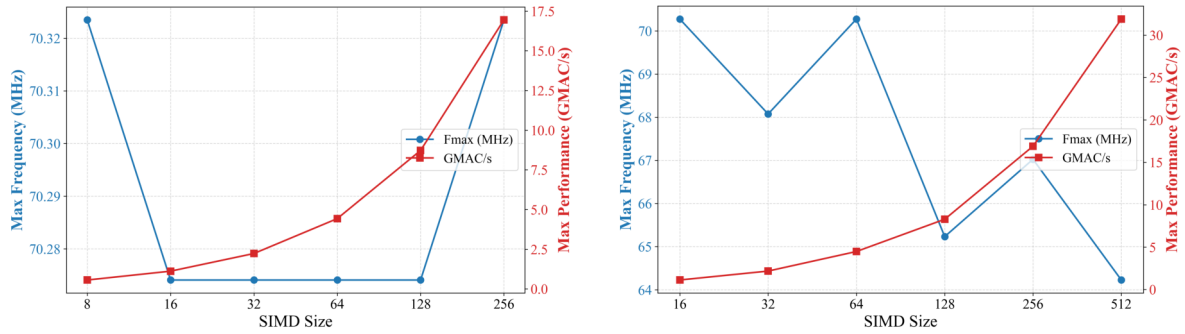


Figure 18 – Throughput scaling at PE=32 and Figure 19 – Throughput scaling at PE=64 and PAD_LANES=256 (28 input channels). PAD_LANES=512 (56 input channels).

Source: Author.

The evaluation demonstrates that while expanding the SIMD datapath theoretically raises the computation threshold, physical routing complexity inevitably compromises the maximum operational frequency. In unconstrained scenarios (low PE counts), parallelization yields robust throughput improvements. However, at critical structural thresholds, combinational congestion incurs frequency penalties significant enough to degrade total performance, underscoring that wider datapaths do not guarantee faster execution.

For dense configurations, the static 70 MHz synthesis constraint flattens the frequency scaling curve. This artificial cap masks the true interconnect penalties, making aggressive SIMD expansion appear uniformly beneficial on paper. However, achieving these theoretical throughput maximums assumes the physical silicon can successfully distribute a synchronized clock signal across a massive, highly congested logic array without timing violations. Because routing such a complex clock tree is physically prohibitive, evaluating the true viability of these configurations requires transitioning the analysis from theoretical throughput to structural efficiency. This transition explicitly compares the exponential growth of silicon area and power dissipation against these theoretical performance gains.

3.4.5 Silicon Area Analysis

In contrast to the static operational targets that constrained the timing evaluation, the synthesis compiler optimizes physical layout as a continuous objective function, minimizing logic gates and sequential elements without a predefined floor. Consequently, the extracted area

metrics exhibit a distributed continuum across the design space. The total synthesized area ranges from $9,882 \mu\text{m}^2$ to over $700,000 \mu\text{m}^2$, reflecting the vast structural divergence between the parameterized extremes. Table 6 details the statistical distribution.

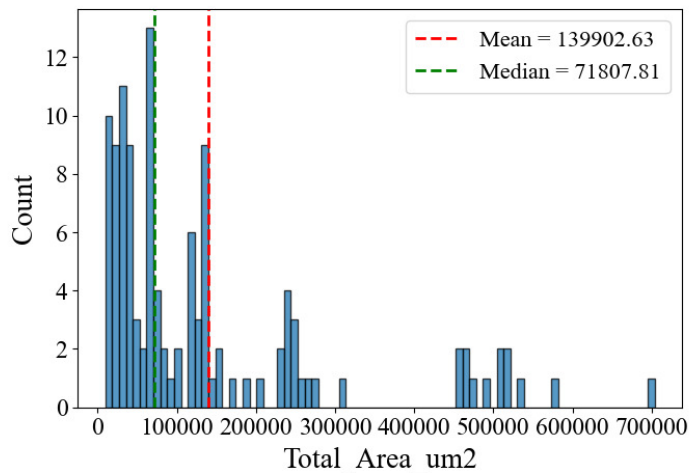
Table 6 – Statistical distribution of total synthesized area.

Metric	Value (μm^2)
Mean	139,902.63
Median	71,807.81
Standard Deviation	152,703.47
Minimum	9,882.81
Maximum	703,586.64
25th Percentile	35,698.52
75th Percentile	158,231.34
Total Samples (N)	116

Source: Author.

The density distribution (Figure 20) lacks the artificial clustering observed in the frequency analysis. Instead, it demonstrates a logarithmic decay typical of architectural scaling, where the majority of viable parameter combinations map to the lower and middle area quartiles, while extreme tensor dimensions produce sparse, high-cost outliers.

Figure 20 – Density distribution of total synthesized silicon area.



Source: Author.

3.4.5.1 SIMD Scaling versus Logic Partitioning

To isolate the structural costs of parallelization, the analysis partitions the total footprint into combinational logic (e.g., multiplier arrays, adders, multiplexers) and non-combinational logic (e.g., flip-flops, memory registers).

At lower tensor dimensions, scaling the parallel datapath width (SIMD) induces a proportional expansion of the total silicon footprint. As illustrated in Figure 21, increasing the datapath from $\text{SIMD} = 8$ to $\text{SIMD} = 64$ expands the total area from approximately $17,500 \mu\text{m}^2$ to $25,000 \mu\text{m}^2$. This growth is driven entirely by the combinational logic required to instantiate additional parallel multiplier-accumulator (MAC) lanes.

Crucially, the non-combinational area remains strictly constant across all SIMD sweeps. An analysis of the SystemVerilog datapath explains this behavior: internal register allocations—specifically the `inp_data_reg` buffer and the `weights` memory array—are dimensioned based on the static data volume (`PAD_LANES` and `PE`). The `SIMD` parameter dictates the concurrent processing bandwidth but does not alter the aggregate data volume retained in hardware. Therefore, datapath parallelization incurs zero sequential logic overhead.

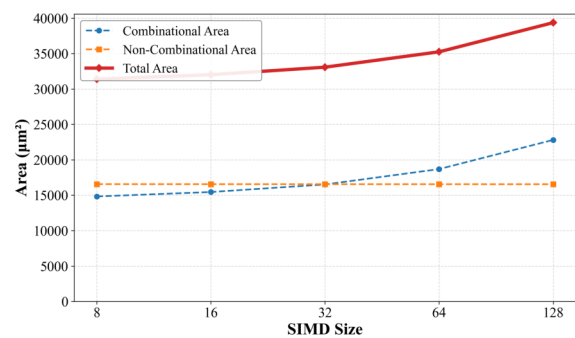
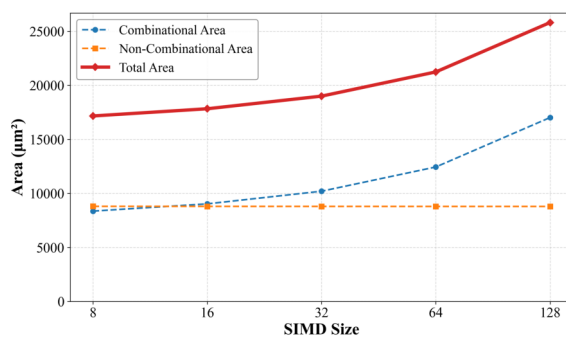


Figure 21 – Area partitioning vs. SIMD width at $\text{PE}=8$ and $\text{PAD_LANES}=128$ (14 input channels).

Figure 22 – Area partitioning vs. SIMD width at $\text{PE}=16$ and $\text{PAD_LANES}=128$ (14 input channels).

Source: Author.

In configurations processing high-dimensional tensors (Figures 23 and 24), the dynamic shifts in combinational area caused by SIMD scaling become mathematically insignificant relative to the total design size. For the largest configuration ($\text{PE} = 64$, 56 input channels), the baseline area exceeds $500,000 \mu\text{m}^2$. Expanding the datapath from $\text{SIMD} = 16$ to $\text{SIMD} = 512$ results in a nearly flat trajectory on the aggregate footprint. This confirms that for large spatial geometries, expanding the parallel compute capability is physically inexpensive compared to the baseline allocation required to establish the data infrastructure.

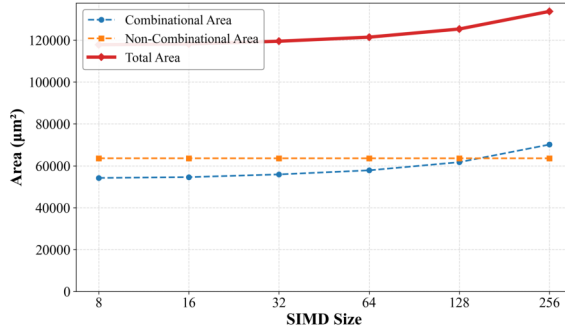


Figure 23 – Area partitioning vs. SIMD width at PE=32 and PAD_LANES=256 (28 input channels).

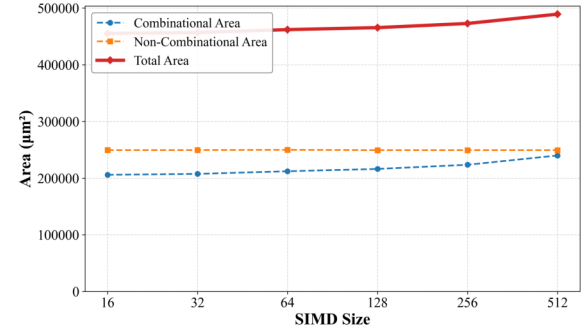


Figure 24 – Area partitioning vs. SIMD width at PE=64 and PAD_LANES=512 (56 input channels).

Source: Author.

3.4.5.2 Storage Dominance versus Output Channel Scaling

The static plateau of non-combinational logic observed during the SIMD sweeps (Figures 21 through 24) demonstrates that expanding the parallel datapath strictly alters combinational arithmetic resources. However, scaling the architectural depth—specifically, the number of active output channels (PE)—exposes a fundamentally different structural behavior. To quantify this distinction, the subsequent parameter sweep isolates the PE variable while fixing the SIMD width and input dimensions.

As demonstrated in Figures 25 and 26, incrementing the PE parameter forces a steep, linear growth in both combinational and non-combinational area. While scaling the SIMD width previously left the non-combinational footprint entirely flat, scaling from PE = 8 to PE = 64 drives the total footprint from 80,000 μm² to nearly 500,000 μm².

This structural inflation stems directly from the weight-resident execution model. The RTL array logic [MAX_INPUT_DIM-1:0] weights [PE-1:0] provisions dedicated local flip-flops for the entire tensor geometry of every active output channel. Consequently, while adding SIMD lanes merely widens the shared multiplier array, incrementing PE linearly replicates the massive localized storage requirement. Concurrently, the combinational area scales upward to support this storage; the physical multiplexers required to route data from the PE-indexed weight registers into the shared MAC array grow exponentially in complexity as the array depth increases.

Retaining full neural network weight tensors within localized pipeline registers introduces pronounced scaling limitations. While standard industry architectures utilize hierarchical SRAM buffers or block RAM to house static coefficients, deploying discrete logic registers for bulk storage dominates the silicon floorplan. This architectural decision renders the arithmetic compute arrays physically negligible by comparison and establishes memory allocation as the primary

constraint on scalability.

The synthesis reports provide a concrete measure of this dominance. For the largest evaluated configuration (PE = 64, SIMD = 512, 56 input channels) the weight register array (BLK_WEIGHTS) occupies $352,877.53 \mu\text{m}^2$ of the total synthesized area of $489,212.18 \mu\text{m}^2$ — equivalent to 72.1% of the complete silicon footprint. The arithmetic logic, control circuitry, and all remaining datapath structures collectively occupy less than 28% of the die area. This decomposition quantifies the architectural asymmetry: the weight-resident execution model allocates the majority of physical silicon not to computation, but to parameter storage.

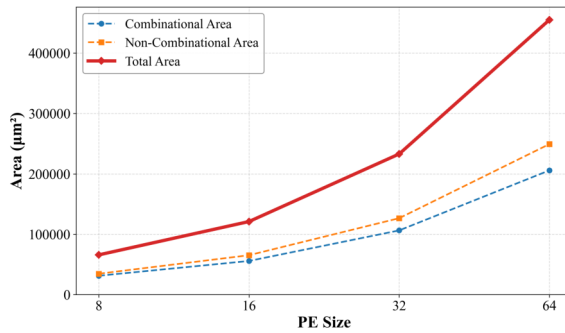


Figure 25 – Area scaling vs. output channels at SIMD=16 and PAD_LANES=512 (56 input channels).

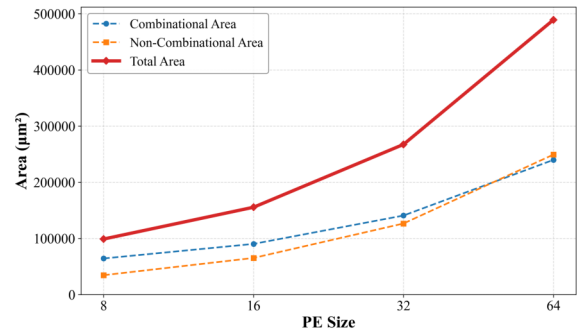


Figure 26 – Area scaling vs. output channels at SIMD=512 and PAD_LANES=512 (56 input channels).

Source: Author.

3.4.6 Power Dissipation

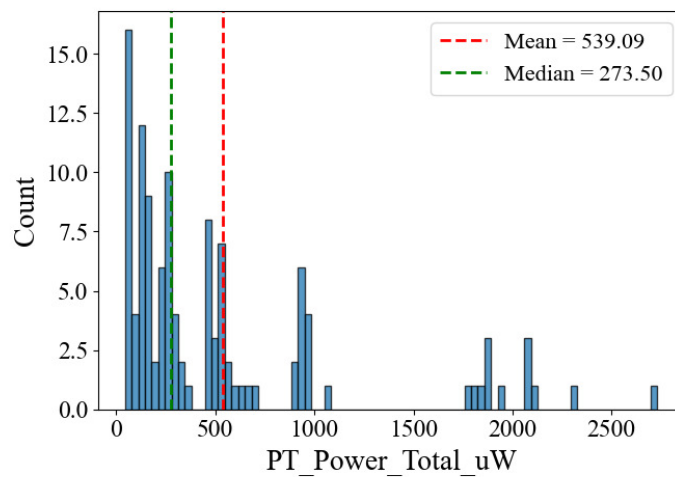
The power evaluation directly parallels the area analysis. The statistical distribution of total synthesized power (Figure 27) exhibits the same continuous, logarithmic decay observed in the physical footprint. The total power ranges from $44.10 \mu\text{W}$ for the most constrained architectures to $2,730.00 \mu\text{W}$ for the densest configurations. Table 7 details this variance.

Table 7 – Statistical distribution of total synthesized power.

Metric	Value (μW)
Mean	539.09
Median	273.50
Standard Deviation	601.71
Minimum	44.10
Maximum	2,730.00
25th Percentile	137.00
75th Percentile	602.00
Total Samples (N)	116

Source: Author.

Figure 27 – Density distribution of total synthesized silicon power.



Source: Author.

Total power dissipation in CMOS architectures comprises three fundamental components: switching, internal, and leakage power. Switching power is dynamic; it measures the energy required to charge and discharge external load capacitances as logic gates toggle states. Internal (or short-circuit) power is also dynamic, quantifying the transient energy lost when both PMOS and NMOS networks are briefly active during a state transition, creating a direct path from the supply voltage to ground. Finally, leakage (or static) power is the steady-state energy lost through transistor channels and dielectric layers even when the circuit is entirely idle.

3.4.6.1 SIMD Scaling and Static Dominance

The scaling behavior of the partitioned power metrics reinforces the conclusions drawn from the physical area footprint. Expanding the SIMD width yields minimal impact on the

dynamic (switching and internal) power components. In the constrained configuration (Figure 28), scaling from SIMD = 8 to SIMD = 64 produces negligible dynamic power variance. In the high-density configuration (Figure 29), expanding the datapath to SIMD = 512 registers a marginal decrease in switching power, alongside stagnant internal dissipation.

Conversely, the leakage power metric scales continuously with SIMD width across all configurations. Because leakage is a function of the total silicon volume (the aggregate number of instantiated transistors), the steady addition of combinational logic gates required to widen the MAC array inherently increases static power drain, regardless of dynamic switching activity.

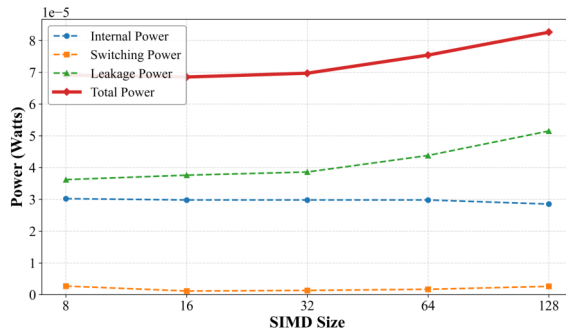


Figure 28 – Power scaling vs. SIMD width at PE=8 and PAD_LANES=128 (14 input channels).

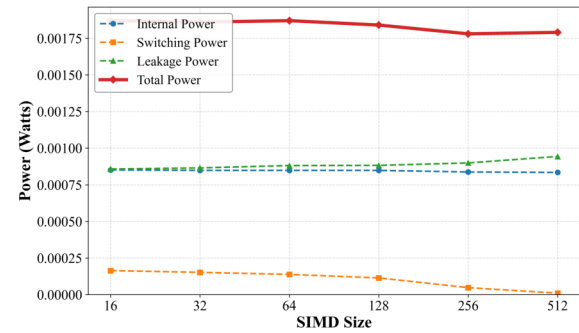


Figure 29 – Power scaling vs. SIMD width at PE=64 and PAD_LANES=512 (56 input channels).

Source: Author.

3.4.6.2 Output Channel Scaling

The dominance of storage logic over the computational array dictates the total power scaling. Isolating the active output channels (PE) exposes a rapid expansion in both internal and leakage power components. As illustrated in Figures 30 and 31, sweeping PE from 8 to 64 induces a massive scaling of total power, driven primarily by the internal power associated with clocking the expansive sequential logic arrays.

The magnitude of this power scaling remains nearly identical regardless of the underlying SIMD width. The power profile of a narrow datapath (SIMD = 16) matches the profile of a wide datapath (SIMD = 512) when scaling the internal storage requirements.

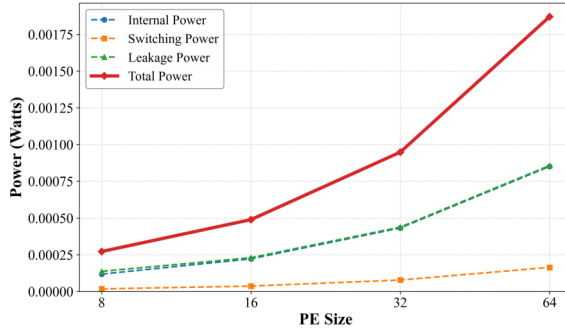


Figure 30 – Power scaling vs. output channels at SIMD=16 and PAD_LANES=512 (56 input channels).

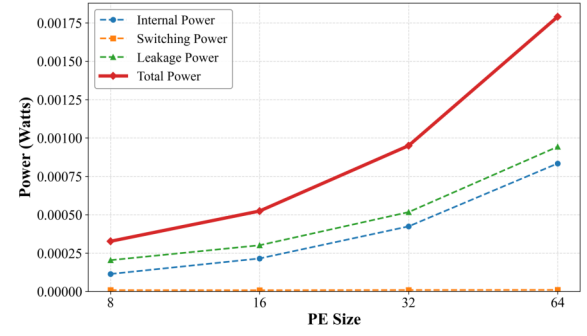


Figure 31 – Power scaling vs. output channels at SIMD=512 and PAD_LANES=512 (56 input channels).

Source: Author.

The power estimation confirms that architectural scalability is bottlenecked by static storage, not parallel computation. Generating deep, localized register arrays to maintain weight residency incurs severe penalties in both static leakage and transient internal power. While expanding the parallel MAC array (SIMD) induces a proportional increase in static leakage power, the absolute magnitude of this increase remains marginal compared to the dominant dynamic consumption of the storage baseline. Because the physical instantiation of flip-flops requires significantly more energy to clock and maintain than standard combinational arithmetic, the architectural decision to store complete tensor subsets within pipeline registers dictates the ultimate thermal and power limit of the design.

3.4.7 Power and Area Efficiency Ranking

To determine the practical viability of each hardware configuration, the evaluation must normalize throughput against resource usage. This is formalized through two derived metrics: Power Efficiency (calculated as theoretical GMAC/s/ μ W) and Area Efficiency (calculated as theoretical GMAC/s/ μ m²).

An analysis of the extracted data reveals a direct mathematical correlation between these two efficiencies. Because dynamic power scales proportionally with the physical capacitance of the instantiated standard cells, architectures optimized for minimal area footprint inherently exhibit minimal power dissipation. Consequently, the performance rankings for power efficiency and area efficiency remain virtually identical across the evaluated design space. To eliminate redundant analysis, the subsequent ranking structures prioritize power efficiency, serving as a unified metric for overall architectural resource utilization.

To systematically evaluate the impact of hardware geometry on efficiency, the 116 successfully synthesized configurations were stratified into four distinct classes based on their active

output channels ($PE \in \{8, 16, 32, 64\}$). Within each class, the highest-performing configurations for distinct input channel limits ($MAX_INP_CHANNELS$) were isolated and ranked relative to the global dataset. To ensure clarity, configurations with dimensionally adjacent input channels exhibiting equivalent efficiency profiles were filtered, yielding a focused comparison of distinct architectural boundaries.

3.4.7.1 Efficiency Analysis: 8-Output-Channel Configurations

Table 8 details the most efficient configurations operating at $PE = 8$. This class dominates the global efficiency rankings, securing the absolute peak position. The architecture exhibiting the highest efficiency across the entire design space ($SIMD = 128$, 14 input channels) achieves 26.06 GMAC/s while dissipating only 82.60 μW . This configuration benefits directly from the unconstrained routing environment discussed in Subsection 3.4.4; it sustains a high maximum frequency (254.45 MHz) without triggering the interconnect congestion that plagues denser arrays.

Notably, efficiency in this class degrades as the input channel requirement expands. When the input vector scales to accommodate 113 input channels ($SIMD = 1024$), the global efficiency rank drops to 13th. This reinforces the previous conclusion: inflating localized sequential storage directly attacks the power profile, nullifying the efficiency of the smaller MAC array.

Table 8 – Highest efficiency configurations for $PE=8$, categorized by input dimension.

Global Rank	SIMD	Inp. Channels	F_Max (MHz)	GMAC/s	Power (μW)	Area (μm^2)	GMAC/s / μW
1	128	14	254.45	26.06	82.60	25,812.52	0.3154
4	32	10	266.67	7.88	54.80	15,123.95	0.1437
5	64	21	254.45	15.03	106.00	29,074.17	0.1418
8	16	8	274.73	4.19	44.10	12,095.70	0.0949
9	32	17	251.89	7.68	85.60	22,878.34	0.0897
10	256	28	70.27	14.39	163.00	50,094.54	0.0883
11	512	56	68.54	28.07	327.00	98,913.42	0.0859
13	1024	113	68.54	56.15	702.00	203,796.25	0.0800

Source: Author.

3.4.7.2 Efficiency Analysis: 16-Output-Channel Configurations

Doubling the output channel dimension to $PE = 16$ introduces moderate scaling penalties, as shown in Table 9. The most optimal configuration within this class ($SIMD = 64$) retains a high global ranking (2nd) by matching the 259 MHz operational tier, delivering 14.78 GMAC/s. However, this efficiency drops faster than the $PE = 8$ class as the input constraints increase. For higher tensor geometries (e.g., 28 input channels), the configurations fail timing closure at high frequencies, dropping to the constrained 70 MHz limit. The resulting surge in required logic area to manage 16 fully independent filter arrays pushes the efficiency ratios down, confining the larger geometries to the lower half of the top quartile.

Table 9 – Highest efficiency configurations for PE=16, categorized by input dimension.

Global Rank	SIMD	Inp. Channels	F_Max (MHz)	GMAC/s	Power (μ W)	Area (μ m ²)	GMAC/s / μ W
2	64	7	259.74	14.78	68.40	20,420.98	0.2160
12	32	10	250.00	7.87	92.90	25,923.98	0.0827
14	32	14	257.73	7.99	121.00	33,079.05	0.0661
16	256	28	70.27	15.99	262.00	78,081.46	0.0610
17	512	56	70.27	31.98	524.00	155,529.82	0.0610
21	1024	113	65.83	59.92	1,080.00	312,686.94	0.0555

Source: Author.

3.4.7.3 Efficiency Analysis: 32 and 64-Output-Channel Configurations

The transition to high-density output channels (PE = 32 and PE = 64) confirms the severe limitations of localized weight residency. Table 10 details the PE = 32 class. Not a single configuration within this tier breaches the top quartile of the global ranking. The extreme register requirements force the baseline power floor to rise dramatically; the most efficient design in this class (SIMD = 256) dissipates 471.0 μ W, nearly six times the dissipation of the leading PE = 8 configuration, without offering proportional throughput scaling.

Table 10 – Highest efficiency configurations for PE=32, categorized by input dimension.

Global Rank	SIMD	Inp. Channels	F_Max (MHz)	GMAC/s	Power (μ W)	Area (μ m ²)	GMAC/s / μ W
29	256	28	70.32	16.94	471.00	133,727.70	0.0360
30	512	56	70.22	33.84	950.00	267,378.63	0.0356
31	128	14	70.27	8.47	238.00	67,607.46	0.0356
43	1024	113	60.13	57.95	1,930.00	536,867.27	0.0300
59	8	8	261.78	2.08	137.00	34,650.21	0.0152

Source: Author.

This efficiency decay culminates in the PE = 64 class (Table 11). At these dimensions, structural feasibility actively collapses. The majority of parameter combinations within this tier completely failed timing closure during synthesis due to insurmountable routing congestion across the massive storage logic boundaries. The handful of configurations that successfully generated netlists rank exclusively in the bottom tier globally (55th to 103rd). The baseline power requirements surge into the milliwatt domain, rendering these specific static architectures entirely unsuitable for edge-deployed computational acceleration.

Table 11 – Highest efficiency configurations for PE=64, categorized by input dimension.

Global Rank	SIMD	Inp. Channels	F_Max (MHz)	GMAC/s	Power (μ W)	Area (μm^2)	GMAC/s / μ W
55	512	56	64.23	31.89	1,790.00	489,212.18	0.0178
82	256	85	60.02	15.21	2,730.00	703,586.64	0.0056
96	128	71	59.99	7.63	2,310.00	578,544.68	0.0033
103	64	64	70.27	4.48	2,090.00	514,929.92	0.0021

Source: Author.

The systematic derivation of power and area efficiency definitively bounds the utility of this specific Matrix-Vector Unit architecture. While extending the SIMD datapath provides computational acceleration, scaling the output dimensions reveals a hard limit governed by the area and power requirements of the localized weight registers. Consequently, optimal deployment mandates constraining the active output channel dimensions ($\text{PE} \leq 16$) to ensure the operational throughput justifies the physical resource expenditure.

3.4.8 System-Level Evaluation: The Extended Roofline Model

While static structural metrics define the physical boundaries of the datapath, architectural viability is ultimately governed by the system-level interaction between parallel computation and external memory delivery. To evaluate operational throughput under realistic edge deployment constraints, the analysis transitions to the Extended Roofline Model.

A structural departure from the canonical Roofline formulation arises from the two-cycle state transition overhead defined in Equation 3.10. In the standard model, the compute roof projects as a horizontal asymptote, representing a throughput ceiling independent of workload size. The overhead term +2 introduces a size-dependent penalty: as the product $\text{PE} \cdot \text{fold_qnt}$ decreases, the two transition cycles represent a proportionally larger fraction of total execution time, depressing the effective compute ceiling below its theoretical maximum. Consequently, the compute roof in the figures that follow appears as a slightly declining function of operational intensity rather than a flat horizontal boundary. For the large-dimension configurations evaluated here, this overhead is numerically negligible; for small-dimension configurations operating at high clock frequencies, it represents a measurable throughput reduction that the standard Roofline formulation does not capture.

To systematically evaluate the system-level scalability of the weight-resident architecture, this study isolates three specific configurations from the empirical efficiency rankings. To facilitate comparative analysis, these configurations are classified into three author-defined architectural archetypes based on their distinct physical and operational characteristics:

- Resource-Constrained Node (Model 1): $\text{PE} = 8$, $\text{SIMD} = 32$, $F_{\text{max}} = 266.67 \text{ MHz}$, Compute Ceiling = 7.88 GMAC/s. This archetype represents a highly area-efficient, low-

footprint accelerator suitable for strict edge-deployment limits.

- Optimal Balanced Node (Model 2): $PE = 8$, $SIMD = 128$, $F_{\max} = 254.45$ MHz, Compute Ceiling = 26.06 GMAC/s. This archetype corresponds to the configuration yielding the highest global efficiency in Section 3.4.7, maintaining a high operational frequency without incurring severe structural routing penalties.
- Monolithic Scale-Up Node (Model 3): $PE = 64$, $SIMD = 512$, $F_{\max} = 64.23$ MHz, Compute Ceiling = 31.89 GMAC/s. This archetype represents a high-density, brute-force architecture designed to maximize theoretical throughput through extensive spatial scaling, serving as the baseline for evaluating interconnect congestion.

These architectures are evaluated against varying external memory bandwidth constraints selected to represent realistic limits for standard embedded system memory interfaces: an extreme edge constraint (1.0 GB/s), an optimal mid-tier interface (6.0 GB/s), and a high-performance memory bus (25.0 GB/s).

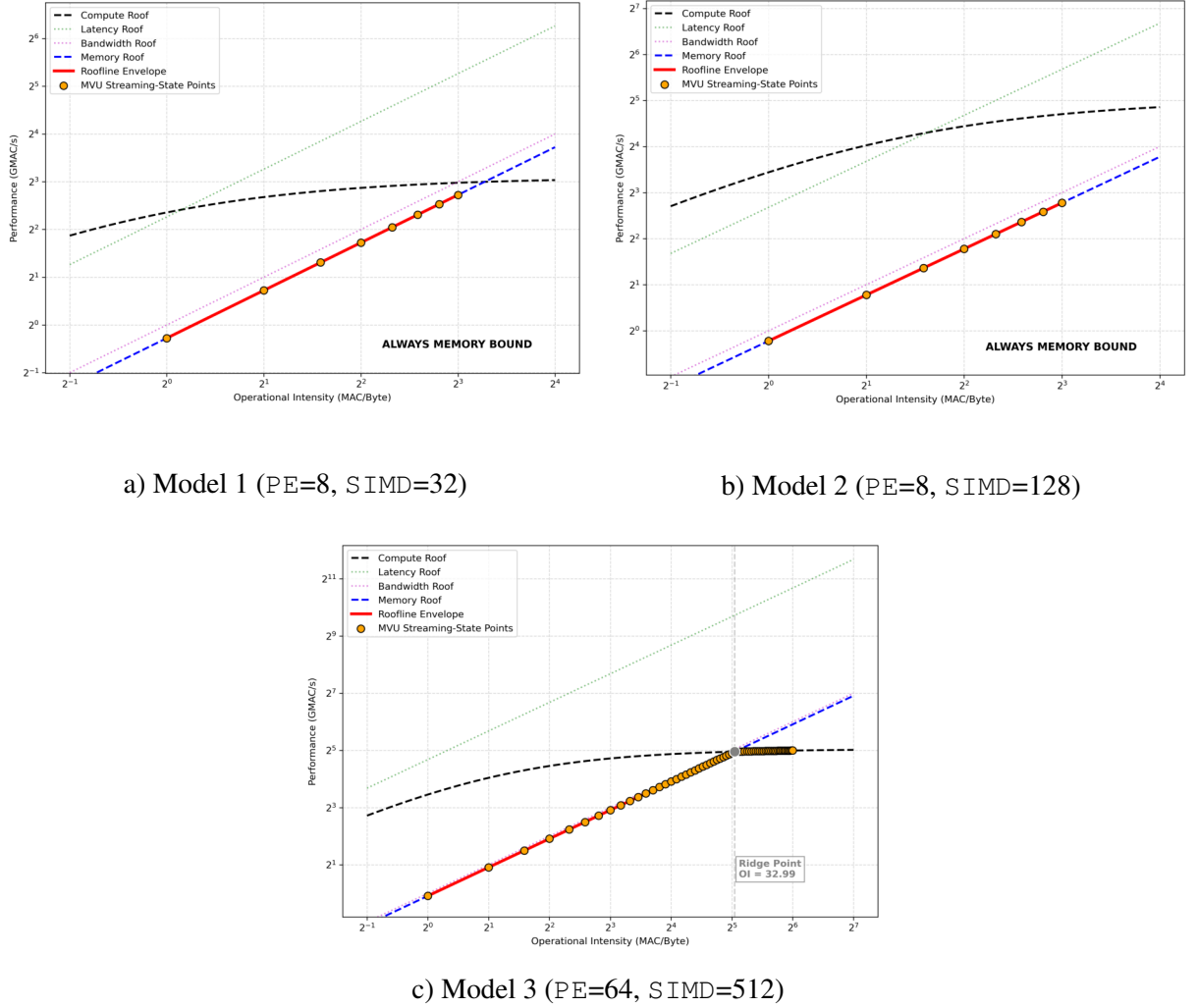
3.4.8.1 Case Study 1: 1.0 GB/s

Figure 32 illustrates the performance envelope under a severely restricted 1.0 GB/s memory interface. At this operational extreme, the inherent advantages of monolithic scaling become apparent. Because Model 3 possesses a massive localized storage capacity ($PE = 64$), it sustains an operational intensity of 64 MAC/B. This intense data reuse pushes its operational coordinates far beyond its ridge point ($OI_{\text{ridge}} = 32.99$), allowing the monolithic core to remain fully compute-bound and deliver its peak 31.89 GMAC/s.

Conversely, both Model 1 and Model 2 possess a lower structural intensity ($OI = 8$). Under the 1.0 GB/s constraint, neither configuration contains sufficient localized data reuse to mask the sluggish memory fetch times. Both nodes are strictly memory-bound, trapped beneath the memory roof slope, rendering their higher internal clock frequencies irrelevant. This physical limitation confirms that massive, monolithic arrays mask slow memory subsystems highly effectively.

The latency-masking behavior scales with the operational intensity of the configuration. In Model 3, the large input vector ($PAD_LANES = 512$, 56 input channels) yields a transfer term S/BW that is substantially larger than the access latency L at 1.0 GB/s, rendering the latency contribution negligible relative to total transfer time. The latency roof therefore appears far above the bandwidth roof in Model 3's Roofline plot. Thus, configurations with small input dimensions are more sensitive to access latency, while large-dimension configurations are governed almost entirely by sustained transfer bandwidth.

Figure 32 – Roofline models under an extreme memory constraint (1.0 GB/s bandwidth, 20 ns latency)



Source: Author.

3.4.8.2 Case Study 2: 6.0 GB/s

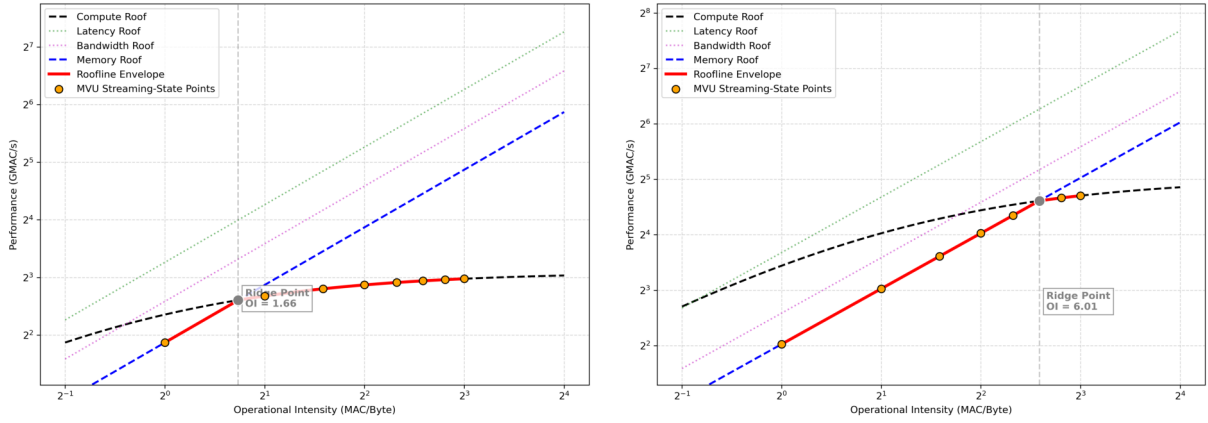
Scaling the memory interface to a moderate 6.0 GB/s radically alters the system dynamics, as demonstrated in Figure 33. The increased bandwidth lowers the required ridge point for all configurations.

Crucially, the ridge point for the highly efficient Model 2 drops to $OI_{\text{ridge}} = 6.01$. Because its physical hardware operates at $OI = 8$, the node successfully crosses the memory-bound threshold and saturates its arithmetic logic. By simply providing 6.0 GB/s of bandwidth, Model 2 achieves its absolute maximum throughput of 26.06 GMAC/s. Model 3 also remains compute-bound, its ridge point dropping to $OI_{\text{ridge}} = 4.99$.

However, at this inflection point, the monolithic approach begins to expose critical inefficiencies. Model 3 requires $489,212.18 \mu\text{m}^2$ of silicon and $1,790 \mu\text{W}$ of power to achieve 31.89 GMAC/s. Model 2 achieves a highly competitive 26.06 GMAC/s while consuming a

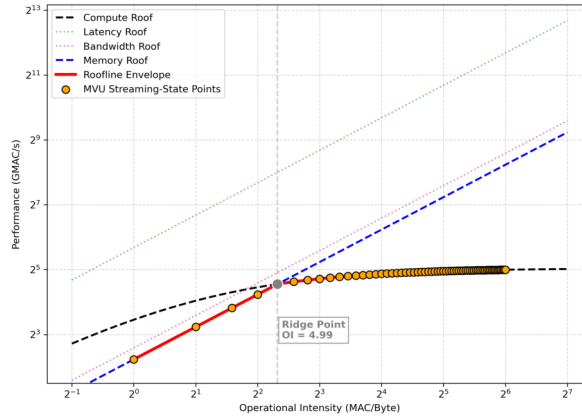
fraction of those resources ($25,813 \mu\text{m}^2$ and $83 \mu\text{W}$).

Figure 33 – Roofline models at the operational inflection point (6.0 GB/s bandwidth, 10 ns latency).



a) Model 1 (PE=8, SIMD=32)

b) Model 2 (PE=8, SIMD=128)



c) Model 3 (PE=64, SIMD=512)

Source: Author.

3.4.8.3 Case Study 3: 25.0 GB/s

In a high bandwidth memory paradigm (25.0 GB/s), the architectural limitations of the monolithic "Scale-Up" approach fail entirely. As shown in Figure 34, all three architectures operate in a strictly compute-bound state.

High-speed memory interfaces are completely wasted on Model 3 because its massive internal routing congestion stunts its clock frequency to 64.23 MHz. It cannot execute math any faster, permanently stalling its performance at 31.89 GMAC/s despite the massive data availability.

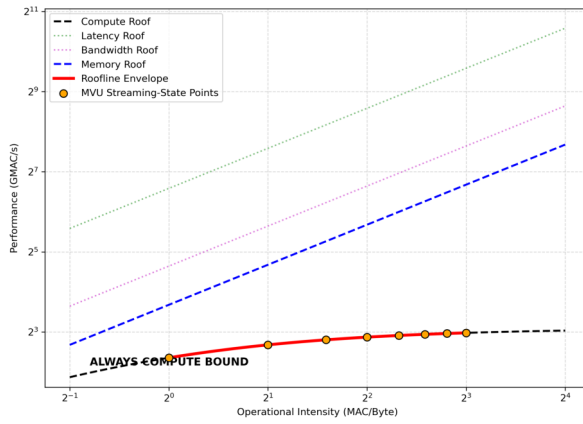
This limitation provides the central system-level conclusion of this evaluation, explicitly validating a spatial *Scale-Out* paradigm over a monolithic *Scale-Up* approach for high-

performance edge deployment. Because a single Model 2 node requires only ≈ 6.0 GB/s to hit its compute ceiling, a 25.0 GB/s memory bus can comfortably support four Model 2 nodes running entirely in parallel.

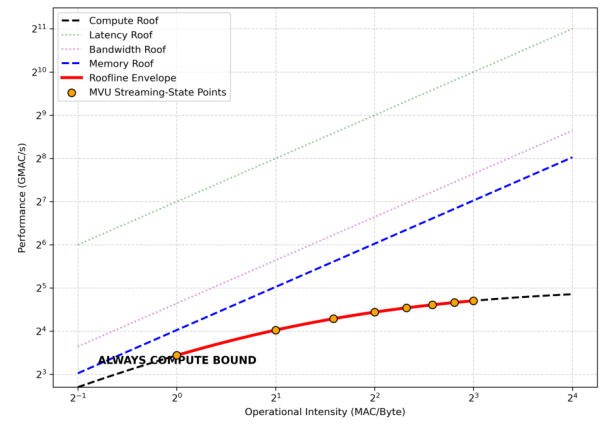
Distributing the computation across a 4-node Scale-Out array of Model 2 cores delivers a combined 104.24 GMAC/s (4×26.06 GMAC/s). This distributed topology eclipses the monolithic Model 3's maximum throughput of 31.89 GMAC/s, while simultaneously consuming 82% less power and occupying 79% less silicon area.

It is necessary to acknowledge the singular architectural caveat revealed during the 1.0 GB/s constraint evaluation: monolithic designs possess superior data-reuse characteristics that effectively mask severe memory latency. In deployment scenarios where external memory bandwidth is catastrophically restricted and physical resource boundaries (silicon area and thermal power limits) are functionally infinite, a monolithic, high-capacity resident array provides a mathematically viable methodology to sustain compute-bound throughput. However, within the pragmatic context of edge computing—where area efficiency and power dissipation are primary design directives—the sheer physical cost required to establish that memory-masking capacity renders the monolithic approach structurally untenable. Consequently, this system-level resilience definitively proves that small, parallelized, highly efficient macro-nodes represent the superior architectural topology for edge-deployed matrix acceleration.

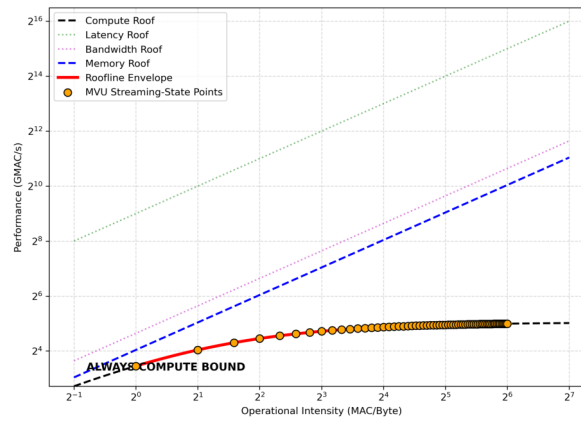
Figure 34 – Roofline models under high-bandwidth conditions (25.0 GB/s bandwidth, 1 ns latency).



a) Model 1 (PE=8, SIMD=32)



b) Model 2 (PE=8, SIMD=128)



c) Model 3 (PE=64, SIMD=512)

Source: Author.

Across all three bandwidth scenarios and all evaluated configurations, the latency roof consistently projects above the bandwidth roof in the Extended Roofline plots. This ordering reflects the relationship defined in Equation 3.9; the total memory transfer time decomposes into an initial access latency L and a bandwidth-dependent transfer term S/BW . For all of the input vector sizes evaluated, the transfer term is orders of magnitude bigger than access latency across all three bandwidth scenarios, positioning bandwidth as the binding memory constraint. The latency roof's higher placement confirms that memory latency alone — absent the bandwidth term — would permit substantially higher attainable performance. The practical implication is that bandwidth provisioning, not latency reduction, governs the memory-side optimization target for this architecture.

3.4.9 Energy-Delay Product Assessment

While standalone power metrics quantify thermal dissipation and latency defines execution speed, EDP provides a unified metric that penalizes architectures achieving low power through prolonged execution times. A lower EDP indicates a more optimal balance of speed and energy efficiency.

The EDP is calculated per inference cycle. The mathematical derivation requires the static power dissipation (μW), the total MAC operations required for a complete inference ($MAC_{s_{total}}$), and the peak throughput (GMAC/s) defined by the Roofline evaluation. The delay (D) represents the total seconds required to process the workload, and the Energy (E) represents the total Joules consumed during that delay.

$$D = \frac{MAC_{s_{total}}}{\text{GMAC/s} \times 10^9} \quad [\text{seconds}] \quad (3.16)$$

$$E = \left(\frac{\text{Power}_{\mu\text{W}}}{10^6} \right) \times D \quad [\text{Joules}] \quad (3.17)$$

$$EDP = E \times D \quad [\text{Joules} \cdot \text{seconds}] \quad (3.18)$$

To provide a standardized comparative baseline, the EDP is calculated for the three representative models assuming a generic inference workload of 1,000,000 MACs. The evaluation assumes the unconstrained 25.0 GB/s memory scenario, permitting all configurations to operate at their respective compute-bound throughput limits. Table 12 details the derived energy and temporal metrics.

Table 12 – Energy-Delay Product (EDP) comparison for a 10^6 MAC workload.

Architecture	Power (μW)	Delay (μs)	Energy (nJ)	EDP ($\text{J} \cdot \text{s} \times 10^{-13}$)
Model 1 (PE=8, SIMD=32)	54.80	126.90	6.95	8.82
Model 2 (PE=8, SIMD=128)	82.60	38.37	3.17	1.22
Model 3 (PE=64, SIMD=512)	1,790.00	31.36	56.11	17.59

Source: Author.

The resulting metric distribution quantifies the inefficiency of the scale-up approach. Model 3 completes the standardized computation $7.02 \mu\text{s}$ faster than Model 2, yet requires approximately $18\times$ more total energy to execute the same task. Consequently, Model 3 yields an EDP an order of magnitude higher than Model 2. This mathematical derivation confirms that expanding the internal register depth to force monolithic scaling degrades the fundamental energy-delay balance, establishing the distributed, high-efficiency topology of Model 2 as the structurally superior methodology for edge-deployed acceleration.

3.5 Methodological Limitations

The evaluation of the MVU presents several methodological and architectural limitations that constrain the absolute precision of the extracted metrics. These constraints stem from foundational design decisions, synthesis parameterization, and infrastructural reliability.

3.5.1 Architectural Constraints

The primary structural limitation of the evaluated MVU is the fully unrolled, register-based weight-resident architecture. Instantiating standard flip-flops to store complete network tensor geometries exponentially inflates the non-combinational silicon footprint and associated static leakage power. Standard industrial accelerators mitigate this by utilizing dense, hierarchical SRAM or block RAM (BRAM) arrays. The omission of BRAM integration restricts the spatial scalability of the MVU, artificially capping the maximum viable dimensions of the logical processing elements (PE) due to layout congestion.

Furthermore, the internal parallel datapath lacks pipelining within the MAC array. Executing the entire SIMD multiplication and wide-tree accumulation within a single clock cycle creates a deep combinational logic path. This extended logic depth restricts the maximum operational frequency and prevents the architecture from exploiting sparsity logic to bypass zero-value computations, degrading overall energy proportionality.

3.5.2 Synthesis Methodology Constraints

The logic synthesis methodology utilized a static 70 MHz target clock frequency across all 116 hardware configurations. This unvarying constraint distorts the maximum throughput scaling analysis. For smaller configurations (e.g., PE = 8), the synthesis compiler actively optimized for area and power once the 70 MHz timing closure was achieved, rather than iteratively pushing the logic to its true physical frequency limit. Conversely, large configurations failing the 70 MHz constraint extrapolated their $F_{\max}$ metrics from negative slack calculations. Extracting the precise frequency limits requires iterative, multi-pass synthesis sweeps per configuration—a procedure omitted due to computational time constraints.

Power estimation accuracy is similarly bounded. The evaluation derived dynamic power metrics using a generic, statistically uniform 10% toggle rate across all input stimuli via Synopsys PrimeTime. The framework did not integrate functional simulation switching data (e.g., SAIF or VCD files). Relying on probabilistic activity models precludes the analysis of data-dependent dissipation inherent to specific neural network inference workloads.

Finally, the hardware description lacked rigid structural modularization. While virtual hierarchical grouping provided preliminary layout visibility, the synthesis compiler amalgamated sub-blocks during optimization. This aggregation prevented the precise isolation of area and power metrics for individual datapath components (e.g., quantifying the isolated dissipation of

the requantization logic versus the primary accumulator array). Evaluating the specific standard cells composing the critical paths was omitted, restricting the analysis to macro-level area and power consumption rather than technology-dependent gate optimization.

3.5.3 Infrastructural Instability

The execution of the design space exploration was hindered by infrastructure instability within the remote CADmicro environment. Periodic server storage depletion caused unrecoverable data loss and terminated automated simulation batches. These systematic interruptions significantly reduced the available execution window, directly forcing the omission of extreme dimensional testing (e.g., $\text{SIMD} = 2048$) and exhaustive iterative frequency sweeps.

3.6 Final Considerations

The design space exploration successfully bounds the physical and operational limitations of the parameterized MVU. The derived synthesis metrics and Roofline modeling mathematically confirm that monolithic architectural expansion incurs severe area and power penalties, ultimately stalling performance under realistic memory bandwidth constraints. While methodological limitations—specifically regarding static timing targets and non-hierarchical memory implementation—constrain the absolute precision of the extracted metrics, the aggregate trends validate the superior energy-delay profile of distributed, scale-out topologies. The subsequent chapter summarizes these findings, addressing the primary research objectives and defining the requisite trajectory for future microarchitectural development.

4 CONCLUSION

This research systematically quantified the operational boundaries of a weight-resident Matrix-Vector Unit under varying spatial dimensions and memory constraints, successfully fulfilling the primary academic objectives. The central product of this work comprises the functionally verified SystemVerilog datapath paired with an automated synthesis and Extended Roofline evaluation framework, enabling reproducible design space exploration.

The analytical findings directly corroborate the prevailing architectural paradigms utilized in state-of-the-art commercial and academic accelerators, such as the Eyeriss framework (Chen; Emer; Sze, 2016). The data demonstrates that while large contiguous arrays effectively mask high memory latency through extensive data reuse, their reliance on localized logic registers for tensor storage incurs prohibitive area and thermal penalties. In constrained edge environments, optimizing the Energy-Delay Product requires restricting individual node dimensions and distributing computation across smaller, highly efficient processing elements. By proving that a scale-out topology operating under a 25.0 GB/s memory interface eclipses a contiguous scale-up architecture while consuming 82% less power, this thesis mathematically validates the industry transition toward localized, hierarchical scratchpad memories and decentralized spatial dataflows.

4.1 Challenges

Executing this design space exploration demanded overcoming substantial technical and infrastructural challenges. Transitioning from abstract algorithmic principles to precise RTL implementation required rigorous independent study of digital logic design and hardware optimization. Furthermore, mastering the Synopsys EDA toolchain to extract valid physical metrics constituted a considerable technical barrier, compounded by persistent instability and storage failures within the remote simulation servers. Resolving these overlapping hardware and software limitations forced the adoption of robust scripting automation and disciplined debugging methodologies. Transcending these initial unfamiliarities ultimately transformed theoretical computer architecture concepts into a concrete understanding of physical silicon constraints.

4.2 Future Work

To advance the precision and applicability of this evaluation, subsequent research must address the identified microarchitectural and methodological limitations. Primarily, future iterations must replace the fully unrolled discrete logic registers with hierarchical memory subsystems utilizing localized BRAM. Implementing explicit BRAM structures for weight and activation

staging transitions the architecture from a static state machine to a continuous streaming model, permitting asynchronous read-request scheduling and output tiling.

Analytically, the evaluation framework requires expansion to incorporate functional simulation waveforms (VCD or SAIF data) to extract exact, data-dependent dynamic power profiles across distinct neural network workloads. Mapping the Operational Intensity of specific convolutional and fully-connected layer topologies to the extracted Roofline boundaries will provide designers with targeted design recommendations based on precise input/output channel and model requirements. Finally, modularizing the RTL code to isolate component-specific dissipation (e.g., the accumulator versus the demultiplexer) and analyzing the exact standard cells comprising the combinational critical paths will establish a definitive framework for executing structural optimization on full-scale edge accelerators.

REFERENCES

- ALAM, S. A. *et al.* On the rtl implementation of finn matrix vector unit. **ACM Trans. Embed. Comput. Syst.**, Association for Computing Machinery, New York, NY, USA, v. 22, n. 6, nov. 2023. ISSN 1539-9087. Available at: <https://doi.org/10.1145/3547141>.
- ALWANI, M. *et al.* Fused-layer cnn accelerators. **2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)**, p. 1–12, 2016. Available at: <https://api.semanticscholar.org/CorpusID:5804465>.
- Apple Inc. **Introducing M1 Pro and M1 Max: the most powerful chips Apple has ever built.** 2021. Available at: <https://www.apple.com/newsroom/2021/10/introducing-m1-pro-and-m1-max-the-most-powerful-chips-apple-has-ever-built/>. Access at: May 17, 2026.
- Apple Inc. **Apple unveils M1 Ultra, the world’s most powerful chip for a personal computer.** 2022. Available at: <https://www.apple.com/newsroom/2022/03/apple-unveils-m1-ultra-the-worlds-most-powerful-chip-for-a-personal-computer/>. Access at: May 17, 2026.
- Apple Inc. **Apple’s new MacBook Pro features the incredibly powerful M4 family of chips and ushers in a new era with Apple Intelligence.** 2024. Available at: <https://www.apple.com/newsroom/2024/10/apples-new-macbook-pro-features-the-incredibly-powerful-m4-family-of-chips-and-ushers-in-a-new-era-with-apple-intelligence/>. Access at: May 18, 2026.
- Apple Inc. **MacBook Air (13-inch, M3, 2024) - Tech Specs.** 2024. Available at: <https://support.apple.com/en-lb/118551>. Access at: May 18, 2026.
- Apple Inc. **Apple introduces the new MacBook Air with M5.** 2026. Available at: <https://nr.apple.com/DC6x5U7pJ9>. Access at: May 18, 2026.
- Apple Inc. **Mac Studio - Technical Specifications.** 2026. Available at: <https://www.apple.com/mac-studio/specs/>. Access at: May 18, 2026.
- BANBURY, C. *et al.* **MLPerf Tiny Benchmark.** 2021. Available at: <https://arxiv.org/abs/2106.07597>.
- Blender Foundation. **Blender Open Data Benchmark.** 2026. Available at: <https://opendata.blender.org/>. Access at: May 17, 2026.
- BLOTT, M. *et al.* **FINN-R: An End-to-End Deep-Learning Framework for Fast Exploration of Quantized Neural Networks.** 2018. Available at: <https://arxiv.org/abs/1809.04570>.
- CABEZAS, V. C.; PÜSCHEL, M. Extending the roofline model: Bottleneck analysis with microarchitectural constraints. *In: 2014 IEEE International Symposium on Workload Characterization (IISWC)*. [S.l.: s.n.], 2014. p. 222–231.
- CHEN, Y.-H.; EMER, J.; SZE, V. Eyeriss: A spatial architecture for energy-efficient dataflow for convolutional neural networks. *In: 2016 ACM/IEEE 43rd Annual International Symposium on Computer Architecture (ISCA)*. [S.l.: s.n.], 2016. p. 367–379.
- CHEN, Y.-H. *et al.* Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks. **IEEE Journal of Solid-State Circuits**, v. 52, n. 1, p. 127–138, 2017.

CHEN, Y.-H. *et al.* Eyeriss v2: A flexible accelerator for emerging deep neural networks on mobile devices. **IEEE Journal on Emerging and Selected Topics in Circuits and Systems**, v. 9, n. 2, p. 292–308, 2019.

FOWERS, J. *et al.* A configurable cloud-scale dnn processor for real-time ai. *In: Proceedings of the 45th Annual International Symposium on Computer Architecture*. IEEE Press, 2018. (ISCA '18), p. 1–14. ISBN 9781538659847. Available at: <https://doi.org/10.1109/ISCA.2018.00012>.

FRANSOO, J. C.; PEELS, R.; UDENIO, M. Navigating supply chain dynamics for sustained AI growth. *In: COHEN, M. C.; DAI, T. (ed.). AI in Supply Chains: Perspectives from Global Thought Leaders*. Cham, Switzerland: Springer Cham, 2026, (Springer Series in Supply Chain Management). p. 37–54. Available at: <https://research.tilburguniversity.edu/en/publications/771fa325-5fbb-4b03-b6bd-bd0415552a84>. Access at: May 16, 2026.

GONZALEZ, R.; HOROWITZ, M. Energy dissipation in general purpose microprocessors. **IEEE Journal of Solid-State Circuits**, v. 31, n. 9, p. 1277–1284, 1996.

GOODFELLOW, I.; BENGIO, Y.; COURVILLE, A. **Deep Learning**. [S.l.: s.n.]: MIT Press, 2016. <http://www.deeplearningbook.org>.

HOROWITZ, M. 1.1 computing's energy problem (and what we can do about it). *In: 2014 IEEE International Solid-State Circuits Conference Digest of Technical Papers (ISSCC)*. [S.l.: s.n.], 2014, p. 10–14.

HOROWITZ, M.; INDERMAUR, T.; GONZALEZ, R. Low-power digital design. *In: Proceedings of 1994 IEEE Symposium on Low Power Electronics*. [S.l.: s.n.], 1994, p. 8–11.

IEEE Computer Society and the IEEE Standards Association Corporate Advisory Group. **IEEE Standard for SystemVerilog—Unified Hardware Design, Specification, and Verification Language**. 3 Park Avenue, New York, NY 10016-5997, USA, 2018. IEEE Std 1800-2017 (Revision of IEEE Std 1800-2012). Approved 6 December 2017, Published 21 February 2018.

JACOB, B. *et al.* **Quantization and Training of Neural Networks for Efficient Integer-Arithmetic-Only Inference**. 2017. Available at: <https://arxiv.org/abs/1712.05877>.

JOUPPI, N. P. *et al.* **TPU v4: An Optically Reconfigurable Supercomputer for Machine Learning with Hardware Support for Embeddings**. 2023. Available at: <https://arxiv.org/abs/2304.01433>.

JOUPPI, N. P. *et al.* **In-Datacenter Performance Analysis of a Tensor Processing Unit**. 2017. Available at: <https://arxiv.org/abs/1704.04760>.

KWON, H.; SAMAJDAR, A.; KRISHNA, T. Maeri: Enabling flexible dataflow mapping over dnn accelerators via reconfigurable interconnects. **SIGPLAN Not.**, Association for Computing Machinery, New York, NY, USA, v. 53, n. 2, p. 461–475, mar. 2018. ISSN 0362-1340. Available at: <https://doi.org/10.1145/3296957.3173176>.

LORENZO, O. G. *et al.* 3dymr: a dynamic roofline model including memory latency information. **J. Supercomput.**, Kluwer Academic Publishers, USA, v. 70, n. 2, p. 696–708, nov. 2014. ISSN 0920-8542. Available at: <https://doi.org/10.1007/s11227-014-1163-4>.

MICHELI, G. D. **Synthesis and optimization of digital circuits**. London, England: McGraw-Hill Education (ISE Editions), 1994.

NVIDIA. **NVIDIA Blackwell platform arrives to power a new era of computing**. 2024. Available at: <https://nvidianews.nvidia.com/news/nvidia-blackwell-platform-arrives-to-power-a-new-era-of-computing>. Access at: May 17, 2025.

PATTERSON, D. A.; HENNESSY, J. L. **Computer organization and design MIPS edition**. 6. ed. Oxford, England: Morgan Kaufmann, 2020. (The Morgan Kaufmann Series in Computer Architecture and Design).

Reuters. **AI frenzy is driving new global supply chain crisis**. 2025. Available at: <https://www.reuters.com/world/china/ai-frenzy-is-driving-new-global-supply-chain-crisis-2025-12-03/>. Access at: May 17, 2026.

RUSSELL, S.; NORVIG, P. **Artificial intelligence: A modern approach, global edition**. 4. ed. London, England: Pearson Education, 2021.

SAVITCH, W. J.; STIMSON, M. J. Time bounded random access machines with parallel processing. **J. ACM**, Association for Computing Machinery, New York, NY, USA, v. 26, n. 1, p. 103–118, jan. 1979. ISSN 0004-5411. Available at: <https://doi.org/10.1145/322108.322119>.

SESHAN, K.; SCHEPIS, D. (ed.). **Handbook of Thin Film Deposition**. 4th. ed. [S.l.: s.n.]: Elsevier, 2018. ISBN 978-0-12-812311-9.

Synopsys. **SAED 14nm FinFET Educational Design Kit Databook**. Revision 1.1. [S.l.]. Technology: SAED14nm FinFET. Process: SAED14nm FinFET 1P9M 0.8v/1.5v/1.8v. Primary simulation models based on the Predictive Technology Model (PTM) by the Nanoscale Integration and Modeling Group (NIMO) of Arizona State University (ASU).

SZE, V. *et al.* **Efficient processing of deep neural networks**. San Rafael, CA: Morgan & Claypool, 2020. (Synthesis Lectures on Computer Architecture).

TANENBAUM, A. S. **Structured computer organization**. Pearson, 2013. ISBN 0132916525. Available at: http://openlibrary.org/books/OL25620852M/Structured_computer_organization.

Tom's Hardware. **The RAM pricing crisis has only just started, Team Group GM warns**. 2025. Available at: <https://www.tomshardware.com/pc-components/dram/the-ram-pricing-crisis-has-only-just-started-team-group-gm-warns-says-problem-will-get-worse-in-2026-as-dram-and-nand-prices-double-in-one-month>. Access at: May 17, 2026.

UMUROGLU, Y. *et al.* Finn: A framework for fast, scalable binarized neural network inference. In: **Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays**. ACM, 2017. (FPGA '17), p. 65–74. Available at: <http://dx.doi.org/10.1145/3020078.3021744>.

WANG, L.-T.; WU, C.-W.; WEN, X. **VLSI Test Principles and Architectures**. Oxford, England: Morgan Kaufmann, 2006. (The Morgan Kaufmann series in systems on silicon).

WESTE, N.; HARRIS, D. **CMOS VLSI Design**. 4. ed. Upper Saddle River, NJ: Pearson, 2010.

WILLIAMS, S.; WATERMAN, A.; PATTERSON, D. Roofline: an insightful visual performance model for multicore architectures. **Commun. ACM**, Association for Computing Machinery, New York, NY, USA, v. 52, n. 4, p. 65–76, abr. 2009. ISSN 0001-0782. Available at: <https://doi.org/10.1145/1498765.1498785>.

ZHANG, C. Layered chips instruments and firm response: Evidence from micron's high bandwidth memory investment program. *In: **Proceedings of the 2026 11th International Conference on Financial Innovation and Economic Development (ICFIED 2026)***. Atlantis Press, 2026. p. 579–587. ISBN 978-94-6239-642-5. ISSN 2352-5428. Available at: https://doi.org/10.2991/978-94-6239-642-5_56.

APPENDIX

APPENDIX A – APPLE SILICON GPU PERFORMANCE IN BLENDER OPEN DATA BENCHMARK

This annex presents the raw benchmark scores and sample sizes extracted from the Blender Open Data platform (Blender Foundation, 2026).

Table 13 – Apple Silicon GPU Performance Metrics

Device Name	Median Score	Number of Benchmarks
Apple M5 Max (GPU - 40 cores)	8065.09	122
Apple M3 Ultra (GPU - 80 cores)	7502.95	14
Apple M5 Max (GPU - 32 cores)	6578.65	14
Apple M3 Ultra (GPU - 60 cores)	6505.79	23
Apple M4 Max (GPU - 40 cores)	5278.95	156
Apple M4 Max (GPU - 32 cores)	4463.37	79
Apple M5 Pro (GPU - 20 cores)	4298.01	48
Apple M3 Max (GPU - 40 cores)	4262.90	51
Apple M5 Pro (GPU - 16 cores)	3741.82	20
Apple M3 Max (GPU - 30 cores)	3438.11	39
Apple M2 Ultra (GPU - 76 cores)	3407.67	4
Apple M2 Ultra (GPU - 60 cores)	2825.63	10
Apple M4 Pro (GPU - 20 cores)	2576.46	143
Apple M4 Pro (GPU - 16 cores)	2377.23	89
Apple M5 (GPU - 8 cores)	1864.45	21
Apple M2 Max (GPU - 38 cores)	1785.87	30
Apple M5 (GPU - 10 cores)	1770.62	89
Apple M3 Pro (GPU - 18 cores)	1767.02	40
Apple M3 Pro (GPU - 14 cores)	1659.02	37
Apple M1 Ultra (GPU - 64 cores)	1656.68	10
Apple M2 Max (GPU - 30 cores)	1450.49	21
Apple M1 Ultra (GPU - 48 cores)	1440.81	5
Apple M4 (GPU - 10 cores)	1087.90	223
Apple M4 (GPU - 8 cores)	1072.56	68
Apple M1 Max (GPU - 32 cores)	959.85	78
Apple M3 (GPU - 10 cores)	921.74	29
Apple M2 Pro (GPU - 19 cores)	899.20	13
Apple M3 (GPU - 8 cores)	875.61	10

Continued on next page

Table 13 – Continued from previous page

Device Name	Median Score	Number of Benchmarks
Apple M1 Max (GPU - 24 cores)	799.93	37
Apple M2 Pro (GPU - 16 cores)	780.18	26
Apple A18 Pro (GPU - 5 cores)	607.33	14
Apple M1 Pro (GPU - 16 cores)	483.97	55
Apple M1 Pro (GPU - 14 cores)	444.54	32
Apple M2 (GPU - 10 cores)	354.50	19
Apple M2 (GPU - 8 cores)	302.32	20
Apple M1 (GPU - 8 cores)	262.98	30
Apple M1 (GPU - 7 cores)	244.18	12

Source: Blender Foundation (2026).

APPENDIX B – CODE

Source Code B.1 – MVU Logic Implemented in SystemVerilog

```

1  /*****
2  *      Projeto de Formatura I - SCC0670
3  *
4  *      By: Artur Brenner Weber
5  *      email: arturweber@usp.br
6  *      Last Update: 26/5/2026
7  *
8  *      Original version written by Eduardo Sperle Honorato
9  *      based on work from "On the RTL Implementation of FINN
10 *      Matrix Vector Unit".
11 *****/
12
13 `timescale 1ns/1ps
14 `include "rtl_cfg.svh"
15
16 module pe #
17 (
18     parameter byte unsigned DATA_WIDTH = `DATA_WIDTH,
19     parameter shortint MAX_VAL = (1<<< (DATA_WIDTH-1)) - 1,
20     //Maximum output value based on data width. Used to
21     clip the accumulator output when scaling values
22     parameter shortint MIN_VAL = -(1 <<< (DATA_WIDTH-1)),
23     //Minimum output value based on data width. Used to
24     clip the accumulator output when scaling values
25     parameter byte unsigned ACC_BIT_WIDTH=32,
26     //Accumulator data width
27     parameter byte unsigned M_INT_PRECISION=32,
28     //Precision of the scaler. Used in
29     scaling
30     parameter longint signed PRECISION_CORRECTION=1 <<
31     (M_INT_PRECISION-1), //Rounding correction, used to properly
32     round numbers
33     parameter byte unsigned TEMP_DATA_WIDTH=ACC_BIT_WIDTH+M_INT_PRECISION+1, //Data width for
34     scaling operation. out_temp= scale * acc_out + correction
35     parameter byte unsigned NUM_ACTS_FUN=2,
36     //Number of implemented
37     activation functions: Identity, ReLU
38     parameter shortint unsigned SIMD = `SIMD,
39     //SIMD width, received from
40     testbench
41     parameter shortint unsigned PE = `PE,
42     //Number of logical PEs,
43     i.e. output channels
44     parameter shortint unsigned MAX_CHANNELS = `MAX_CHANNELS,
45     //Maximum number of input channels
46     parameter shortint unsigned KERNEL_X = `KERNEL_X,
47     //Kernel width
48     parameter shortint unsigned KERNEL_Y = `KERNEL_Y,
49     //Kernel height
50     parameter int unsigned MAX_DOT_LANES =
51     MAX_CHANNELS*KERNEL_X*KERNEL_Y, //Number of values to be
52     multiplied per filter/PE
53     parameter int unsigned MAX_FOLDS = (MAX_DOT_LANES + SIMD - 1) /
54     SIMD, //Number of folds needed to process MAX_DOT_LANES with
55     SIMD lanes
56     parameter int unsigned PAD_LANES = MAX_FOLDS * SIMD,
57     //Amount of numbers in filter/input after
58     padding
59     parameter int unsigned MAX_INPUT_DIM = PAD_LANES * DATA_WIDTH,
60     //Length of each filter/input after padding in bits
61     parameter shortint unsigned MAX_OUTPUT_DIM = PE*DATA_WIDTH
62     //Length of output from all filters in bits
63 )
64 (
65     `ifndef SYNTHESIS
66     //=====DEBUG=====

```

```

40     output int unsigned
41         check_state, check_next_state,
42     output int unsigned
43         check_read_posx,
44     output int unsigned
45         check_read_posy,
46     output int unsigned
47         check_weight_ind,
48     output int unsigned
49         check_PE_ind,
50     output logic
51         check_write_complete,
52     output logic
53         check_computing_complete,
54     output logic [PE*MAX_INPUT_DIM-1:0]
55         check_weights,
56     output logic [PE*SIMD*DATA_WIDTH-1:0]
57         check_current_weights,
58     output logic [PE*DATA_WIDTH-1:0]
59         check_output_PE,
60     output logic [MAX_INPUT_DIM-1:0]
61         check_input,
62     output logic [SIMD*DATA_WIDTH-1:0]
63         check_input_part,
64     // output logic [2*DATA_WIDTH+$clog2(SIMD)-1:0]
65         check_out_add,
66     output logic [ACC_BIT_WIDTH-1:0]
67         check_out_acc,
68     output logic signed [TEMP_DATA_WIDTH-1:0]
69         check_out_temp,
70     //=====DEBUG=====
71 `endif
72 //=====INPUTS=====
73     input logic
74         rst_n,
75     input logic
76         clk,
77     input logic
78         set_cfg_n,
79     input logic [(NUM_ACTS_FUN <= 1) ? 1 : $clog2(NUM_ACTS_FUN)-1:0]
80         act_fun,
81     input logic
82         wr_en,
83     input logic
84         str_wr,
85     input logic
86         inp_rd,
87     input logic [MAX_INPUT_DIM-1:0]
88         inp_data,
89     input logic [$clog2(MAX_FOLDS+1)-1:0]
90         fold_qnt,
91     input logic [$clog2(PE+1)-1:0]
92         pe_qnt,
93     //=====OUTPUTS=====
94     output logic [MAX_OUTPUT_DIM-1:0]
95         out,
96     output logic
97         output_ready,
98     output logic
99         ready_to_receive
100 );
101
102 enum logic [1:0] { empty=0, writing=1, ready=2, computing=3 } state,
103     next_state;
104
105 logic signed [DATA_WIDTH-1:0]
106     out_PE [PE-1:0];
107 `ifndef SYNTHESIS
108 logic signed [TEMP_DATA_WIDTH-1:0]
109     out_temp;
110 `endif
111 logic [MAX_INPUT_DIM-1:0]
112     weights [PE-1:0];
113 logic signed [M_INT_PRECISION-1:0]
114     scales;
115 logic [MAX_INPUT_DIM-1:0]
116     inp_data_reg;

```

```

83     logic signed [ACC_BIT_WIDTH-1:0]
84         out_acc;
85     logic
86         write_complete;
87     logic
88         computing_complete;
89
90     logic [$clog2(MAX_FOLDS+2)-1:0]
91         read_posx;
92     logic [$clog2(PE+1)-1:0]
93         read_posy;
94     localparam int unsigned
95         W_FOLD_IDX = (MAX_FOLDS <= 1) ? 1 : $clog2(MAX_FOLDS);
96     logic [W_FOLD_IDX-1:0]
97         weights_ind;
98     localparam int unsigned
99         W_PE_IDX = (PE <= 1) ? 1 : $clog2(PE);
100     logic [W_PE_IDX-1:0]
101         PE_ind;
102
103     logic [$clog2(MAX_FOLDS+2)-1:0]
104         fold_qnt_reg;
105     logic [$clog2(PE+1)-1:0]
106         pe_qnt_reg;
107     localparam int unsigned
108         W_ACT = (NUM_ACTS_FUN <= 1) ? 1 : $clog2(NUM_ACTS_FUN);
109     logic [W_ACT-1:0]
110         act_fun_reg;
111
112     always_comb begin : state_change_logic
113         next_state = state; // state change
114         case (state)
115             empty: begin
116                 if (str_wr) //if start writing is enabled then go to writing
117                     state
118                     next_state = writing;
119             end
120             writing: begin
121                 if (write_complete) //if the weights have already been filled,
122                     change to ready state
123                     next_state = ready;
124             end
125             ready: begin
126                 if (inp_rd) //if input is ready goes to computing state
127                     next_state = computing;
128             end
129             computing: begin
130                 if (computing_complete) begin
131                     if (inp_rd) //if computing complete and input ready keeps
132                         computing state
133                         next_state = computing;
134                     else //otherwise go to ready state to wait for the inp_rd
135                         signal
136                         next_state = ready;
137                 end
138             end
139         endcase
140     end
141
142     always_ff @(posedge clk) begin : state_change_ff
143         //resets the state to empty when rst_n is low, otherwise updates to the
144         next state
145         if (!rst_n)
146             state <= empty;
147         else
148             state <= next_state;
149         end
150
151     always_ff @(posedge clk) begin : main_logic
152         if (!rst_n) begin
153             // Resetting flags used by the control unit
154             computing_complete <= '0;
155             write_complete <= '0;
156         end

```

```

143         output_ready      <= '0;
144         ready_to_receive  <= '0;
145
146         // Resetting small index counters to be safe
147         weights_ind       <= '0;
148         PE_ind            <= '0;
149         read_posx         <= '0;
150         read_posy         <= '0;
151
152         // Initialize critical tiny registers to prevent X propagation
153         scales             <= '0;
154         inp_data_reg       <= '0;
155     end
156     else if (!set_cfg_n) begin
157         fold_qnt_reg <= fold_qnt;
158         pe_qnt_reg <= pe_qnt;
159         act_fun_reg <= act_fun;
160     end
161     // Latch new input data only when it is accepted
162     else if (((state == ready) && inp_rd) || ((state == computing) &&
        computing_complete && inp_rd)) begin
163         inp_data_reg <= inp_data;
164         // Clear per-inference state here so out_PE never carries old/X
        // lanes across PEs or inputs
165         out_PE <= '{default: '0};
166         out <= '0;
167         out_acc <= '0;
168         weights_ind <= '0;
169         PE_ind <= '0;
170         computing_complete <= 1'b0;
171         output_ready <= 1'b0;
172         ready_to_receive <= 1'b0;
173     end
174     else begin
175         case(state)
176             //=====WRITING STATE=====
177             writing: begin
178                 logic [$bits(read_posx)-1:0] read_posx_next;
179                 logic [$bits(read_posy)-1:0] read_posy_next;
180                 output_ready <= '0;
181                 ready_to_receive <= '0;
182                 read_posx_next = read_posx;
183                 read_posy_next = read_posy;
184
185                 if (wr_en) begin
186                     if (!write_complete) begin
187                         // weights phase: read_posy = 0..pe_qnt_reg-1
188                         if (read_posy < pe_qnt_reg) begin
189                             // write fold
190                             weights[read_posy][read_posx * SIMD *
                                DATA_WIDTH +: SIMD * DATA_WIDTH] <=
                                inp_data[SIMD*DATA_WIDTH-1:0];
191
192                             // finished this PE's folds?
193                             if (read_posx == fold_qnt_reg - 1) begin
194                                 read_posy_next = read_posy + 1;
195                                 read_posx_next = '0;
196                             end
197                             else begin
198                                 read_posx_next = read_posx + 1;
199                                 read_posy_next = read_posy;
200                             end
201                         end
202                     end
203                     // scale phase: single beat after all PEs,
204                     // using signed so compiler doesnt complain
205                     // about signed/unsigned assignments
206                     scales <=
207                         signed'(inp_data[M_INT_PRECISION-1:0]);
208                     write_complete <= '1;
209                     read_posx_next = '0;
210                     read_posy_next = '0;
211                 end
212                 read_posx <= read_posx_next;
213                 read_posy <= read_posy_next;
214             end
215         end
216     end

```

```

214     end
215     //=====WRITING STATE=====
216     //=====READY STATE=====
217     ready: begin // just set to zero buffers, flags and counters
218         output_ready <= '0;
219         ready_to_receive <= '1;
220         computing_complete <= '0;
221         weights_ind <= 0;
222         PE_ind <= 0;
223     end
224     //=====READY STATE=====
225     //=====COMPUTING STATE=====
226     computing: begin
227         //=====ACCUMULATOR=====
228         logic signed [ACC_BIT_WIDTH-1:0] acc_next;
229         //Uses signed to avoid warnings of signed/unsigned
230         attribution
231         acc_next = (weights_ind == 0) ? signed'(0) : out_acc;
232
233         for (shortint unsigned simd_ind = 0; simd_ind < SIMD;
234             simd_ind++) begin
235             acc_next +=
236                 $signed(weights[PE_ind][(weights_ind * SIMD + simd_ind)
237                     * DATA_WIDTH +: DATA_WIDTH]) *
238                 $signed(inp_data_reg[(weights_ind * SIMD + simd_ind) *
239                     DATA_WIDTH +: DATA_WIDTH]);
240         end
241
242         out_acc <= acc_next;
243         //=====ACCUMULATOR=====
244
245         //=====ACTIVATION FUNCTION=====
246         if (weights_ind == fold_qnt_reg - 1) begin
247             logic signed [TEMP_DATA_WIDTH-1:0] temp_next;
248             logic signed [DATA_WIDTH-1:0] result_dw;
249             logic signed [DATA_WIDTH-1:0] out_PE_next [PE-1:0];
250
251             // Calculate math instantly
252             temp_next = ((scales * acc_next) +
253                 PRECISION_CORRECTION) >>> M_INT_PRECISION;
254
255             `ifndef SYNTHESIS
256                 out_temp <= temp_next;
257             `endif
258
259             // Clip values
260             if (temp_next > MAX_VAL)
261                 result_dw = signed'(MAX_VAL[DATA_WIDTH-1:0]);
262             else if (temp_next < 0 && act_fun_reg == 1)
263                 result_dw = '0;
264             else if (temp_next < MIN_VAL)
265                 result_dw = signed'(MIN_VAL[DATA_WIDTH-1:0]);
266             else
267                 result_dw = signed'(temp_next[DATA_WIDTH-1:0]);
268
269             // Create the "next state" of the entire array instantly
270             out_PE_next = out_PE;
271             out_PE_next[PE_ind] = result_dw;
272
273             // Write back to internal registers
274             out_PE <= out_PE_next;
275
276             // Publish to the 1D output bus cleanly
277             if (PE_ind == pe_qnt_reg - 1) begin
278                 // Unconditionally pack the 2D array into the 1D
279                 output bus.
280                 // Because there are no "if" statements checking
281                 variable indexes here,
282                 // this wires the flip-flops directly WITHOUT a
283                 massive MUX!
284                 for (int i = 0; i < PE; i++) begin
285                     // Cast the signed internal math back to raw
286                     unsigned bits for the output port
287                     out[i * DATA_WIDTH +: DATA_WIDTH] <=
288                         unsigned'(out_PE_next[i]);
289                 end
290                 output_ready <= '1;
291                 ready_to_receive <= '1;

```

```

282         computing_complete <= '1;
283     end
284     else begin
285         out <= '0;
286         output_ready <= '0;
287         ready_to_receive <= '0;
288         computing_complete <= '0;
289     end
290 end
291 //=====ACTIVATION FUNCTION=====
292
293 //=====STATE PROGRESSION=====
294 if (PE_ind == pe_qnt_reg - 1 && weights_ind == fold_qnt_reg
    - 1) begin
295     // Outputs handled in Activation block above.
296 end
297 else if (weights_ind == fold_qnt_reg - 1) begin
298     //when weight fold is completed means that one
        output(filter or element) is finished
299     //set weight index to zero and increment PE index
        weights_ind <= 0;
        PE_ind <= PE_ind + 1;
300
301 end
302 else begin
303     //in normal situation just increment the weight index,
        meaning to compute the next weight fold
304     weights_ind <= weights_ind + 1;
305 end
306 //=====STATE PROGRESSION=====
307 end
308 //=====COMPUTING STATE=====
309 default: begin
310     output_ready <= '0;
311     ready_to_receive <= '0;
312 end
313 endcase
314 end
315 end
316 end
317
318 `ifndef SYNTHESIS
319 //=====DEBUG=====
320 always_ff @(posedge clk) begin
321     if (!rst_n) begin
322         check_state <= '0;
323         check_next_state <= '0;
324         check_read_posy <= '0;
325         check_read_posx <= '0;
326         check_write_complete <= '0;
327         check_input <= '0;
328         check_input_part <= '0;
329         check_current_weights <= 'b0;
330         check_output_PE <= 'b0;
331         check_out_acc <= '0;
332         check_out_temp <= '0;
333         check_PE_ind <= 0;
334         check_weight_ind <= 0;
335     end
336     else begin
337         check_state <= state;
338         check_next_state <= next_state;
339         check_weight_ind <= weights_ind;
340         check_PE_ind <= PE_ind;
341         check_input <= inp_data_reg;
342         check_input_part <= inp_data_reg[weights_ind * SIMD * DATA_WIDTH +:
            SIMD * DATA_WIDTH];
343         check_read_posx <= read_posx;
344         check_read_posy <= read_posy;
345         for (int unsigned pe_indx = 0; pe_indx < PE; pe_indx++) begin
346             if (pe_indx < pe_qnt_reg) begin
347                 check_current_weights[pe_indx * SIMD * DATA_WIDTH +: SIMD *
                    DATA_WIDTH] <= weights[pe_indx][weights_ind * SIMD *
                    DATA_WIDTH +: SIMD * DATA_WIDTH];
348             end
349             else begin
350                 check_current_weights[pe_indx * SIMD * DATA_WIDTH +: SIMD *
                    DATA_WIDTH] <= '0;
351             end
352         end
353     end
354 end

```

```

353         // Repack the 2D array back into a 1D vector for the testbench
           debug port
354         for (int i = 0; i < PE; i++) begin
355             check_output_PE[i * DATA_WIDTH +: DATA_WIDTH] <= out_PE[i];
356         end
357         check_computing_complete <= computing_complete;
358         check_write_complete <= write_complete;
359         check_out_acc <= out_acc;
360         check_out_temp <= out_temp;
361     end
362 end
363 //Output weights to the testbench
364 genvar dbg_pe;
365 generate
366     for (dbg_pe = 0; dbg_pe < PE; dbg_pe++) begin : DBG_WEIGHTS
367         assign check_weights[dbg_pe * MAX_INPUT_DIM +: MAX_INPUT_DIM] =
           weights[dbg_pe];
368     end
369 endgenerate
370 //=====DEBUG=====
371 `endif
372
373 endmodule

```

Source Code B.2 – Python Golden Model for generating inputs and expected outputs.

```

1 #####
2 #           Projeto de Formatura I - SCC0670           #
3 #                                                     #
4 #           By: Artur Brenner Weber                   #
5 #           email: arturweber@usp.br                  #
6 #           Last Update: 26/5/2026                   #
7 #                                                     #
8 #           Written based on a Jupyter Notebook originally by #
9 #           Eduardo Sperle Honorato.                  #
10 #####
11
12 import argparse, math, random, os
13 import numpy as np
14 import torch
15 import torch.nn as nn
16 import brevitass.nn as qnn
17 from brevitass.quant import Int8ActPerTensorFixedPoint
18
19 PRECISION_N = 32
20 USE_RELU = True # True for ReLU, False for Identity
21
22 def seed_all(seed: int):
23     random.seed(seed)
24     np.random.seed(seed)
25     torch.manual_seed(seed)
26     torch.cuda.manual_seed_all(seed)
27     torch.backends.cudnn.deterministic = True
28     torch.backends.cudnn.benchmark = False
29
30 class GoldenLinear(nn.Module):
31     def __init__(self, dot_len: int, pe_qnt: int, bit_width: int):
32         super().__init__()
33         self.quant_inp = qnn.QuantIdentity(bit_width=bit_width,
           return_quant_tensor=True)
34         self.fc = qnn.QuantLinear(
35             dot_len, pe_qnt,
36             bias=False,
37             weight_bit_width=bit_width,
38             output_quant=Int8ActPerTensorFixedPoint,
39             return_quant_tensor=True
40         )
41
42     def forward(self, x):
43         x = self.quant_inp(x)
44         x = self.fc(x)
45         return x
46
47 def clip_int8(x: torch.Tensor) -> torch.Tensor:
48     return torch.clamp(x, -128, 127).to(torch.int8)
49
50 def golden_hw_int8(x_int8, w_int8, Mint, use_relu=True):

```

```

51     x = x_int8.to(torch.int32)
52     w = w_int8.to(torch.int32)
53     acc = x @ w.t()
54     acc64 = acc.to(torch.int64)
55     rounding = 1 << (PRECISION_N - 1)
56     y = (acc64 * int(Mint) + rounding) >> PRECISION_N
57     if use_relu:
58         y = torch.clamp(y, 0, 127)
59     else:
60         y = torch.clamp(y, -128, 127)
61     return y.to(torch.int8)
62
63 def pack_to_folds(vec_int8: torch.Tensor, simd: int, fold_qnt: int):
64     D = vec_int8.numel()
65     total = simd * fold_qnt
66     if D > total:
67         raise ValueError(f"D={D} > simd*fold_qnt={total}")
68     padded = torch.zeros(total, dtype=torch.int8)
69     padded[:D] = vec_int8
70     return padded.view(fold_qnt, simd) # (F, SIMD)
71
72 def main():
73     ap = argparse.ArgumentParser()
74     ap.add_argument("--seed", type=int, default=1)
75     ap.add_argument("--bit_width", type=int, default=8)
76     ap.add_argument("--cin", type=int, default=16) # input channels
77     ap.add_argument("--kx", type=int, default=3)
78     ap.add_argument("--ky", type=int, default=3)
79     ap.add_argument("--pe_qnt", type=int, default=16)
80     ap.add_argument("--simd", type=int, default=64)
81     ap.add_argument("--n_inputs", type=int, default=1000)
82     ap.add_argument("--out_dir", default="vectors")
83     args = ap.parse_args()
84
85     seed_all(args.seed)
86
87     dot_len = args.cin * args.kx * args.ky
88     fold_qnt = math.ceil(dot_len / args.simd)
89
90     model = GoldenLinear(dot_len, args.pe_qnt, args.bit_width).eval()
91
92     # Random FP input -> quantized int8 input
93     x_fp = torch.randn(args.n_inputs, dot_len)
94
95     with torch.no_grad():
96         q_in = model.quant_inp(x_fp)
97         s_in = float(model.quant_inp.act_quant.scale().item())
98         x_int8 = clip_int8(torch.round(q_in / s_in))
99
100         w_int8 = clip_int8(model.fc.quant_weight().int())
101         w_scale = model.fc.weight_quant.scale().detach().cpu()
102         s_out = float(model.fc.output_quant.scale().item())
103
104     # Enforce per-tensor weight scale (one scale per layer)
105     if w_scale.numel() != 1:
106         raise RuntimeError(
107             f"Per-channel weight scale detected (numel={w_scale.numel()}). "
108             "RTL uses one scale per layer; configure Brevitas for per-tensor "
109             "weight scale."
110         )
111     s_w = float(w_scale.item())
112
113     M = (s_in * s_w) / s_out
114     Mint = int(round(M * (1 << PRECISION_N)))
115
116     # Golden int8 matching RTL integer requant
117     y_hw = golden_hw_int8(x_int8, w_int8, Mint, use_relu=USE_RELU)
118
119     os.makedirs(args.out_dir, exist_ok=True)
120     weights_path = os.path.join(args.out_dir, "weights.txt")
121     inputs_path = os.path.join(args.out_dir, "inputs.txt")
122     exp_path = os.path.join(args.out_dir, "expected.txt")
123     cfg_path = os.path.join(args.out_dir, "cfg.txt")
124
125     with open(cfg_path, "w") as f:
126         f.write(f"cin {args.cin}\n")
127         f.write(f"kx {args.kx}\n")
128         f.write(f"ky {args.ky}\n")

```

```

128         f.write(f"dot_len {dot_len}\n")
129         f.write(f"simd {args.simd}\n")
130         f.write(f"fold_qnt {fold_qnt}\n")
131         f.write(f"pe_qnt {args.pe_qnt}\n")
132         f.write(f"Mint {Mint}\n")
133         f.write(f"act_fun {1 if USE_RELU else 0}\n")
134
135     # weights.txt format (ONE Mint per layer):
136     # for p in 0..pe_qnt-1:
137     #   for fold in 0..fold_qnt-1: SIMD ints
138     # then final line: Mint
139     with open(weights_path, "w") as f:
140         for p in range(args.pe_qnt):
141             folds = pack_to_folds(w_int8[p], args.simd, fold_qnt)
142             for fold in range(fold_qnt):
143                 f.write(" ".join(str(int(v)) for v in folds[fold].tolist()) +
144                        "\n")
145             f.write(str(int(Mint)) + "\n")
146
147     # inputs.txt: each input is fold_qnt lines of SIMD ints
148     with open(inputs_path, "w") as f:
149         for b in range(args.n_inputs):
150             folds = pack_to_folds(x_int8[b], args.simd, fold_qnt)
151             for fold in range(fold_qnt):
152                 f.write(" ".join(str(int(v)) for v in folds[fold].tolist()) +
153                        "\n")
154
155     # expected.txt: 1 line per input, pe_qnt ints
156     with open(exp_path, "w") as f:
157         for b in range(args.n_inputs):
158             f.write(" ".join(str(int(v)) for v in y_hw[b].tolist()) + "\n")
159
160     print("Generated in:", args.out_dir)
161     print(f" dot_len={dot_len}, fold_qnt={fold_qnt}, pe_qnt={args.pe_qnt},
162           simd={args.simd}")
163     print(f" scales: s_in={s_in}, s_w={s_w}, s_out={s_out}")
164     print(f" M={M}, Mint(Q32)={Mint}")
165     print(f" USE_RELU={USE_RELU}")
166
167 if __name__ == "__main__":
168     main()

```

Source Code B.3 – Testbench with memory simulation and Golden Model inputs and outputs check.

```

1  // =====
2  //                               Projeto de Formatura I - SCC0670
3  //
4  //       Name: Artur Brenner Weber
5  //       email: arturweber@usp.br
6  //       Last Update: 26/5/2026
7  //
8  //       Based on Testbench written by Eduardo Sperle Honorato.
9  // =====
10
11 `timescale 1ns/1ps
12 `include "rtl_cfg.svh"
13
14 module tb_pe;
15
16     // -----
17     // Compile-time parameters (match DUT build)
18     // -----
19     parameter shortint unsigned SIMD          = `SIMD;
20     parameter shortint unsigned PE            = `PE;
21     parameter byte unsigned DATA_WIDTH       = `DATA_WIDTH;
22
23     parameter byte unsigned ACC_BIT_WIDTH     = 32;
24     parameter byte unsigned M_INT_PRECISION   = 32;
25     parameter longint signed PRECISION_CORRECTION = 1 << (M_INT_PRECISION-1);
26     parameter byte unsigned TEMP_DATA_WIDTH   = ACC_BIT_WIDTH +
27         M_INT_PRECISION + 1;
28
29     parameter byte unsigned NUM_ACTS_FUN      = 2;
30     parameter int unsigned ACT_FUN_SEL       = 1;

```

```

31  localparam int unsigned      W_ACT = (NUM_ACTS_FUN <= 1) ? 1 :
    $clog2(NUM_ACTS_FUN);
32
33  parameter shortint           MAX_VAL = (1 <<< (DATA_WIDTH-1)) - 1;
34  parameter shortint           MIN_VAL = -(1 <<< (DATA_WIDTH-1));
35
36  parameter shortint unsigned  MAX_CHANNELS = 'MAX_CHANNELS;
37  parameter shortint unsigned  KERNEL_X = 'KERNEL_X;
38  parameter shortint unsigned  KERNEL_Y = 'KERNEL_Y;
39  parameter int unsigned       MAX_DOT_LANES = MAX_CHANNELS*KERNEL_X*KERNEL_Y;
40  parameter int unsigned       MAX_FOLDS = (MAX_DOT_LANES + SIMD - 1) / SIMD;
41
42  parameter int unsigned       PAD_LANES = MAX_FOLDS * SIMD;
43  parameter int unsigned       MAX_INPUT_DIM = PAD_LANES * DATA_WIDTH;
44  parameter int unsigned       MAX_OUTPUT_DIM = PE * DATA_WIDTH;
45
46  // -----
47  // File handles + paths (module scope)
48  // -----
49  integer weight_file;
50  integer input_file;
51  integer exp_file;
52  int r;
53
54  string weights_path;
55  string inputs_path;
56  string expected_path;
57
58  // -----
59  // Runtime-configurable knobs
60  // -----
61  int CIN      = 16;
62  int KX       = 3;
63  int KY       = 3;
64  int NIN      = 1000;
65  int PEQNT    = 16;
66  string VEC_DIR = ".";
67
68  int DOT_LEN;
69  int FOLD_QNT_INT;
70
71  // -----
72  // Memory-model knobs (absolute time, then converted to cycles using FCLK)
73  // -----
74  real FCLK_HZ    = 1.0e9;    // accelerator frequency (Hz)
75  real MEM_B_GBPS = 30.0;    // bandwidth in GB/s (decimal, 1 GB = 1e9 bytes)
76  real MEM_L_S    = 2.0e-9;  // latency in seconds
77
78  // -----
79  // DUT inputs
80  // -----
81  logic clk;
82  logic rst_n;
83  logic set_cfg_n; // active-low in DUT
84  logic [W_ACT-1:0] act_fun;
85  logic wr_en;
86  logic str_wr;
87  logic inp_rd;
88  logic [MAX_INPUT_DIM-1:0] inp_data;
89
90  logic [$clog2(MAX_FOLDS+1)-1:0] fold_qnt;
91  logic [$clog2(PE+1)-1:0] pe_qnt;
92
93  // DUT outputs
94  logic [MAX_OUTPUT_DIM-1:0] out;
95  logic output_ready;
96  logic ready_to_receive;
97
98  // -----
99  // DEBUG signals
100 // -----
101 int unsigned check_state, check_next_state;
102 int unsigned check_read_posx, check_read_posy;
103 int unsigned check_weight_ind, check_PE_ind;
104 logic check_write_complete, check_computing_complete;
105 logic [PE*MAX_INPUT_DIM-1:0] check_weights;
106 logic [PE*SIMD*DATA_WIDTH-1:0] check_current_weights;
107 logic [PE*DATA_WIDTH-1:0] check_output_PE;

```

```

108 logic [MAX_INPUT_DIM-1:0] check_input;
109 logic [SIMD*DATA_WIDTH-1:0] check_input_part;
110 logic [ACC_BIT_WIDTH-1:0] check_out_acc;
111 logic signed [TEMP_DATA_WIDTH-1:0] check_out_temp;
112
113 // -----
114 // VCD dump for power (will be converted to SAIF via vcd2saif)
115 // -----
116 initial begin
117     $display("[TB] Dumping VCD activity to work/dut.vcd ...");
118     $dumpfile("work/dut.vcd");
119     $dumpvars(0, tb_pe);
120 end
121
122 // -----
123 // Clock generation
124 // -----
125 // IMPORTANT: This TB generates a clock derived from
126 // FCLK_HZ so the cycle counts and the memory-model math
127 // match the simulated clock.
128 //
129 // timescale is 1ns/1ps, so:
130 //   period_ns = 1e9 / FCLK_HZ
131 //   half_period_ns = period_ns / 2
132 // -----
133 real CLK_PERIOD_NS;
134 real CLK_HALF_NS;
135
136 initial begin
137     clk = 0;
138     // default before plusargs are parsed; will be recomputed later too
139     CLK_PERIOD_NS = 1.0e9 / FCLK_HZ;
140     CLK_HALF_NS = CLK_PERIOD_NS / 2.0;
141     forever begin
142         #(CLK_HALF_NS) clk = ~clk;
143     end
144 end
145
146 // -----
147 // Helpers
148 // -----
149 task automatic clear_inp_data;
150     inp_data = '0;
151 endtask
152
153 function automatic int ceil_div_int(int a, int b);
154     return (a + b - 1) / b;
155 endfunction
156
157 // ceil(x) for real -> int
158 function automatic int ceil_real_to_int(real x);
159     int xi;
160     xi = $rtoi(x);
161     if (x > xi) return xi + 1;
162     else return xi;
163 endfunction
164
165 // Convert seconds to cycles with ceil
166 function automatic int sec_to_cycles(real t_s, real f_hz);
167     return ceil_real_to_int(t_s * f_hz);
168 endfunction
169
170 // -----
171 // DUT instance
172 // -----
173 pe #(
174     .DATA_WIDTH(DATA_WIDTH),
175     .MAX_VAL(MAX_VAL),
176     .MIN_VAL(MIN_VAL),
177     .ACC_BIT_WIDTH(ACC_BIT_WIDTH),
178     .M_INT_PRECISION(M_INT_PRECISION),
179     .PRECISION_CORRECTION(PRECISION_CORRECTION),
180     .TEMP_DATA_WIDTH(TEMP_DATA_WIDTH),
181     .NUM_ACTS_FUN(NUM_ACTS_FUN),
182     .SIMD(SIMD),
183     .PE(PE),
184     .MAX_INPUT_DIM(MAX_INPUT_DIM),
185     .MAX_OUTPUT_DIM(MAX_OUTPUT_DIM)

```

```

186 ) dut (
187     .check_state(check_state),
188     .check_next_state(check_next_state),
189     .check_read_posx(check_read_posx),
190     .check_read_posy(check_read_posy),
191     .check_weight_ind(check_weight_ind),
192     .check_PE_ind(check_PE_ind),
193     .check_write_complete(check_write_complete),
194     .check_computing_complete(check_computing_complete),
195     .check_weights(check_weights),
196     .check_current_weights(check_current_weights),
197     .check_output_PE(check_output_PE),
198     .check_input(check_input),
199     .check_input_part(check_input_part),
200     .check_out_acc(check_out_acc),
201     .check_out_temp(check_out_temp),
202     .rst_n(rst_n),
203     .clk(clk),
204     .set_cfg_n(set_cfg_n),
205     .act_fun(act_fun),
206     .wr_en(wr_en),
207     .str_wr(str_wr),
208     .inp_rd(inp_rd),
209     .inp_data(inp_data),
210     .fold_qnt(fold_qnt),
211     .pe_qnt(pe_qnt),
212     .out(out),
213     .output_ready(output_ready),
214     .ready_to_receive(ready_to_receive)
215 );
216
217 // -----
218 // Load weights (no memory model here)
219 // -----
220 task automatic load_weights_from_file(string path);
221     int simd_vals[SIMD];
222     int scale_val;
223 begin
224     weight_file = $fopen(path, "r");
225     if (weight_file == 0) begin
226         $display("ERROR: Could not open %s", path);
227         $fatal(1);
228     end
229
230     $display("\n[TB] Loading weights from %s", path);
231
232     // Keep wr_en asserted to stream at 1 beat per cycle
233     wr_en = 1;
234
235     // Stream folds for all PEs
236     for (int p = 0; p < pe_qnt; p++) begin
237         for (int fold = 0; fold < fold_qnt; fold++) begin
238
239             for (int s = 0; s < SIMD; s++) begin
240                 r = $fscanf(weight_file, "%d", simd_vals[s]);
241                 if (r != 1) begin
242                     $display("ERROR: weights file ended early p=%0d fold=%0d s=%0d", p,
243                             fold, s);
244                     $fatal(1);
245                 end
246             end
247
248             inp_data = '0;
249             for (int s = 0; s < SIMD; s++) begin
250                 inp_data[s*DATA_WIDTH +: DATA_WIDTH] = simd_vals[s][DATA_WIDTH-1:0];
251             end
252
253             @(posedge clk);
254         end
255     end
256
257     // Final Mint (one per layer)
258     r = $fscanf(weight_file, "%d", scale_val);
259     if (r != 1) begin
260         $display("ERROR: missing final Mint line (one per layer)");
261         $fatal(1);
262     end

```

```

263     $display("[TB] Loaded layer scale(Mint) = %0d (0x%0h)", scale_val,
264             scale_val);
265
266     // Make the scale beat explicit and synchronous
267     inp_data = '0;
268     inp_data[M_INT_PRECISION-1:0] = scale_val[M_INT_PRECISION-1:0];
269
270     // Keep wr_en high through the sampling edge
271     wr_en = 1;
272     @(posedge clk); // DUT samples scale here
273
274     // Now drop wr_en and clear inp_data
275     wr_en = 0;
276     inp_data = '0;
277
278     $fclose(weight_file);
279     $display("[TB] Weights loaded.\n");
280 end
281 endtask
282
283 // -----
284 // Run + check + memory model
285 // -----
286 task automatic run_and_check(string in_path, string exp_path, int nin);
287     int simd_vals [SIMD];
288     int exp_vals  [PE];
289     logic signed [DATA_WIDTH-1:0] dut_val;
290     logic [MAX_OUTPUT_DIM-1:0] out_sample;
291
292     // Timing counters
293     longint unsigned cycle_ctr;
294     longint unsigned start_cycle;
295     longint unsigned done_cycle;
296
297     // Memory model per input
298     int s_in_bytes;
299     real Tmem_s;
300     int Tmem_c;
301     int Tidle_c;
302     int Tcomp_c;
303     int Tcomp_prev_c;
304     int Tmem_BW_c;
305     int Tmem_L_c;
306
307     real ops_per_input;
308     real macs_per_input;
309     real gops_t;
310     real gmacs_t;
311     real oi_gop;
312     real oi_gmac;
313     real Tmem_BW_s;
314     real Tmem_L_s;
315
316 begin
317     Tcomp_prev_c = 0;
318
319     input_file = $fopen(in_path, "r");
320     if (input_file == 0) begin
321         $display("ERROR: Could not open %s", in_path);
322         $fatal(1);
323     end
324
325     exp_file = $fopen(exp_path, "r");
326     if (exp_file == 0) begin
327         $display("ERROR: Could not open %s", exp_path);
328         $fatal(1);
329     end
330
331     cycle_ctr = 0;
332
333     // Cycle counter (local to this task)
334     fork
335     begin
336         forever begin
337             @(posedge clk);
338             cycle_ctr++;
339         end
340     end

```

```

340 join_none
341
342 $display("[TB] Running %0d inputs from %s, checking vs %s", nin, in_path,
exp_path);
343 $display("[TB][MEM] FCLK_HZ=%g MEM_L_S=%g MEM_B_GBPS=%g", FCLK_HZ, MEM_L_S,
MEM_B_GBPS);
344
345 // bytes transferred per input (includes padding)
346 s_in_bytes = FOLD_QNT_INT * SIMD * (DATA_WIDTH/8);
347
348 for (int t = 0; t < nin; t++) begin
349
350     // Pack full inp_data across folds *before* waiting
351     inp_data = '0;
352     for (int fold = 0; fold < fold_qnt; fold++) begin
353         for (int s = 0; s < SIMD; s++) begin
354             r = $fscanf(input_file, "%d", simd_vals[s]);
355             if (r != 1) begin
356                 $display("ERROR: inputs file ended early t=%0d fold=%0d s=%0d", t,
fold, s);
357                 $fatal(1);
358             end
359         end
360         for (int s = 0; s < SIMD; s++) begin
361             int lane = fold*SIMD + s;
362             inp_data[lane*DATA_WIDTH +: DATA_WIDTH] =
simd_vals[s][DATA_WIDTH-1:0];
363         end
364     end
365
366     // --- Memory model (serialized): wait Tmem_c cycles before issuing
inp_rd ---
367     // Tmem_s = L + s/B
368     Tmem_s = MEM_L_S + (real'(s_in_bytes) / (MEM_B_GBPS * 1.0e9));
369     Tmem_c = sec_to_cycles(Tmem_s, FCLK_HZ);
370     Tmem_L_s = MEM_L_S;
371     Tmem_L_c = sec_to_cycles(Tmem_L_s, FCLK_HZ);
372     Tmem_BW_s = real'(s_in_bytes) / (MEM_B_GBPS * 1.0e9);
373     Tmem_BW_c = sec_to_cycles(Tmem_BW_s, FCLK_HZ);
374
375     // wait until DUT ready
376     do @(posedge clk); while (ready_to_receive != 1);
377
378     // Inject idle cycles to emulate memory fetch time (overlap version)
379     Tidle_c = (Tmem_c > Tcomp_prev_c) ? (Tmem_c - Tcomp_prev_c) : 0;
380     repeat (Tidle_c) @(posedge clk);
381
382     // Issue input: make it a clean 1-cycle pulse aligned to posedge
383     inp_rd <= 1'b1;
384     start_cycle = cycle_ctr;
385     @(posedge clk);
386     inp_rd <= 1'b0;
387
388     // Wait for a clock edge where output_ready is high
389     do @(posedge clk); while (output_ready != 1);
390     out_sample = out;
391     done_cycle = cycle_ctr;
392
393     Tcomp_c = int'(done_cycle - start_cycle);
394     Tcomp_prev_c = Tcomp_c;
395
396     // Calculate instantaneous performance for this specific input
397     // Ops = PEs * (2 * SIMD * fold_qnt) | MACs = PEs * SIMD * fold_qnt
398     ops_per_input = real'(pe_qnt) * ((2.0 * real'(SIMD) * real'(fold_qnt)) +
3.0); // +3 for scaling
399     macs_per_input = real'(pe_qnt) * real'(SIMD) * real'(fold_qnt);
400
401     // GOPS = (Ops / 1e9) / (Cycles_this_input / FCLK_HZ)
402     gops_t = (ops_per_input / 1.0e9) / (real'(Tidle_c + Tcomp_c) / FCLK_HZ);
403     gmacs_t = (macs_per_input / 1.0e9) / (real'(Tidle_c + Tcomp_c) / FCLK_HZ);
404
405     // Calculates Operational Intensity
406     oi_gop = ops_per_input / real'(s_in_bytes);
407     oi_gmac = macs_per_input / real'(s_in_bytes);
408
409     $display("[TB][t=%0d] Tbw_s= %gs Tl_s= %gs Tbw_c= %0d Tl_c= %0d Tmem=%0d
Tidle=%0d Tcomp=%0d | %g GOP/s | %g GMAC/s | %g OP/Byte | %g
MAC/Byte",

```

```

410         t, Tmem_BW_s, Tmem_L_s, Tmem_BW_c, Tmem_L_c, Tmem_c, Tidle_c,
           Tcomp_c, gops_t, gmacs_t, oi_gop, oi_gmac);
411
412     for (int p = 0; p < pe_qnt; p++) begin
413         r = $fscanf(exp_file, "%d", exp_vals[p]);
414         if (r != 1) begin
415             $display("ERROR: expected file ended early t=%0d p=%0d", t, p);
416             $fatal(1);
417         end
418     end
419
420     for (int p = 0; p < pe_qnt; p++) begin
421         // Slice the specific PE's result out of the 1D bus
422         dut_val = out_sample[p*DATA_WIDTH +: DATA_WIDTH];
423
424         if ($signed(dut_val) != exp_vals[p]) begin
425             $display("\n==== MISMATCH ====");
426             $display("t=%0d p=%0d", t, p);
427             $display("DUT out = %0d (0x%0h)", $signed(dut_val), dut_val);
428             $display("EXP out = %0d", exp_vals[p]);
429             $display("Raw out bus = 0x%0h", out); // We can print the raw bus
           again!
430             $display("cfg: CIN=%0d KX=%0d KY=%0d DOT_LEN=%0d SIMD=%0d
           fold_qnt=%0d pe_qnt=%0d act_fun=%0d",
           CIN, KX, KY, DOT_LEN, SIMD, fold_qnt, pe_qnt, act_fun);
431             $display("state=%0d weight_ind=%0d PE_ind=%0d out_acc=%0d
           out_temp=%0d",
           check_state, check_weight_ind, check_PE_ind,
           $signed(check_out_acc), $signed(check_out_temp));
432             $display("=====\n");
433             $fatal(1);
434         end
435     end
436
437 end
438
439 end
440
441 $fclose(input_file);
442 $fclose(exp_file);
443 $display("\n[TB] ALL TESTS PASSED.\n");
444 end
445 endtask
446
447 // -----
448 // Main
449 // -----
450 initial begin
451     if (!$value$plusargs("CIN=%d", CIN)) CIN = 16;
452     if (!$value$plusargs("KX=%d", KX)) KX = 3;
453     if (!$value$plusargs("KY=%d", KY)) KY = 3;
454     if (!$value$plusargs("NIN=%d", NIN)) NIN = 10;
455     if (!$value$plusargs("PEQNT=%d", PEQNT)) PEQNT = 6;
456     void'($value$plusargs("VEC_DIR=%s", VEC_DIR));
457
458     // Memory-model plusargs (all optional)
459     void'($value$plusargs("FCLK_HZ=%f", FCLK_HZ));
460     void'($value$plusargs("MEM_B_GBPS=%f", MEM_B_GBPS));
461     void'($value$plusargs("MEM_L_S=%f", MEM_L_S));
462
463     // Recompute sim clock after FCLK plusarg is known
464     CLK_PERIOD_NS = 1.0e9 / FCLK_HZ;
465     CLK_HALF_NS = CLK_PERIOD_NS / 2.0;
466
467     DOT_LEN = CIN * KX * KY;
468     FOLD_QNT_INT = ceil_div_int(DOT_LEN, SIMD);
469
470     // Initial stable state (Time 0)
471     fold_qnt = FOLD_QNT_INT[$bits(fold_qnt)-1:0];
472     pe_qnt = PEQNT[$bits(pe_qnt)-1:0];
473     act_fun = ACT_FUN_SEL[W_ACT-1:0];
474
475     wr_en = 0;
476     str_wr = 0;
477     inp_rd = 0;
478     set_cfg_n = 1; // Keep config disabled initially
479     rst_n = 0; // Assert reset initially
480     clear_inp_data();
481
482     $display("[TB] CIN=%0d KX=%0d KY=%0d DOT_LEN=%0d SIMD=%0d => fold_qnt=%0d;

```

```

483         pe_qnt=%0d; NIN=%0d; VEC_DIR=%s; ACT_FUN_SEL=%0d",
           CIN, KX, KY, DOT_LEN, SIMD, FOLD_QNT_INT, PEQNT, NIN, VEC_DIR,
           ACT_FUN_SEL);
484
485         // Hold reset for a few cycles to clear all DUT registers
486         repeat (2) @(posedge clk);
487         rst_n = 1; // Release reset
488
489         // Give it one cycle of breathing room after reset
490         @(posedge clk);
491
492         // Strobe the Config (Inputs have been stable for 3 cycles now)
493         set_cfg_n = 0;
494         @(posedge clk); // DUT cleanly samples fold_qnt and pe_qnt right here
495
496         set_cfg_n = 1; // Lock it in
497         @(posedge clk);
498
499         // Force DUT into writing state
500         str_wr <= 1;
501         wait (check_state == 1); // Wait for FSM to enter 'writing'
502         @(posedge clk);
503
504         weights_path = {VEC_DIR, "/weights.txt"};
505         inputs_path  = {VEC_DIR, "/inputs.txt"};
506         expected_path = {VEC_DIR, "/expected.txt"};
507
508         load_weights_from_file(weights_path);
509
510         // Wait until DUT says weights are complete before dropping str_wr
511         wait (check_write_complete == 1);
512         @(posedge clk);
513         str_wr <= 0;
514
515         // Optional but safe: wait for DUT to transition to 'ready' state
516         wait (check_state == 2);
517         @(posedge clk);
518
519         run_and_check(inputs_path, expected_path, NIN);
520
521     $finish;
522 end
523
524 endmodule

```

Source Code B.4 – Synopsys Design Compiler synthesis script.

```

1 #####
2 #                               Projeto de Formatura I - SCC0670                               #
3 #                               #                                                                    #
4 #                               By: Artur Brenner Weber                                             #
5 #                               email: arturweber@usp.br                                           #
6 #                               Last Update: 26/5/2026                                             #
7 #                               #                                                                    #
8 #####
9
10 # =====
11 # SYNOPSIS GARBAGE REDIRECTION
12 # =====
13 # Redirect .mr, .pvl, and .syn files (Presto intermediate files)
14 define_design_lib WORK -path ./work
15
16 # Redirect the Setup Verification File (default.svf)
17 set_svf ./work/pe.svf
18
19 # Redirect the alib-52 folder (Advanced Library Characterization cache)
20 set alib_library_analysis_path ./work
21
22 # Redirect the command.log
23 set sh_command_log_file ./work/command.log
24 # =====
25
26 set TOP pe
27
28 set LIB_DB $env(SAED14_LIB_DB)
29 set CLK_PERIOD_NS $env(CLK_PERIOD_NS)
30

```

```

31 # Make sure DC can see the generated header and local RTL
32 set search_path [list ../work]
33 set target_library [list $LIB_DB]
34 set link_library "*" $target_library
35
36 # Read RTL
37 analyze -format sverilog [list pe_artur.sv]
38 elaborate $TOP
39 current_design $TOP
40 link
41
42 # =====
43 # VIRTUAL HIERARCHY GROUPING
44 # =====
45 # Group the Weight Registers
46 group -design_name VIRTUAL_WEIGHTS -cell_name BLK_WEIGHTS [get_cells
    "weights_reg*"]
47
48 # Group the Input Data Registers
49 group -design_name VIRTUAL_INPUTS -cell_name BLK_INPUTS [get_cells
    "inp_data_reg*"]
50
51 # Group the Accumulators
52 group -design_name VIRTUAL_ACCUM -cell_name BLK_ACCUM [get_cells "out_acc_reg*"]
53
54 # Group the Output Registers
55 group -design_name VIRTUAL_OUTPUT -cell_name BLK_OUTPUT [get_cells
    "out_PE_reg*"]
56 # =====
57
58 # Check for structural RTL issues
59 check_design > $env(OUT_DIR)/check_design_$env(BASE_ID).rpt
60
61 # Clock constraint (starting point)
62 create_clock -name core_clk -period $CLK_PERIOD_NS [get_ports clk]
63 set_clock_uncertainty 0.05 [get_clocks core_clk]
64
65 # Simple IO constraints (keeps DC from assuming infinite IO time)
66 set_input_delay 0.1 -clock core_clk [remove_from_collection [all_inputs]
    [get_ports clk]]
67 set_output_delay 0.1 -clock core_clk [all_outputs]
68 set_load 0.05 [all_outputs]
69
70 set_fix_multiple_port_nets -all -buffer_constants
71
72 compile_ultra
73
74 file mkdir $env(OUT_DIR)
75 file mkdir work/netlist
76
77 report_qor > $env(OUT_DIR)/qor_$env(BASE_ID).rpt
78 report_area -hierarchy > $env(OUT_DIR)/area_hier_$env(BASE_ID).rpt
79 report_timing -max_paths 10 -delay_type max >
    $env(OUT_DIR)/timing_max_$env(BASE_ID).rpt
80 report_power -hierarchy > $env(OUT_DIR)/power_est_hier_$env(BASE_ID).rpt
81
82 write -format verilog -hierarchy -output work/netlist/pe_mapped.v
83 write_sdc work/netlist/pe_mapped.sdc
84
85 quit

```

Source Code B.5 – Synopsys Primetime Static Timing Analysis script.

```

1 #####
2 #                               #
3 #                               #
4 #   By: Artur Brenner Weber     #
5 #   email: arturweber@usp.br    #
6 #   Last Update: 26/5/2026      #
7 #                               #
8 #####
9
10 # Redirect the PrimeTime command.log
11 set sh_command_log_file ../work/pt_command.log
12
13 set TOP pe
14 set LIB_DB $env(SAED14_LIB_DB)

```

```

15
16 set search_path [list ../work ../work/netlist]
17 set target_library [list $LIB_DB]
18 set link_library "*" $target_library
19
20 set power_enable_analysis true
21
22 read_verilog work/netlist/pe_mapped.v
23 current_design $TOP
24 link_design $TOP
25
26 # Load initial constraints from Design Compiler
27 read_sdc work/netlist/pe_mapped.sdc
28 file mkdir $env(OUT_DIR)
29
30 # =====
31 # PASS 1: POWER AT TARGET FREQUENCY
32 # =====
33 puts "--- PASS 1: Analyzing at Target Frequency ($env(FCLK_HZ) Hz) ---"
34
35 update_timing
36 set_switching_activity -static_probability 0.5 -toggle_rate 0.1 -base_clock
   core_clk [all_inputs]
37 update_power
38
39 report_power -hierarchy > $env(OUT_DIR)/power_target_$env(BASE_ID).rpt
40 report_timing -max_paths 5 > $env(OUT_DIR)/timing_target_$env(BASE_ID).rpt
41
42 # Critical path report
43 report_timing -max_paths 1 -delay_type max -input_pins -transition_time
   -capacitance -nets \
44   > $env(OUT_DIR)/critical_path_target_$env(BASE_ID).rpt
45
46 report_qor > $env(OUT_DIR)/qor_target_$env(BASE_ID).rpt
47
48 # =====
49 # PASS 2: POWER AT ACTUAL FMAX (Silicon Limit)
50 # =====
51 puts "--- PASS 2: Analyzing at Maximum Reachable Frequency (Fmax) ---"
52
53 # Extract the Worst Negative Slack (WNS) from the Target Run
54 set wns 0.0
55 set timing_paths [get_timing_paths -max_paths 1 -delay_type max]
56 if {[sizeof_collection $timing_paths] > 0} {
57     set wns [get_attribute $timing_paths slack]
58 }
59
60 # Calculate the real minimum clock period
61 set target_period [expr 1000000000.0 / $env(FCLK_HZ)]
62 set real_period [expr {$target_period - $wns}]
63
64 # Overwrite the clock with the REAL period for Fmax analysis
65 create_clock -name core_clk -period $real_period [get_ports clk]
66 update_timing
67
68 # Update power for the new, higher frequency
69 update_power
70
71 report_power -hierarchy > $env(OUT_DIR)/power_fmax_$env(BASE_ID).rpt
72 report_timing -max_paths 5 > $env(OUT_DIR)/timing_fmax_$env(BASE_ID).rpt
73
74 # Critical path report at max frequency
75 report_timing -max_paths 1 -delay_type max -input_pins -transition_time
   -capacitance -nets \
76   > $env(OUT_DIR)/critical_path_fmax_$env(BASE_ID).rpt
77
78 report_qor > $env(OUT_DIR)/qor_fmax_$env(BASE_ID).rpt
79
80 set fmax_hz [expr {0.99/($real_period*1e-9)}]
81
82 puts "-----"
83 puts "SUMMARY FOR BASE_ID: $env(BASE_ID)"
84 puts "Target Period : $target_period ns"
85 puts "Worst Slack    : $wns ns"
86 puts "Real Period    : $real_period ns"
87 puts "PT_FMAX_HZ     : $fmax_hz"
88 puts "-----"
89

```

90 | quit
